THOMAS MORE

Artificial Intelligence

Van schaakcomputer tot taalmodel

- **Opleiding:** Bedrijfsmanagement
- **Auteur:** Elke Boonen
- **Docent:**Elke Boonen

9 903340 330978

Academiejaar 2026-2027

Artificial Intelligence

van schaakcomputer tot taalmodel

YP0827

Applied Data Intelligence — Thomas More Mechelen

The question of whether a computer can think is no more interesting than the question of whether a submarine can swim.

— Edsger W. Dijkstra

Elke Boonen

Inhoudstafel

DEEL I: Slimme Regels

1 Wat is AI? 6
1.1 Wat betekent het om intelligent te zijn? 7
1.2 Waarom data de brandstof is van New AI 10
1.3 Randomness - het onderschatte fundament 11
1.4 De wiskundige gereedschapskist van AI 12
1.5 Denken & doen 18
2 Minimax 19
2.1 De probleemruimte als boom 19
2.2 MAX, MIN en de nulsomlogica 21
2.3 Recursie en backtracking 22
2.4 Toepassingen voorbij het bord 22
2.5 De rekenkost van volledigheid 23
2.6 Denken & doen 25
3 Genetische Algoritmen 26
3.1 Van biologie naar datastructuur 26
3.2 De levenscyclus: vijf stappen 27
3.3 Selectie, crossover en mutatie in pseudocode 30
3.4 Denken & Doen 32

DEEL II: Klassiek Machine Learning

4 Naïve Bayes 35
4.1 Kansen als startpunt 36
4.2 De naïeve aanname die alles verandert 38
4.3 Het nulprobleem en Laplace smoothing 39
4.4 Tellen, combineren, classificeren 40
4.5 Aanbevelingen 41
4.6 Denken & Doen 42
5 Het *k*-Nearest Neighbor 43
5.1 Hoe werkt *k*NN? 43
5.2 Hoe meet je gelijkenis? 44
5.3 Het belang van normalisatie 46
5.4 Sorteren en stemmen 46
5.5 Aanbevelingen 47
5.6 Denken & doen 48
6 Decision Trees 49
6.1 Hoe een Decision Tree werkt 50
6.2 Recursief splitsen 52
6.3 Denken & Doen 53

7 Logistic Regression 54
7.1 De sigmoid-functie als vertaler 55
7.2 De beslissingsgrens 56
7.3 Voorspellen en trainen 56
7.4 Aanbevelingen 57
7.5 Denken & Doen 58

DEEL III: Aanbevelingen & Neurale Netwerken

8 Recommandations 60
8.1 De brandstof: expliciete en impliciete data 60
8.2 Hoe filtert een systeem? Drie basisstrategieën 61
8.3 Architecturale keuzes: hoe schaal je CF? 62
8.4 De ratingmatrix en het sparsity-probleem 63
8.5 Denken & Doen 64
9 Matrix Factorization 65
9.1 Wanneer buren te duur worden 65
9.2 Latente factoren — verborgen smaken 66
9.3 Van één grote matrix naar twee kleine 66
9.4 Van volledige matrix naar Funk SVD 67
9.5 Hoe weet je of je Funk SVD goed is? 68
9.6 Leren via gradient descent 69
9.7 Denken & Doen 72
10 Neurale netwerken 73
10.1 Van hersencel naar wiskundig model 73
10.2 Het meerlaags netwerk 75
10.3 Leren van je fouten 76
10.4 Van één neuron naar een netwerk 77
10.5 Wanneer kies je een NN boven klassieke ML? 79
10.6 Denken & Doen 80

DEEL IV: Generatieve AI

11 Large Language Models 82
11.1 Van tellen naar voorspellen 82
11.2 Tokenisatie: tekst in hapklare stukjes 83
11.3 Embeddings: betekenis als richting 84
11.4 Een taalmodel in miniatuur 86
11.5 De Transformer en self-attention 88
11.6 Pre-training en fine-tuning: eerst lezen, dan opvoeden 89
11.7 Aan de knoppen: temperatuur 90
11.8 Denken & Doen 91
12 AI-agents 92
12.1 Van chatbot naar agent 92
12.2 Tools en plugins: handen voor het brein 93
12.3 Geheugen: kort en lang 94
12.4 De agent als vertaallaag 95
12.5 Multi-agent patronen: wanneer één agent niet volstaat 96

12.6 Aan de slag 96
12.7 Denken & Doen 98

DEEL 5: Labo

13 Van model naar cloud 100
13.1 Van script naar service 101
13.2 DevOps: waarom deze stap bestaat 102
13.3 Containerisatie: je applicatie in een doos 102
13.4 CI/CD: automatiseer de weg naar productie 103
13.5 Deployen naar de cloud 104
13.6 Kostenbeheer in de cloud 105
13.7 Hoe je jezelf hierin bijschoolt 105
13.8 Denken & Doen 106

EINDNOTEN

14 Bibliografie 108
14.1 Boeken 108
14.2 Overig materiaal 108
15 Nawoord 111

Slimme Regels

1. Wat is AI?
2. Minimax
3. Genetische Algoritmen

Wat is AI?

CHAPTER 1

Artificiële intelligentie is ondertussen helemaal geen nieuw begrip. We worden ermee rond de oren geslagen, waardoor het soms voelt alsof het een plotse revolutie is, maar de droom om machines te laten redeneren en beslissingen te nemen loopt al eeuwen als een rode draad door de wiskunde en de informatica.

De hype is echter wel reëel en dat is ook begrijpelijk, want vandaag kan vrijwel iedereen een tekst laten schrijven door ChatGPT, een afbeelding genereren met Midjourney, of een vraag stellen aan een slimme assistent op zijn telefoon. AI lijkt toegankelijk, laagdrempelig, bijna speels. Dat is ook zo, toch voor wie het als gebruiker benadert. Maar er is een groot verschil tussen AI gebruiken en AI écht begrijpen, want achter die eenvoudige interface gaat een wereld schuil van wiskunde, algoritmen en ontwerpkeuzes die allesbehalve vanzelfsprekend zijn.

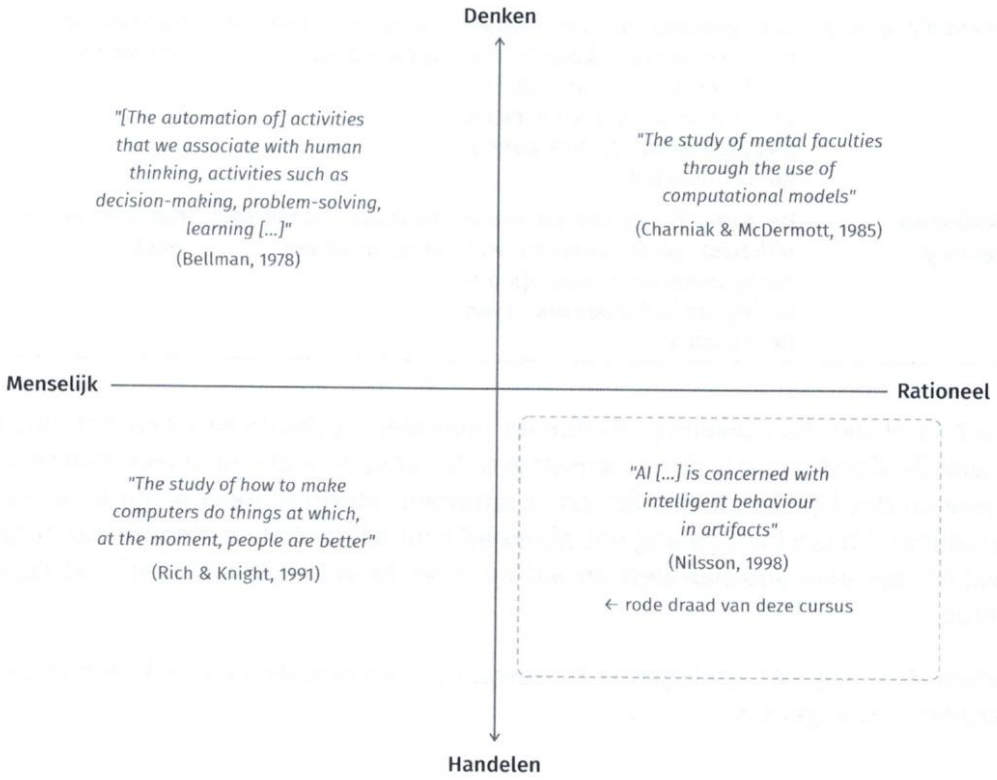
AI is, ondanks de toegankelijke buitenkant, een technisch en wiskundig zwaar vakgebied. Deze cursus neemt je mee achter de schermen, maar voor we daar in duiken starten we met drie fundamentele vragen:

- Wat telt als intelligent gedrag?
- Waarom is data de brandstof van moderne AI?
- Welke rol speelt willekeur in vrijwel elk AI-algoritme?

1.1. *Wat betekent het om intelligent te zijn?*

Voordat je een machine intelligent kunt noemen, moet je weten wat intelligentie betekent. Het antwoord is minder vanzelfsprekend dan het lijkt én en het bepaalt welk type algoritme je nodig hebt.

De klassieke opdeling plaatst AI op twee assen.[†]: **menselijk versus rationeel**, en **denken versus gedrag**. Dat geeft vier kwadranten.



Figuur 2: De vier dimensies van AI — Russell & Norvig

Het kwadrant lees je als volgt. De horizontale as onderscheidt twee visies op wat een AI-systeem moet nastreven: gedrag dat lijkt op dat van een mens, of gedrag dat wiskundig optimaal is. De verticale as onderscheidt twee niveaus waarop dat doel bereikt wordt: door te redeneren, of door te handelen.

Dat geeft vier kwadranten én vier fundamenteel verschillende onderzoekstradities binnen AI. Elk kwadrant stelt een andere vraag, gebruikt andere technieken, en meet succes op een andere manier.

In deze cursus staat het rechterbenedenhoek centraal: rationeel handelen. Niet nabootsen wat een mens zou doen, maar de actie kiezen die wiskundig de beste uitkomst geeft.

[†]Stuart Russell & Peter Norvig, *Artificial Intelligence: A Modern Approach*, 4de editie, Pearson, 2021.

Tabel 1: De vier dimensies van AI - Russell & Norvig

Dimensie	Wat betekent het?	Voorbeelden	De vraag
Menselijk denken	Cognitieve processen nabootsen: hoe redeneren mensen, hoe verwerken ze taal, hoe lossen ze problemen op?	NLP en LLMs	Denkt het zoals een mens?
Rationeel denken	Logisch correcte conclusies trekken uit aannames. Het probleem: menselijk denken zit vol tegenstrijdigheden en logica is daar een slechte match voor.	Expert systems en formele logica	Redeneert het correct?
Menselijk gedrag	Zich gedragen op een manier die ononderscheidbaar is van een mens. Dit is de aanpak van de Turingtest: als je het verschil niet ziet, is het dan niet per definitie intelligent?	Turingtest, chatbots, sociale robots	Handelt het zoals een mens?
Rationeel gedrag	**De actie kiezen die de beste uitkomst geeft, gegeven wat het systeem weet over zijn omgeving. Dit is de rode draad van deze cursus.**	**Minimax, recommenders, AI agents**	**Handelt het optimaal?**

Toch is er een kanttekening. Rationeel handelen optimaliseert een doelfunctie, maar die doelfunctie is door mensen gedefinieerd én mensen maken fouten. Een systeem dat kijktijd maximaliseert is rationeel ten opzichte van zijn doel, maar produceert misschien gedrag dat niemand écht wilde. Dat spanningsveld, tussen wat het systeem optimaliseert en wat we eigenlijk willen, keert later in de cursus terug.

Naast de vraag *wat* intelligentie is, bestaat er een onderscheid in hoe *breed* een systeem intelligent is.

Tabel 2: ANI, AGI en ASI

Type	Definitie	Status	Opmerking
Artificial Narrow Intelligence (ANI)	Eén specifieke taak, beter dan een mens in dat domein.	☑ Nu	Alle systemen in deze cursus
Artificial General Intelligence (AGI)	Brede intelligentie over elk domein, zoals een mens.	⌛ Theorie	Geen bestaand systeem
Artificial Super Intelligence (ASI)	Overtreft menselijke intelligentie in alle domeinen.	? Theorie	Veiligheidsonderzoek

Alle systemen die je in deze cursus bestudeert zijn **ANI**. Een recommendation engine is uitzonderlijk goed in één taak: voorspellen welk item een specifieke gebruiker interessant vindt. Grote taalmodellen wekken soms de indruk van AGI, maar een LLM heeft geen begrip, geen geheugen over sessies heen, en geen doelgericht handelen zonder expliciete instructie. Het is een capabele ANI - niet meer, niet minder.

Over al die dimensies heen is er één concept dat de hele cursus structureert: de **intelligente agent**.

Definition 1.1. *Een **intelligente agent** is een systeem dat zijn omgeving waarneemt via sensoren, op basis van die waarneming een beslissing neemt, en via actuatoren acties onderneemt die de omgeving beïnvloeden.*

In de praktijk vertaalt dat naar de **Sense → Plan → Act → Learn**-lus. Deze lus dateert van de eerste robotica-onderzoeken in de jaren zestig. Wat écht nieuw is, is de Learn-component: moderne agents passen hun gedrag aan op basis van feedback.

Tabel 3: Sense → Plan → Act → Learn: van Old naar New AI

Component	Klassieke agent (Old AI)	Moderne AI agent (New AI)
Sense	Bordpositie in schaak	Gebruikersvraag in tekst, klikgedrag
Plan	Minimax boom doorzoeken	LLM beslist welke tool te gebruiken
Act	Zet uitvoeren op het bord	Recommendation engine aanroepen
Learn	✗ Deep Blue leerde niet	☑ Model hertraint op nieuwe data

Old AI vs. New AI

Old AI: kennis is expliciet geprogrammeerd door een mens. Het systeem past regels toe maar leert niet.

New AI: kennis wordt geëxtraheerd uit data door het systeem zelf. Het algoritme leert patronen zonder dat een mens die patronen expliciet benoemt.

⚠ Old AI is niet verouderd. Minimax en genetische algoritmen zijn vandaag nog steeds relevant, bijvoorbeeld voor problemen waar de spelregels bekend zijn en data schaars is.

In 1997 versloeg Deep Blue van IBM wereldkampioen Garry Kasparov. Deep Blue evalueerde tot 200 miljoen schaakposities per seconde via een handgeprogrammeerd zoekalgoritme. Indrukwekkend, maar ook de grens van de aanpak. Voor elk nieuw domein moest een expert zijn kennis handmatig vertalen naar regels. Een spamfilter? Duizenden handgeschreven condities. Elke nieuwe spamtactiek? Nieuwe regels. Het schaalprobleem was een probleem, want de wereld is te complex voor een handgemaakte set van regels.

"AI" is een paraplubegrip. De lagen kennen helpt, niet om uit het hoofd te leren, maar omdat elke laag andere aannames en trade-offs heeft.

Tabel 4: De gelaagde structuur van AI — AI ⊃ ML ⊃ NN ⊃ DL

Laag	Wat houdt het in?
Artificial Intelligence	Elk systeem dat taken uitvoert waarvoor normaal menselijke intelligentie vereist is. Inclusief rule-based systemen.
Machine Learning	Algoritmen die leren van data. Ze optimaliseren een doelfunctie op basis van voorbeelden, zonder expliciete regels.
Neural Networks	Een specifieke klasse van ML, geïnspireerd op de structuur van het menselijk brein. Geschikt voor complexe, niet-lineaire patronen.
Deep Learning	Neurale netwerken met meerdere lagen. Vereist grote hoeveelheden data maar levert state-of-the-art resultaten in beeld, tekst en geluid.

1.2. Waarom data de brandstof is van New AI

Het kernverschil tussen Old en New AI is niet rekenkracht, het is de rol van data. In Old AI is data input voor een vooraf bepaald algoritme. In New AI is data de grondstof waaruit het algoritme zelf zijn kennis destilleert. Dat heeft drie concrete gevolgen die je doorheen de cursus terugziet:

Data als surrogaat voor domeinkennis. Een spamfilter op basis van Naïve Bayes heeft geen expert nodig die uitlegt wat spam is. Hij leert het patroon zelf uit gelabelde e-mails. Een recommendation engine heeft geen redacteur nodig die beschrijft wat gebruikers leuk vinden. Hij leest het af uit klikgedrag en beoordelingen.

Data als maat voor onzekerheid. Veel AI-systemen geven niet alleen een uitkomst, maar ook een zekerheidsgraad. Naïve Bayes: "dit bericht is voor 94% spam." Matrix factorization: "deze gebruiker zou dit item waarschijnlijk 4,2 op 5 beoordelen." Die zekerheidsgraad is enkel mogelijk als het model genoeg data heeft gezien.

Data als bron van generalisatie. Een model dat enkel zijn trainingsdata reproduceert is nutteloos. Het doel is een model dat *nieuwe*, ongeziene voorbeelden correct behandelt. Dat vermogen om te generaliseren komt volledig uit de kwaliteit en hoeveelheid trainingsdata.

> ⚠ **Meer data is niet altijd beter**
>
> Data heeft kwantiteit én kwaliteit. Een recommendation engine getraind enkel op data van mensen die actief reviews schrijven, leert voornamelijk de voorkeuren van die specifieke groep, niet die van de gemiddelde gebruiker. Het algoritme leert de patronen die in de data zitten, inclusief de fouten en vooroordelen.

1.3. Randomness - het onderschatte fundament

Randomness lijkt op het eerste gezicht het tegenovergestelde van wat een precies, berekenbaar systeem nodig heeft, maar toch is willekeur ingebakken in vrijwel elk modern AI-algoritme.

Laat ons dan eens verder bekijken. Ben jij willekeurig? Je kan nu opstaan om koffie te halen, niemand heeft je dat opgedragen, er is geen regel die het voorschrijft. Dat voelt als vrije, willekeurige keuze. Maar tegelijk: je sliep gisteren slecht, je hebt twee uur gewerkt, je bloedsuiker zakt. Je brein reageert op een interne toestand die jij niet volledig controleert. Was die keuze dan echt willekeurig of deterministisch (voorspelbaar), maar te complex om te voorspellen? Dit is geen retorische vraag. Wetenschappers zijn het er niet over eens. Wat wel duidelijk is: mensen zijn slechte random generators als het aankomt op bewuste keuzes. Denk aan een getal tussen één en tien. De meeste mensen zeggen zeven: consistent, voorspelbaar, massaal. Mentalisten bouwen daar hele shows op. We denken willekeurig te kiezen, maar volgen onbewust patronen.

Een computer doet hetzelfde, maar alleen eerlijker. Een random generator volgt een vaste formule en geeft dat toe. Dat brengt ons bij de vraag: wat is echte willekeur, en waarom hebben AI-systemen het zo hard nodig?

Wanneer je `new Random().Next()` aanroept, krijg je geen echt willekeurig getal. Je krijgt een **pseudo-random getal**: de uitvoer van een deterministisch algoritme dat willekeur simuleert. Een vaak gebruikt algoritme daarvoor is **Linear Congruential Generator (LCG)**:

$$r_{n+1} = (a \cdot r_n + c) \bmod m$$

waarbij r_0 de beginwaarde of **seed** is én a, c en m constanten. In Microsoft C runtime gebruikt men de volgende waarden: $a = 214.013$, $c = 2.531.011$, $m = 2^{31}$. De volledige reeks $r_1, r_2, r_3, \ldots$ ligt dus wiskundig vast zodra je de seed kent, maw als je de startwaarde kent, ken je alle toekomstige getallen. Elk volgend getal hangt namelijk af van het vorige via een vaste formule, en precies dat maakte John von Neumann in 1951 ongemakkelijk: *Anyone who considers arithmetical methods of producing random digits is, of course, in a state of sin.* Hij bedoelde dit: wiskunde is deterministisch. Exacte inputs leiden altijd tot exacte outputs. Een algoritme dat "toeval" produceert, liegt dus per definitie want het simuleert willekeur, het genereert ze niet. Twee instanties met dezelfde seed produceren altijd identieke reeksen. Dat maakt experimenten reproduceerbaar, maar echte onvoorspelbaarheid is onmogelijk.

De LCG heeft een fundamenteel probleem: opeenvolgende getallen r_n en r_{n+1} zijn gecorreleerd. Een aanvaller die een paar opeenvolgende getallen observeert, kan de rest voorspellen. Voor AI-training is dat doorgaans geen probleem, maar voor een systeem dat in een competitieve omgeving onvoorspelbaar moet zijn, is de LCG onbruikbaar.

Ware willekeur ontstaat niet uit een programma. Echte randomness komt uit fysische processen die fundamenteel onvoorspelbaar zijn. Radio-ontvangers die atmosferische ruis opvangen (zie `https://www.random.org/`), radioactief verval,

thermische ruis in hardware: geen van deze processen volgt een berekenbaar patroon.

> **Deterministisch is niet altijd een nadeel**
>
> Voor AI-training is reproduceerbaarheid cruciaal. Als gewichten willekeurig geïnitialiseerd worden via een pseudo-random generator met vaste seed, kan elke collega exact hetzelfde experiment reproduceren. Dat is waarom machine learning frameworks altijd een `random_seed`-parameter aanbieden.

Randomness is dus geen bijzaak, ze speelt een structurele rol in heel wat algoritmen die we zullen zien. Bij genetische algoritmen zorgt ze voor diversiteit in de initiële populatie en houdt mutatie het zoekproces eerlijk. Bij neurale netwerken voorkomt willekeurige initialisatie van gewichten dat alle neuronen hetzelfde leren. Bij matrix factorization geeft ze het optimalisatieproces een eerlijke start. En bij taalmodellen bepaalt de temperatuurparameter hoeveel willekeur er in de tokenselectie zit, van altijd het meest waarschijnlijke token tot creatieve variatie. Zelfs de splitsing van een dataset in train en test gebeurt willekeurig, want een deterministische splitsing kan de evaluatie vertekenen. De rode draad is in ieder geval dat willekeur geen fout is in het systeem.

1.4. *De wiskundige gereedschapskist van AI*

AI is eigenlijk een samenspel van heel wat disciplines, maar in de kern komt er heel wat wiskunde aan te pas. Je hoeft niet alles volledig uit te werken, maar je moet weten waarom welke tak van de wiskunde welk probleem oplost.

1.4.1. Kansrekening en statistiek

Veel AI-systemen moeten omgaan met onzekerheid. Kansrekening is de wiskundige taal daarvoor, niet om exacte antwoorden te geven, maar om te redeneren over wat waarschijnlijk is gegeven wat je weet.

Een klassiek voorbeeld is de stelling van Bayes, die je toelaat om de kans te berekenen dat een bericht spam is op basis van de woorden die het bevat. Maar hetzelfde principe duikt op telkens een systeem een voorspelling moet maken met onvolledige informatie: een ontbrekende rating inschatten, het meest waarschijnlijke volgende woord kiezen, of beslissen welke actie de beste uitkomst geeft.

> **Waarom kansen en niet zekerheden?**
>
> Een gebruiker die drie actiefilms heeft bekeken, zal waarschijnlijk een vierde leuk vinden, maar dat is niet zeker. Kansrekening geeft het systeem een manier om die onzekerheid te kwantificeren en er toch mee te handelen. Dat is fundamenteel anders dan een rule-based systeem dat enkel ja of nee kent.

1.4.2. Logaritmen

Een logaritme beantwoordt een simpele vraag: hoe vaak moet ik een grondtal met zichzelf vermenigvuldigen om tot een bepaald getal te komen? $\log_{10}(1000) = 3$, want $10^3 = 1000$. Algemeen geldt:

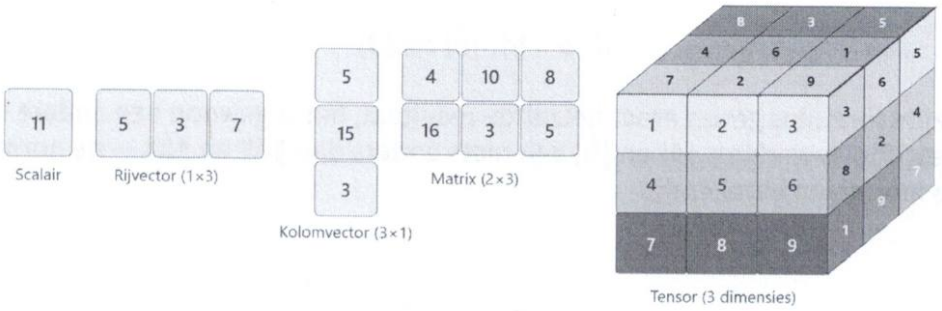
$$\log_b(x) = y \Leftrightarrow b^y = x$$

Grondtal 10 is vertrouwd omdat wij met tien vingers tellen, maar computers tellen in bits: nullen en enen. Daarom duikt in AI en informatica voortdurend $\log_2$ op, de logaritme met grondtal 2.

$\log_2(8) = 3$, want $2^3 = 8$: met 3 bits kan je 8 verschillende waarden coderen (000 tot 111). Wil je 16 waarden coderen, dan heb je $\log_2(16) = 4$ bits nodig. Die relatie, hoeveel bits nodig zijn om N mogelijkheden te onderscheiden, is exact wat $\log_2(N)$ uitdrukt. Handig om te onthouden: $\log_2(1) = 0$, $\log_2(2) = 1$, $\log_2(4) = 2$, $\log_2(8) = 3$.

1.4.3. Lineaire algebra

Lineaire algebra — vectoren, matrices en bewerkingen daarop — is de **taal van data** van AI (*aha, vandaar dus altijd Numpy & Pandas in Python*). Inputs worden voorgesteld als vectoren: een film is een vector van kenmerken, een gebruiker is een vector van ratings, een tekst wordt voorgesteld als een **embedding**: een vector die de betekenis van die tekst vastlegt, zodat je naderhand kan berekenen hoe dicht bijvoorbeeld 'hond' en 'kat' bij elkaar liggen.



Figuur 3: Van scalar tot tensor

1.4.3a. Het dot product: gelijkenis meten

Het dot product van twee vectoren is de meest gebruikte bewerking in deze cursus:

$$\vec{a} \cdot \vec{b} = \sum_{i=1}^{n} a_i \cdot b_i$$

Het meet de gelijkenis tussen twee vectoren door hun paargewijze producten op te tellen. Je komt deze bewerking meermaals tegen, zoals bij cosine similarity, een voorspelde rating, of in een neuraal netwerk.

Stel dat een film wordt voorgesteld als een vector van kenmerken ("actie", "romantiek", "komedie"), en een gebruiker als een vector die aangeeft hoe graag die van elk genre houdt:

$$\vec{f} = (0.9, 0.1, 0.3) \qquad \vec{u} = (0.8, 0.2, 0.5)$$

Dan is het dot product:

$$\vec{f} \cdot \vec{u} = (0.9 \cdot 0.8) + (0.1 \cdot 0.2) + (0.3 \cdot 0.5) = 0.72 + 0.02 + 0.15 = 0.89$$

Hoe hoger die waarde, hoe beter de film en de gebruiker "matchen". Een hoog dot product betekent dat beide vectoren in dezelfde richting wijzen (bv. allebei hoog scoren op actie), terwijl een lage waarde wijst op een mismatch.

1.4.3b. De norm: de lengte van een vector

Er bestaat een tweede manier om het dot product te berekenen: via de hoek θ (*theta*) tussen de twee vectoren. Maar voor je die hoekformule kan gebruiken, heb je nog één bouwsteen nodig: de **norm** van een vector, ofwel zijn lengte. Je berekent die met de stelling van Pythagoras, uitgebreid naar *n* dimensies:

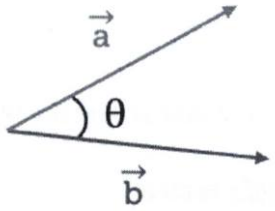
$$\|\vec{a}\| = \sqrt{\sum_{i=1}^{n} a_i^2}$$

Voor $\vec{f} = (0.9, 0.1, 0.3)$ is dat $\|\vec{f}\| = \sqrt{0.9^2 + 0.1^2 + 0.3^2} = \sqrt{0.91} \approx 0.954$.

Met die norm kan je het dot product ook zo schrijven:

$$\vec{a} \cdot \vec{b} = |\vec{a}|\ |\vec{b}| \cos(\theta)$$

Beide formules geven exact hetzelfde resultaat, het is gewoon een andere kijk op dezelfde bewerking. $|\vec{a}|$ en $|\vec{b}|$ zijn niets anders dan $\|\vec{a}\|$ en $\|\vec{b}\|$ — de normen die je hierboven berekende.



$$\vec{a} \cdot \vec{b} = \left|\vec{a}\right| \left|\vec{b}\right| \cos\theta$$

$$\vec{a} \cdot \vec{b} = a_x b_x + a_y b_y$$

Figuur 4: Het dot product
Bron: RMIT University, Learning Lab

Deze hoekversie maakt meteen duidelijk waarom het dot product gelijkenis meet: $\cos(0°) = 1$ (vectoren wijzen exact dezelfde richting uit — ze liggen op mekaar — dus maximale gelijkenis), $\cos(90°) = 0$ (vectoren staan loodrecht op elkaar, geen gelijkenis), en $\cos(180°) = -1$ (vectoren wijzen tegengesteld, dus maximaal verschillend).

1.4.3c. Van vector naar matrix

Een **vector** is een rijtje getallen. Een **matrix** is gewoon een tabel van getallen: rijen en kolommen, meerdere vectoren naast elkaar. Een gebruiker is een vector van ratings; een hele gebruikersgroep is een matrix, met één rij per gebruiker.

Drie bewerkingen lijken oppervlakkig op elkaar, maar doen iets fundamenteel anders. Het is de moeite waard om ze uit elkaar te houden, want je komt ze alle drie tegen.

Scalaire vermenigvuldiging vermenigvuldigt élk element met hetzelfde getal:

$$2 \cdot \begin{pmatrix} 1 & 3 \\ 2 & 4 \end{pmatrix} = \begin{pmatrix} 2 & 6 \\ 4 & 8 \end{pmatrix}$$

Elk element van de matrix wordt gewoon apart met 2 vermenigvuldigd: $2 \cdot 1 = 2$, $2 \cdot 3 = 6$, $2 \cdot 2 = 4$, $2 \cdot 4 = 8$. Geen enkel element "praat" met een ander.

Dot product neemt twee vectoren en versmelt ze tot één enkel getal, exact de bewerking die je hierboven al gebruikte:

$$(1, 3) \cdot (2, 4) = (1 \cdot 2) + (3 \cdot 4) = 2 + 12 = 14$$

Twee vectoren gaan erin, één getal komt eruit. Dat ene getal vat samen hoe sterk beide vectoren "dezelfde richting op wijzen", exact wat je nodig hebt voor cosine similarity of een voorspelde rating.

Matrixvermenigvuldiging is de veralgemening daarvan: in plaats van twee vectoren te versmelten tot één getal, versmelt ze een vector (of een hele matrix) met een matrix tot een nieuwe vector of matrix. Elk element van het resultaat is eigenlijk een apart dot product tussen een rij en een kolom.

$$(1 \;\; 3) \cdot \begin{pmatrix} 2 & 0 \\ 4 & 1 \end{pmatrix} = ((1 \cdot 2 + 3 \cdot 4) \;\; (1 \cdot 0 + 3 \cdot 1)) = (14 \;\; 3)$$

Kijk naar het eerste getal van het resultaat: 14. Dat is het dot product van de rijvector $(1, 3)$ met de eerste kolom van de matrix, $(2, 4)$. Het tweede getal, 3, is het dot product van diezelfde rij met de tweede kolom, $(0, 1)$. Een matrixvermenigvuldiging is dus niets anders dan een hele reeks dot products na elkaar, netjes in een tabel gezet.

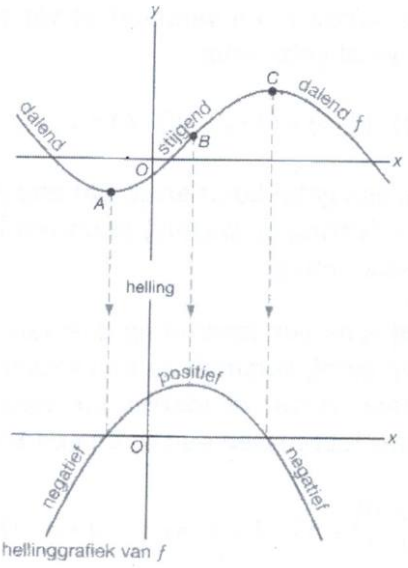
Waarom dit ertoe doet

Bij een neuraal netwerk bereken je niet één neuron per keer, maar een hele laag tegelijk: elk neuron heeft zijn eigen gewichten, maar ze kijken allemaal naar dezelfde input. In plaats van dat dot product keer op keer apart uit te schrijven, doe je het in één keer via $z = \vec{x} \cdot W + \vec{b}$, waarbij W een matrix is met één kolom per neuron. Dezelfde wiskunde, maar dan voor duizenden neuronen tegelijk en dat is waarom een GPU, die gespecialiseerd is in matrixvermenigvuldiging, een neuraal netwerk zo veel sneller traint dan een gewone processor.

1.4.4. Afgeleiden

Een afgeleide meet hoe snel iets verandert. Niet wat de waarde is op een bepaald punt, maar hoe steil de functie op dat punt stijgt of daalt. In de wiskunde noemen we die helling de **richtingscoëfficiënt**.

Voor een rechte lijn $y = ax + b$ is die helling constant: overal dezelfde waarde a. Maar voor een gekromde functie verandert de helling op elk punt. De afgeleide geeft je die helling op precies dat ene punt.



Figuur 5: De helling van een functie.
Bron: Wiskundeleraar

Formeel schrijven we de afgeleide van een functie $f(x)$ als $f'(x)$. Intuïtief lees je dat als: "hoe verandert f als ik x een klein beetje verhoog?"

Afgeleide	Wat betekent het?
$f'(x) > 0$	Functie stijgt op dit punt
$f'(x) < 0$	Functie daalt op dit punt
$f'(x) = 0$	Helling is nul — een top of dal

Waar de afgeleide nul is, verandert de functie niet meer: dat is een lokaal maximum of minimum. Dat is precies waarom de afgeleide zo belangrijk is in AI, het is het gereedschap waarmee een model straks zal zoeken naar het laagst mogelijke punt van een foutfunctie. Je hoeft die afgeleiden zelf niet te berekenen, maar je moet wel begrijpen waarom er afgeleid wordt: om de helling van de foutfunctie te kennen, zodat je weet welke kant op te stappen.

Toch onzeker over de wiskunde?

Merk je dat je vastloopt op een wiskundige notatie of concept? Geen paniek, want je bent niet de enige. Een paar handige hulpmiddelen:

- **Boekje:** Noordhoff *Wiskunde in je pocket* (E. Broekema) is een compact naslagwerk voor basiswiskunde, kansen en formules.
- **Khan Academy**: gratis, Nederlandstalig beschikbaar, legt lineaire algebra en statistiek uit van nul. Zoek op "vectors", "matrices" of "probability" en je krijgt stapsgewijze video's.
- **Wolfram Alpha**: typ gewoon een berekening of formule in, en het toont je de uitwerking stap voor stap. Handig om te controleren of je redenering klopt.
- **Een LLM**: stel gewoon je vraag in gewone taal: "leg me uit wat een dot product is" of "bereken dit stap voor stap". LLMs zijn geduldig, herhalen zo vaak als nodig, en oordelen niet ;)

Wat je *niet* moet doen: de wiskunde overslaan en hopen dat het vanzelf duidelijk wordt. Tien minuten met Khan Academy of een LLM lost meer op dan een uur piekeren.

1.5. *Denken & doen*

1.5.1. Kernconcepten — vat samen in eigen woorden

Probeer elk begrip in één zin uit te leggen zonder terug te bladeren. Check nadien met de theorie.

- De vier kwadranten (mens versus rationeel, denken versus gedrag)
- Rationeel handelen
- ANI versus AGI versus ASI
- De intelligente agent: Sense → Plan → Act → Learn
- Old AI versus New AI
- Seed en reproduceerbaarheid
- Pseudo-random versus echte randomness
- Dot product en vector norm
- Cosine similarity (via de hoek tussen twee vectoren)
- Logaritme, en waarom $\log_2$ overal opduikt in AI
- Afgeleide / richtingscoëfficiënt

1.5.2. Denken

- Twee onderzoekers trainen hetzelfde model. De ene gebruikt een vast startgetal (seed), de andere niet. Wie kan zijn resultaat exact herhalen, en waarom?
- Noem een situatie waarin je juist géén vaste seed wil gebruiken. Waarom is toeval daar net gewenst?
- Waarom is een generator met een groter bereik aan mogelijke waarden moeilijker te "raden" — bijvoorbeeld in cryptografie?

1.5.3. Doen — zelf een LCG doorrekenen

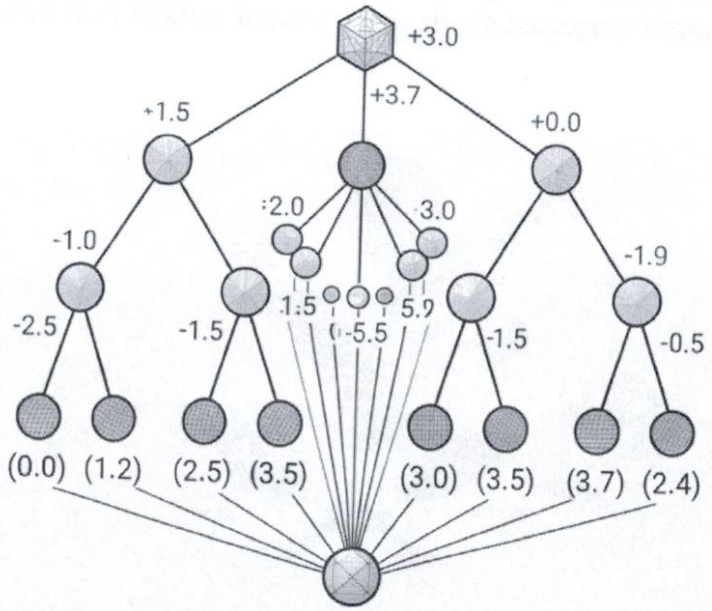
Een **Linear Congruential Generator** berekent elke volgende waarde uit de vorige:

```
r0 = 0

functie volgende(r):
  return (5 * r + 1) mod 8
```

- Bereken r_1 tot en met r_8 met de hand. Na hoeveel stappen herhaalt de reeks zich?
- Herhaal dezelfde berekening met $a = 2$, $c = 1$, $m = 8$, $r_0 = 0$. Wat stel je vast, en waarom is dit een slechte generator?
- Verklaar in één zin waarom priemgetallen als modulus vaak een langere periode opleveren dan machten van 2.

Minimax

CHAPTER 2



Stel je voor dat je schaak speelt. Elke zet die jij doet, roept een tegenzet op. Elke tegenzet beperkt jouw volgende opties. Je speelt niet zomaar tegen een bord, je speelt tegen een tegenstander die actief probeert jouw plannen te dwarsbomen. Precies dat scenario is het vertrekpunt van het **minimax-algoritme**.

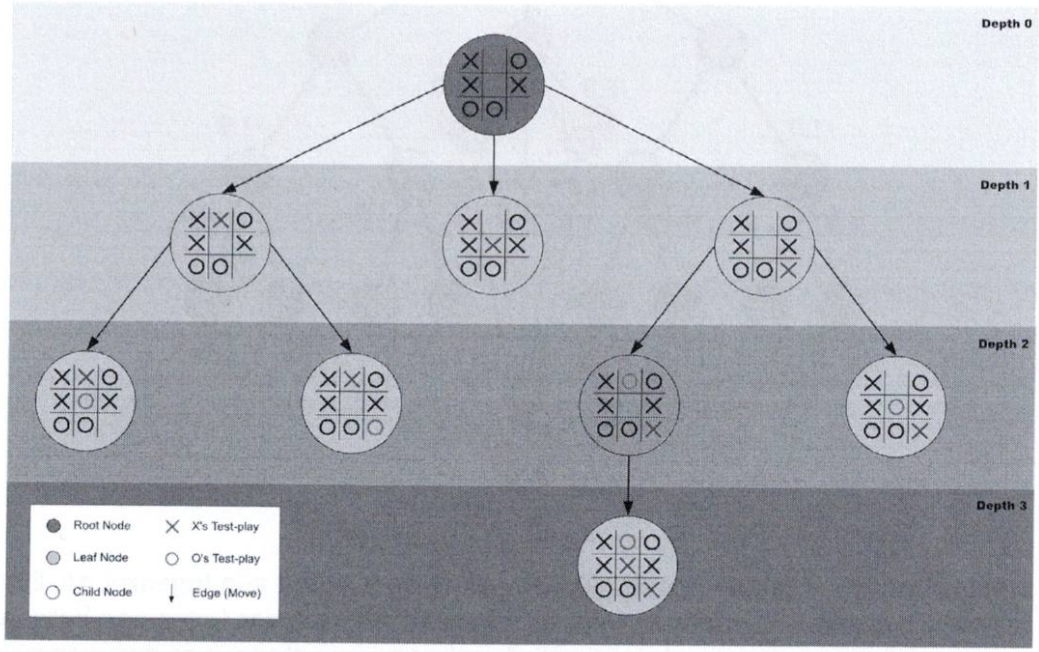
Minimax is een zoekalgoritme voor twee-speler-nulsomspelen: spellen waarbij de winst voor de ene speler een exact even groot verlies voor de andere betekent. Dit soort spelen ken je als tic-tac-toe, schaken, dammen, blad-steen-schaar enz... Het algoritme garandeert de optimale zet, mits je aanneemt dat de tegenstander ook altijd optimaal speelt.

2.1. De probleemruimte als boom

Via een **game tree** breng je alle mogelijke verlopen van een spel in kaart? Je stelt elke mogelijke toestand van het spel voor als een knoop, en elke geldige zet als een tak die leidt naar een nieuwe toestand. De huidige situatie is de root bovenaan. Elke laag dieper in de boom stelt één beurt verder in de toekomst voor.

> **Game trees als kaart van de toekomst**
>
> De toestandsboom is geen beschrijving van hoe het spel *zal* verlopen, maar van hoe het spel *zou kunnen* verlopen. Alle mogelijke paden staan gelijktijdig in de boom en het algoritme kiest er één uit.

De **leaves** (of bladeren) zijn de eindpunten van de boom: posities waarbij het spel voorbij is omdat iemand heeft gewonnen, of het bord vol is. Op die bladeren wordt er een scorefunctie toegepast die de eindtoestand vertaalt naar een getal.

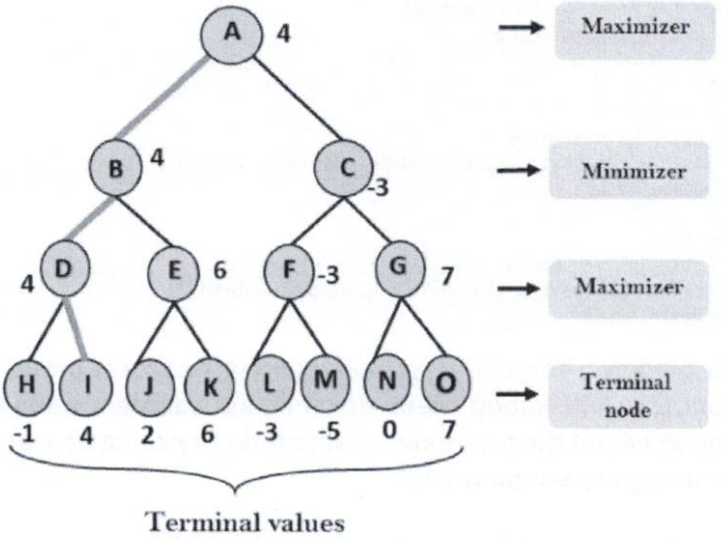


Figuur 7: Visualisatie game tree Tic Tac Toe
Bron: Medium

Het fundamentele probleem is dat die scores enkel beschikbaar zijn aan het *einde* van de boom, in een blad, terwijl een AI-model zijn beslissing moet nemen aan de *top*. Minimax lost dat op via recursie en backtracking.

2.2. MAX, MIN en de nulsomlogica

Wie is wie in een minimax-boom?



Figuur 8: Visualisatie game tree Tic Tac Toe
Bron: Medium

Je speelt als **MAX**: je wilt de score zo hoog mogelijk maken. Je tegenstander is **MIN**: die probeert de score net zo laag mogelijk te duwen. Omdat het om een *zero-sum game* gaat, zijn hun belangen perfect tegengesteld — elke punt winst voor jou is een punt verlies voor MIN, en omgekeerd.

Definition 2.1. *Een **nulsomspel** (zero-sum game) is een situatie waarbij de totale winst van alle spelers samen altijd nul bedraagt. Wat de ene wint, verliest de andere. Er is geen ruimte voor samenwerking of win-winuitkomsten.*

Het algoritme wisselt laag per laag af. Op de MAX-lagen kies je de vertakking met de hoogste waarde. Op de MIN-lagen kiest de tegenstander de vertakking met de laagste waarde. Dit beurtelings maximaliseren en minimaliseren geeft het algoritme zijn naam.

> ⚠ **De aanname van perfecte rationaliteit**
>
> Minimax gaat ervan uit dat MIN altijd de beste beschikbare zet speelt. Dat is voorzichtig maar ook conservatief. Speelt je tegenstander slordig, dan laat je kansen liggen — het algoritme kiest nooit een riskante zet, zelfs niet als die tegen een zwakkere speler wél zou werken.

2.3. Recursie en backtracking

Minimax werkt via **recursie**: het algoritme roept zichzelf aan voor elk kind van de huidige knoop, totdat het een terminale knoop bereikt. Zo daalt het systematisch af door de hele boom, langs elke mogelijke volgorde van zetten.

```
functie minimax(knoop, diepte, isMAX):
    als diepte = 0 of knoop is terminaal:
        geef utility(knoop) terug

    als isMAX:
        beste = -∞
        voor elk kind van knoop:
            beste = max(beste, minimax(kind, diepte-1, ONWAAR))
        geef beste terug
    anders:
        beste = +∞
        voor elk kind van knoop:
            beste = min(beste, minimax(kind, diepte-1, WAAR))
        geef beste terug
```

Eenmaal alle bladeren bezocht, start **backtracking**: de scores propageren omhoog door de boom. Elke MIN-knoop neemt de minimale waarde van zijn kinderen over; elke MAX-knoop neemt de maximale. Zo sijpelt de "beste haalbare uitkomst" stap voor stap omhoog tot aan de wortel.

> **Backtracking als tijdmachine**
>
> Backtracking laat het algoritme redeneren vanuit de toekomst naar het heden. Het is alsof je eerst alle mogelijke eindstanden van een schaakpartij bekijkt, en dan terugwerkt om te bepalen welke eerste zet tot de beste eindstand leidt.

Aan de wortel is er dan voor elke beschikbare zet een berekende minimax-waarde. De zet met de hoogste waarde is de optimale keuze — wiskundig gegarandeerd.

2.4. Toepassingen voorbij het bord

Historisch gezien is minimax enkel van toepassing voor borspelen, maar de principes reiken verder. De bekendste toepassing blijft schaak: IBMs Deep Blue versloeg wereldkampioen Garry Kasparov in 1997 op basis van een sterk geoptimaliseerde variant van minimax. Hedendaagse engines zoals Stockfish combineren minimax met uitgebreide *endgame databases* en heuristische evaluatiefuncties, waardoor ze ver boven het menselijke prestatieniveau uitstijgen.

Toch is het concept van "anticiperen op een tegenstander" breder inzetbaar. In routeoptimalisatie wordt *adversarial search* (concurrentieel zoeken) gecombineerd met technieken als *Ant Colony Optimization*: de "tegenstander" is dan geen mens, maar dynamische beperkingen zoals file, brandstofkosten of tijdschema's. Het systeem zoekt een route die robuust is tegen die verstoringen, precies zoals minimax een zet zoekt die robuust is tegen de beste tegenzet.

Het verschil met klassieke optimalisatie is de aanwezigheid van een actieve "tegenspeler". Bij gewone zoekproblemen ga je ervan uit dat de omgeving neutraal

is. Bij adversarial search ga je ervan uit dat de omgeving actief tegenwerkt met meerdere slimme actoren of spelers die tegen elkaar concurreren en elkaars doelen proberen te dwarsbomen.

2.5. De rekenkost van volledigheid

Waarom rekenen veel tijd kost

De tijdcomplexiteit van minimax is $O(b^d)$, waarbij b de *branching factor* is (het gemiddeld aantal geldige zetten per beurt) en d de diepte van de boom (het aantal beurten vooruit).

Intuïtief: bij elke extra beurt die je vooruit wilt kijken, vermenigvuldig je het totale aantal te onderzoeken knopen met b. Het groeit dus niet lineair, maar *exponentieel*. Voor tic-tac-toe met maximaal 9 zetten is dat beheersbaar. Voor schaken, met een gemiddelde *branching factor* van 35 en partijen van 80 beurten, geeft dat:

$$35^{80} \approx 10^{123}$$

Dat getal overstijgt het aantal atomen in het zichtbare heelal. Volledige minimax is voor schaken dus *onmogelijk*, zelfs op de snelste supercomputer.

Het is dus vaak onmogelijk om alles uit te rekenen, maar een oppervlakkige evaluatie halverwege de boom is echter gevaarlijk: een positie die er goed uitziet op diepte 3 kan op diepte 5 catastrofaal zijn. Hoe dieper je kijkt, hoe betrouwbaarder je beslissing. Er zijn dus goede optimalisatietechnieken nodig zoals alpha-beta pruning.

Alpha-beta pruning is een optimalisatie die grote delen van de toestandsboom overslaat — zonder het uiteindelijke minimax -resultaat te veranderen. De kern van de logica: zodra je ontdekt dat een tak voor de tegenstander (MIN) een betere optie biedt dan wat jij al hebt gevonden, is er geen reden meer om die tak verder te verkennen. MIN zal je toch nooit die kant op sturen.

Example 2.1. Stel: je hebt al een pad gevonden met score +3. Je verkent nu een nieuwe tak en ontdekt al vroeg dat MIN daar minstens -1 kan afdwingen. Omdat $-1 < +3$ zal MAX dit pad nooit kiezen — de rest van de tak mag je negeren.

De twee parameters α (de beste score die MAX tot nu toe heeft gevonden) en β (de beste score die MIN tot nu toe heeft gevonden) worden bijgehouden doorheen de zoektocht. Zodra $\beta \leq \alpha$ geldt op een knoop, wordt de tak gesnoeid.

Zelfde resultaat, minder rekenwerk

Alpha-beta pruning verandert *nooit* het uiteindelijke minimax-besluit. Het is geen benadering. De gesnoede takken bevatten per definitie geen betere uitkomst dan wat al gevonden is. Bij optimale zoekvolgorde halveert alpha-beta

de effectieve diepte — je kijkt even ver vooruit met de wortelkwadraathelft van het rekenwerk.

2.6. Denken & doen

2.6.1. Kernconcepten — vat samen in eigen woorden

Probeer elk begrip in één zin uit te leggen zonder terug te bladeren. Check nadien met de theorie.

- Nulsomspel (zero-sum game)
- Game tree, root en bladeren (leaves)
- MAX versus MIN
- Recursie
- Backtracking
- Alpha-beta pruning (α en β)
- Branching factor en diepte
- Tijdcomplexiteit $O(b^d)$
- Adversarial search

2.6.2. Denken — adversarial search

- Minimax veronderstelt dat de tegenstander altijd optimaal speelt. Is dat een zwakte of een kracht van het algoritme — en wanneer keert het zich tegen je?
- Alpha-beta pruning geeft exact dezelfde uitkomst als pure minimax. Hoe kan je dan takken weggooien zonder het resultaat te beïnvloeden?
- Bedenk een realistisch scenario buiten bordspellen waar je adversarial search zou toepassen. Wie of wat speelt daar de rol van MIN?

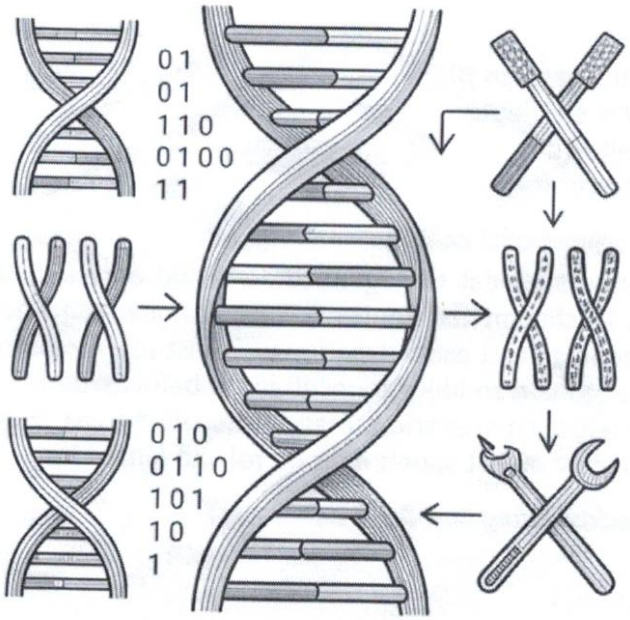
2.6.3. Doen — backtracking met de hand

```
MAX_Wortel
├── Zet A → MIN_Knoop_1
│   ├── Terminaal: +1
│   └── Terminaal: -1
├── Zet B → MIN_Knoop_2
│   ├── Terminaal: 0
│   └── Terminaal: 0
└── Zet C → MIN_Knoop_3
    ├── Terminaal: +1
    └── Terminaal: +1
```

- Bereken via backtracking de minimax-waarde van MIN_Knoop_1, MIN_Knoop_2 en MIN_Knoop_3.
- Bereken de waarde van MAX_Wortel. Welke zet kies je?
- Wijs één tak aan die alpha-beta zou snoeien als je de boom van links naar rechts doorzoekt, en leg uit op welk moment de snoeiconditie optreedt.

Genetische Algoritmen

CHAPTER 3



Soms is een probleem gewoon té groot om exact op te lossen. Neem het plannen van een bezorgroute langs honderd adressen: het aantal mogelijke volgorden is astronomisch groot, en elke aanpak die alle opties één voor één doorrekent, is bij voorbaat kansloos. Toch lossen biologische systemen dit soort complexiteitsproblemen elke dag op. Ze doen dit niet via beredeneerde berekening, maar via miljoenen jaren aan kleine, willekeurige veranderingen die zich opstapelen.

Genetische algoritmen (*genetic algorithms*) lenen die logica rechtstreeks. Je codeert een oplossing als een reeks eigenschappen, laat een grote verzameling kandidaat-oplossingen met elkaar "kruisen", en houdt bij elke stap de best presterende varianten bij. Zo navigeer je stap voor stap door een enorme zoekruimte zonder die ooit volledig te hoeven doorzoeken.

3.1. Van biologie naar datastructuur

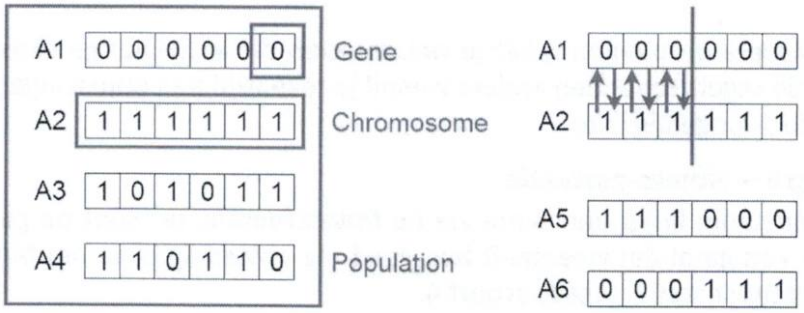
We kennen natuurlijk de 'Survival of the fittest'-theorie van Darwin, maar hoe vertaal je die evolutietheorie naar bruikbare code?

Laten we starten met de terminologie. Een **populatie** is de volledige verzameling kandidaat-oplossingen die op een gegeven moment in je systeem leven. Elk individu daarin heet een **chromosoom**: de complete, gecodeerde blauwdruk van precies één oplossing.

Een chromosoom is opgebouwd uit **genen** — kleinere eenheden die elk één beslissing of eigenschap representeren. De concrete waarde in zo'n gen heet een **allel**. Tot slot definieer je de **fitness**: de kwantitatieve score die aangeeft hoe goed een chromosoom presteert op jouw probleem.

Definition 3.1. *Een populatie is een verzameling van chromosomen. Elk chromosoom bestaat uit genen met een allel als waarde. De fitness meet hoe goed een chromosoom het probleem oplost.*

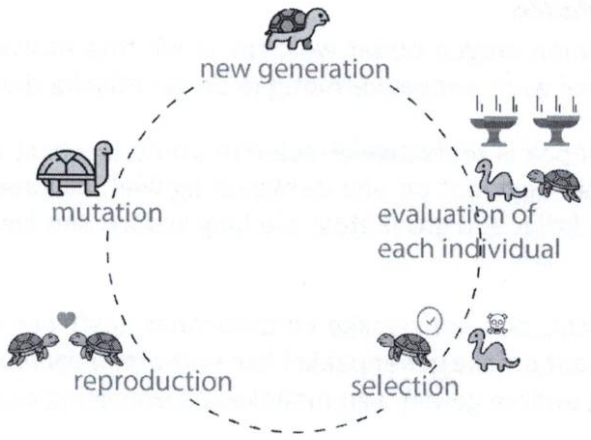
Concreet voorbeeld: je zoekt een binaire reeks van zes bits met zo veel mogelijk enen. Dan is elk chromosoom een reeks zoals `101010`, elk gen één bitpositie, elk allel `0` of `1`, en de fitness gewoon het aantal enen. Simpel, maar dat is wat er speelt in roostersoptimalisatie of chemische molecuulsamenstelling.



Figuur 10: Populatie
Bron: Medium

3.2. De levenscyclus: vijf stappen

Een genetisch algoritme is een cyclus. Je herhaalt dezelfde vijf stappen generatie na generatie, totdat een stopcriterium bereikt is.



Figuur 11: Genetisch algoritme
Bron: Generative Design Primer

3.2.1. Stap 1 — Initialisatie

Je begint met een volledig willekeurige beginpopulatie. Elk gen van elk chromosoom wordt los bepaald door een virtuele muntopgooi. Die willekeur is geen zwakte zoals we al eerder zeiden, maar het is een bewuste keuze.

> **Waarom willekeur bij de start?**
>
> Stel dat je zoekruimte één miljard mogelijke oplossingen bevat. Als je de beginpopulatie concentreert in één hoek van die ruimte, dek je statistisch gezien een fractie van het totaal af. Crossover kan uitsluitend **interpoleren** tussen waarden die al aanwezig zijn in de genenpoel — het kan geen nieuw terrein **buiten** die pool bedenken. Zonder brede initiële randomness loop je het risico dat het globale optimum buiten het bereik van je startwaarden ligt en het algoritme daar nooit bij kan komen.

Bij complexere problemen moet je wel opletten dat willekeurige chromosomen geen harde regels schenden anders verspil je rekentijd aan oplossingen die in de praktijk toch ongeldig zijn.

3.2.2. Stap 2 — Fitness-evaluatie

Elk chromosoom krijgt een score via de **fitnessfunctie**. Je loopt de genen af en berekent een getal dat weergeeft hoe goed die oplossing is. In het binaire voorbeeld: `100100` scoort 2, `111010` scoort 4.

> **⚠ De fitnessfunctie is de enige sturing**
>
> Als je fitnessfunctie de praktijkeisen niet nauwkeurig weerspiegelt, optimaliseert het algoritme feilloos naar de verkeerde uitkomst. Het systeem "weet" niet wat jij écht wilt, het volgt enkel de score die jij definieert. Een fout in de fitnessfunctie is daarmee een blinde vlek die het algoritme zelf nooit kan corrigeren.

3.2.3. Stap 3 — Selectie

Welke chromosomen mogen ouder worden? Je wil fitte individuen vaker kiezen, maar niet exclusief want anders vernietig je de genetische diversiteit.

De bekendste aanpak is **roulettewiel-selectie** (*roulette wheel selection*). Elk chromosoom krijgt een segment op een denkbeeldig wiel, evenredig aan zijn fitness. Wie hoog scoort, krijgt een groot stuk; wie laag scoort, een klein stukje. Dan spin je het wiel.

Het cruciale inzicht: ook het zwakke chromosoom heeft nog een kans. Dat lijkt inefficiënt, maar dat unieke genenpakket kan verborgen combinaties bevatten die later, samen met andere genen, een uitstekende oplossing opleveren.

3.2.4. Stap 4 — Crossover en mutatie

Bij **crossover** (*crossover*) combineer je twee ouders tot twee kinderen. Bij *single-point crossover* kies je een willekeurig splitsingspunt in de bitreeks en wissel je de helften uit.

Example 3.1. Ouder A: `111 | 000` — Ouder B: `000 | 111`
Splitsing na bit 3:
Kind 1: `111111` (fitness 6)
Kind 2: `000000` (fitness 0)

Crossover combineert lokaal succesvolle eigenschappen. Het resultaat kan beter of slechter zijn dan beide ouders.

Na crossover volgt **mutatie** (*mutation*): met een bewust lage kans flip je een willekeurig gen. Een `0` wordt een `1`, of omgekeerd.

> **Waarom is mutatie wiskundig onmisbaar?**
>
> In een complexe zoekruimte bestaan **lokale optima**: "bergtoppen" waar elke kleine stap omlaag leidt, terwijl elders een hogere bergtop bestaat (het **globale optimum**). De selectiedruk drijft de populatie automatisch omhoog langs de dichtstbijzijnde helling — en trekt die vast bovenop het verkeerde lokale maximum.
>
> Mutatie forceert af en toe een **willekeurige sprong** die een chromosoom over het dal heen katapulteert. Zo ontsnapt het algoritme uit de verstikkende grip van een lokaal optimum en kan het nieuwe regio's verkennen.

De mutatiekans instellen is een evenwichtsoefening. Te laag: het algoritme geraakt nooit uit lokale optima. Te hoog: je overschrijft wat crossover net heeft opgebouwd en de evolutie degradeert tot willekeurig zoeken.

3.2.5. Stap 5 — Nieuwe generatie en terminatie

De kinderen vervangen de zwakkere individuen in de populatie, en de cyclus herhaalt. Generatie na generatie schuift de gemiddelde fitness omhoog.

Wanneer stop je? Twee veelgebruikte criteria:

1. **Maximaal aantal generaties**: simpel, maar soms stop je te vroeg of te laat.
2. **Stagnatie**: als de beste fitness gedurende bijvoorbeeld vijftig generaties niet meer stijgt, concludeer je dat verdere verbetering onwaarschijnlijk is.

Het tweede criterium is doorgaans intelligenter: je stopt niet op een vast tijdstip, maar op basis van wat het algoritme zelf signaleert.

3.3. *Selectie, crossover en mutatie in pseudocode*

Het onderstaande fragment illustreert de kernstructuur van een genetisch algoritme.

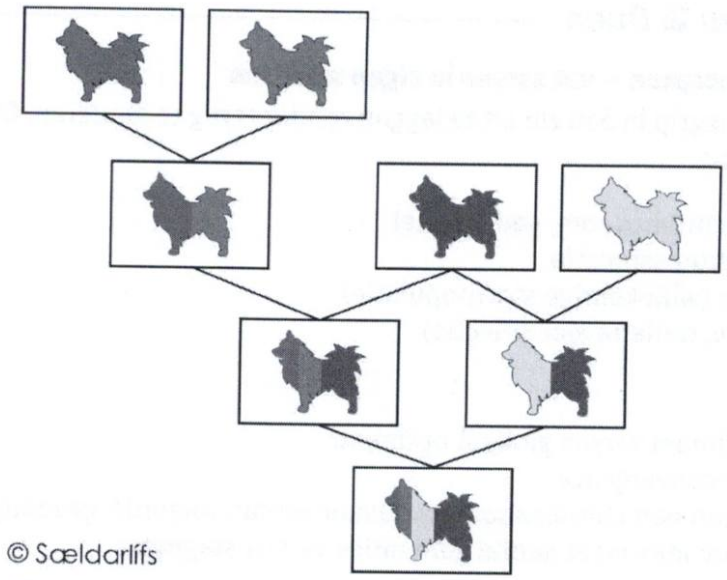
```
functie genetischAlgoritme(populatieGrootte, chromosoomLengte, mutatiekans,
aantalGeneraties):
    populatie = willekeurige startpopulatie van populatieGrootte chromosomen

    voor generatie van 1 tot aantalGeneraties:
        bereken fitness van elk chromosoom in populatie

        nieuwePopulatie = []
        zolang nieuwePopulatie kleiner is dan populatieGrootte:
            ouderA, ouderB = selecteer twee ouders op basis van fitness
            kind1, kind2 = crossover(ouderA, ouderB)
            muteer(kind1, mutatiekans)
            muteer(kind2, mutatiekans)
            voeg kind1 en kind2 toe aan nieuwePopulatie

        populatie = nieuwePopulatie

    geef beste chromosoom uit populatie terug
```

Maar er zijn wel wat valkuilen waar je tegen kan aanlopen. Hoe je een **chromosoom codeert**, bepaalt bijvoorbeeld wat crossover kan bereiken. Binaire codering werkt uitstekend voor problemen waarbij genen onafhankelijk zijn. Maar stel je voor: je plant de volgorde van vijf taken en codeert dat als `12345`. Na crossover kan je `12312` krijgen — taak 1 staat twee keer, taak 5 ontbreekt. Het chromosoom is niet meer geldig.

> ⚠ **Codering bepaalt of crossover zinnig is**
>
> Voor problemen waarbij de **volgorde** of **uniekheid** van waarden essentieel is (bv het *Travelling Salesman Problem*) werkt standaard binaire of numerieke codering niet. Dan zijn speciale crossover-operatoren nodig. Een slechte keuze hier maakt de hele reproductiefase nutteloos.

Een andere valkuil is de selectiedruk te hoog zetten, door enkel de allerbeste chromosomen te laten voortplanten. Zodra de variantie in de populatie naar nul daalt, verliest crossover zijn werking: twee identieke ouders produceren identieke kinderen. Dit heet **premature convergence**: het algoritme convergeert vóór de zoekruimte fatsoenlijk verkend is, naar een lokaal optimum dat het globale optimum nooit bereikt.

Het tegengif is dus net **diversiteit** bewaren: niet enkel de sterksten laten voortplanten, maar ruimte laten voor variatie — precies zoals in de natuur. Een populatie met te weinig genetische variatie loopt vast, terwijl gezonde diversiteit toelaat dat nieuwe combinaties blijven ontstaan.



Figuur 12: Genetische diversiteit: kenmerken vermengen zich generatie na generatie.

Bron: Genetische Diversiteit

Dat is meteen ook de kern van wat een genetisch algoritme zo krachtig maakt: het imiteert geen wiskundige truc, maar een mechanisme dat de natuur al miljoenen jaren gebruikt om complexe problemen op te lossen — bit voor bit, generatie na generatie.

3.4. *Denken & Doen*

3.4.1. Kernconcepten — vat samen in eigen woorden

Probeer elk begrip in één zin uit te leggen zonder terug te bladeren. Check nadien met de theorie.

- Populatie, chromosoom, gen en allel
- Fitness en fitnessfunctie
- Initialisatie (willekeurige startpopulatie)
- Selectie (bv. roulettewiel-selectie)
- Crossover
- Mutatie
- Lokaal optimum versus globaal optimum
- Premature convergence
- Codering van een chromosoom (bv. binair versus volgorde-gevoelig)
- Stopcriteria: maximaal aantal generaties versus stagnatie

3.4.2. Denken — evolutionair zoeken

- Waarom gebruikt een computer voor een denkspel (zoals schaken) een tactische rekenmethode die vooruitdenkt naar de tegenstander, maar voor het ontwerpen van de perfecte vorm van een voorwerp (zoals een wifi-antenne) een methode die lijkt op biologische evolutie?
- Waarom is het cruciaal om via probabilistische selectie ook "zwakke" individuen een overlevingskans te geven, als je het globale optimum wil bereiken?
- Wat zijn de gevolgen voor je systeem als de fitnessfunctie de praktijkeisen niet nauwkeurig weerspiegelt?
- Als je enkel de allerbeste chromosomen laat voortplanten, loopt de populatie het risico op *premature convergence*. Wat betekent dat, en waarom lost meer diversiteit dit probleem op?

3.4.3. Doen — één generatie met de hand

Individu	Bitreeks	Fitness
A	1 0 1 0 0 0	?
B	0 1 1 1 1 0	?
C	1 0 0 0 0 0	?
D	0 0 0 1 1 1	?

Crossover: één-punts, exact in het midden. Mutatie: flip altijd bit 6.

```
functie fitness(bitreeks):
  return aantal_enen(bitreeks)

functie crossover(ouder1, ouder2):
  kind = eerste_helft(ouder1) + tweede_helft(ouder2)
  return muteer_laatste_bit(kind)
```

- Bereken de fitness van elk individu.
- Kruis de beste twee individuen (grootste fitness) en produceer één kind.

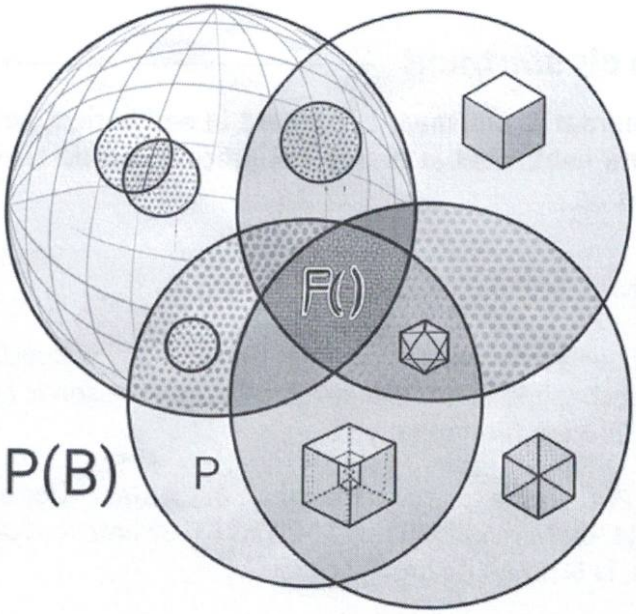
- Vergelijk de fitness van het kind met het gemiddelde van Generatie 0. Is vooruitgang op lange termijn gegarandeerd? Waarom (niet)?

Klassiek Machine Learning

4. Naïve Bayes
5. Het k-Nearest Neighbor
6. Decision Trees
7. Logistic Regression

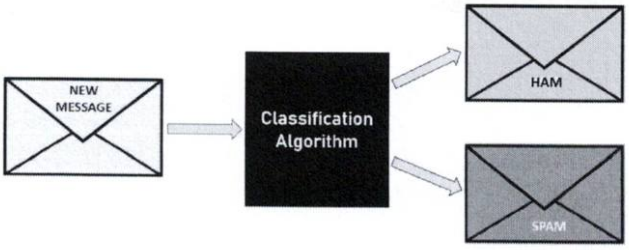
Naïve Bayes

CHAPTER 4



Elke keer dat je een e-mail opent en je inbox netjes gevuld is met echte berichten, heeft er ergens een algoritme stilletjes werk gedaan. Het heeft honderden berichten gescand, woorden gewogen en een beslissing genomen. Een van de meest elegante manieren om dit te doen, is **Naïve Bayes**: snel, transparant en verrassend effectief.

Wat Naïve Bayes bijzonder maakt, is niet de complexiteit maar net de eenvoud. Het algoritme bouwt op één grote, bewuste versimpeling van de werkelijkheid. Die versimpeling klinkt wiskundig naïef en dat is ze ook, maar toch werkt ze verrassend goed.



Figuur 14: Spam of Ham bericht.
Bron: Medium

Voor je in de wiskunde duikt, even de terminologie. Een bericht dat je wél wil ontvangen, een mail van een collega, een factuur, een nieuwsbrief waar je op inschreef, noemen we **ham**. Een bericht dat je niet wil, zoals reclame of oplichting, noemen we **spam**. Elk binnenkomend bericht valt in een van die twee categorieën.

De vraag is: hoe beslis je, puur op basis van de tekst, in welke categorie een nieuw bericht hoort? Je hebt geen zekerheid, alleen aanwijzingen. Bepaalde woorden ("gratis", "jackpot", "klik hier") komen vaker voor in spam dan in ham. Dat gegeven is genoeg om een kansberekening op te bouwen.

4.1. *Kansen als startpunt*

Wat weet je voordat je ook maar één woord in een bericht hebt gelezen? Als je historische data hebt, weet je al hoeveel procent van de berichten doorgaans spam of ham is.

> **Wat is een kans eigenlijk?**
>
> Een **kans** is een getal tussen 0 en 1 dat uitdrukt hoe waarschijnlijk iets is. 0 betekent onmogelijk, 1 betekent zeker, en alles ertussen is een gradatie: 0.7 wil zeggen "in 7 op de 10 gevallen".
>
> De notatie $P(A)$ lees je als "de kans op A". Dat A staat voor een gebeurtenis, bijvoorbeeld "dit bericht is spam". $P(\text{spam}) = 0.67$ betekent dus: op basis van wat je weet, is 67% van de berichten spam.
>
> Belangrijk: een kans zegt niets zeker over dit ene bericht. Het is een inschatting op basis van patronen die je eerder zag, geen garantie.

Stel je dus voor dat de historische kans op spam 67% is: dat is de inschatting vóór je ook maar één woord van het bericht las. Deze kans noemen we de **prior kans**, genoteerd als $P(A)$. Zodra je informatie toevoegt, pas je die inschatting aan.

Daarna stap je over op de **voorwaardelijke kans** $P(A|B)$: de kans op uitkomst A, gegeven dat je B hebt waargenomen. Dat is de kern van redeneren met onzekerheid.

Tot slot is er de **gezamenlijke kans** $P(A \cap B)$: de kans dat zowel A als B tegelijk waar zijn. Deze drie concepten, prior, voorwaardelijk en gezamenlijk, vormen het fundament van alles wat volgt.

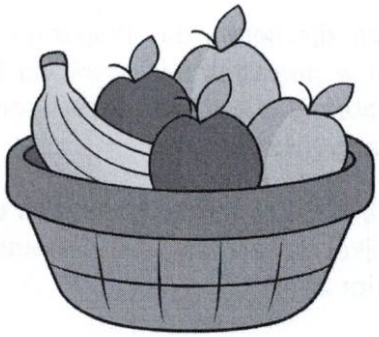
4.1.1. De stelling van Bayes

Hoe combineer je die concepten tot een bruikbaar oordeel? Dat is precies wat de **stelling van Bayes** doet:

$$P(A|B) = \frac{P(B|A) \cdot P(A)}{P(B)}$$

Elk symbool in die formule heeft een naam.

- $P(A)$ is de **prior**: wat je al wist over A voordat je iets over B zag.
- $P(B|A)$ is de **likelihood**: hoe waarschijnlijk B is, áls A waar is.
- $P(B)$ is hoe waarschijnlijk B sowieso is, los van A.
- En $P(A|B)$, het antwoord waar het je om gaat, is de **posterior**: je bijgewerkte kans op A, nu je B hebt waargenomen.



Stel: een fruitmand bevat 60% bananen en 40% appels. Van de bananen is 70% geel, van de appels maar 30%. Je pakt een willekeurige vrucht en ziet dat ze geel is, wat is de kans dat het een banaan is?

Dus $P(\text{banaan}) = 0.6$ en $P(\text{appel}) = 0.4$, dan berekenen we de kans op geel, ongeacht het soort fruit:

$$\begin{aligned} P(\text{geel}) &= P(\text{geel}|\text{banaan}) \cdot P(\text{banaan}) + P(\text{geel}|\text{appel}) \cdot P(\text{appel}) \\ &= 0.7 \cdot 0.6 + 0.3 \cdot 0.4 = 0.54 \end{aligned}$$

Vul dat in bij de stelling van Bayes:

$$P(\text{banaan}|\text{geel}) = \frac{P(\text{geel}|\text{banaan}) \cdot P(\text{banaan})}{P(\text{geel})} = \frac{0.7 \cdot 0.6}{0.54} \approx 0.78$$

Een gele vrucht uit deze mand is dus met ongeveer 78% waarschijnlijkheid een banaan.

Terug naar spam en ham: dezelfde logica met één verschil. Bij de banaan had je maar één stukje bewijs: de kleur. Een e-mail bestaat zelden uit één woord; het is een hele reeks aanwijzingen tegelijk. Elk woord in het bericht is een apart stukje bewijs, en Naïve Bayes combineert die simpelweg door hun kansen met elkaar te vermenigvuldigen. Waarom dat mag, zie je verderop bij de naïeve aanname, voorlopig volstaat het om te weten dát het kan.

Stel je hebt zes berichten in je trainingsdata:

Bericht	Klasse
D1: "send us your password"	Spam
D2: "send us your review"	Spam
D3: "review your password"	Ham
D4: "review us"	Ham
D5: "send us password"	Spam
D6: "send us your account"	Spam

Vier van de zes berichten zijn spam, dus $P(\text{spam}) = \frac{4}{6}$ en $P(\text{ham}) = \frac{2}{6}$: dat is je prior. Voor elk woord tel je hoe vaak het voorkomt binnen de spam-berichten en binnen de ham-berichten, dat geeft je de likelihood per woord, bijvoorbeeld $P(\text{password}|\text{spam}) = \frac{2}{4}$ en $P(\text{password}|\text{ham}) = \frac{1}{2}$.

Stel nu dat er een nieuw bericht binnenkomt: "review us now". Omdat je wil weten welke klasse waarschijnlijker is, bereken je de verhouding tussen beide posteriors in plaats van elke posterior apart:

$$\frac{P(\text{spam}|\text{review,us,now})}{P(\text{ham}|\text{review,us,now})} = \frac{P(\text{review}|\text{spam}) \cdot P(\text{us}|\text{spam}) \cdot P(\text{now}|\text{spam}) \cdot P(\text{spam})}{P(\text{review}|\text{ham}) \cdot P(\text{us}|\text{ham}) \cdot P(\text{now}|\text{ham}) \cdot P(\text{ham})}$$

Is die verhouding groter dan 1, dan wint spam; is ze kleiner dan 1, dan wint ham. Het woord "now" staat in geen van de zes trainingsberichten. Om te vermijden dat dat ene onbekende woord de hele berekening naar nul trekt, gebruik je voor dat woord gewoon de factor 1, zowel bij spam als bij ham. Reken je alles in, dan kom je uit op 0.75: dat is kleiner dan 1, dus classificeert Naïve Bayes "review us now" als ham.

Remark. Die truc om een onbekend woord de factor 1 te geven kom je verderop in dit hoofdstuk nog een keer tegen, maar dan met een naam: **Laplace smoothing**.

4.2. *De naïeve aanname die alles verandert*

Een bericht bestaat dus zelden uit één woord zoals we al zagen. In de echte kansrekening moet je rekening houden met de onderlinge afhankelijkheden tussen woorden: "jackpot" en "winnen" komen nu eenmaal vaak samen voor, maar Naïve Bayes doet dit heel bewust niet. Het algoritme gaat ervan uit dat elk woord volledig onafhankelijk bijdraagt aan de kans op een klasse, los van alle andere

woorden. Volgorde en context doen er niet toe, precies wat je hierboven al deed toen je *P*(review|spam), *P*(us|spam) en *P*(now|spam) gewoon met elkaar vermenigvuldigde.

> ⚠ **Wat 'naïef' hier betekent**
>
> *Naïef* verwijst naar de onafhankelijkheidsaanname die het algoritme maakt: in de praktijk zijn woorden verre van onafhankelijk, maar het algoritme doet alsof dat wél

4.3. *Het nulprobleem en Laplace smoothing*

Je zag het hierboven al met het woord "now": dat woord kwam in geen van de zes trainingsberichten voor, dus was de likelihood ervoor eigenlijk 0. Dat is op zich geen ramp, want Naïve Bayes vermenigvuldigt toch alle kansen met mekaar, maar alles wat je met 0 vermenigvuldigt trekt het resultaat naar 0 én dat is wel een probleem natuurlijk!

Het ontbreken van één enkel woord dat toevallig nog nooit in je trainingdata stond, trekt op die manier de hele berekening naar 0, alsof die klasse plots onmogelijk is. Een perfect legitiem bericht kan zo onterecht als spam worden gemarkeerd, puur omdat het één onbekend woord bevat, ook al is de rest van het bericht overduidelijk ham.

De oplossing heet **Laplace smoothing**: je doet gewoon alsof elk woord, in elke klasse, minstens één keer extra voorkwam. Zo kan een kans nooit meer exact 0 worden.

$$P(\text{woord} \mid \text{klasse}) = \frac{\text{aantal keer woord in klasse} + 1}{\text{totaal woorden in klasse} + |\text{vocabularium}|}$$

Boven bij de teller tel je gewoon +1 op, ook als een woord 0 keer voorkwam, zo wordt 0 minstens 1. Onder bij de noemer tel je het totale aantal verschillende woorden in je vocabularium op, zodat de kansen nog steeds eerlijk optellen tot 1. Een woord dat écht vaak voorkwam, houdt nog steeds een veel hogere kans dan een woord dat nooit voorkwam. Je vervormt de boel een beetje, maar redt de berekening.

> 💡 **Kerninsicht: smoothing als vangnet**
>
> Zonder smoothing is je classifier breekbaar: één onbekend woord brengt de hele berekening ten val. Met smoothing geef je elk onbekend woord een minimale basiswaarschijnlijkheid. De classifier blijft stabiel, ook op nieuwe data.

4.4. Tellen, combineren, classificeren

Hoe ziet Naïve Bayes er uit in de praktijk? Hieronder een stukje pseudocode die laat zien hoe je de kernlogica opbouwt: frequenties tellen, prior berekenen en een bericht classificeren.

```
functie train(bericht, isSpam):
    voor elk woord in bericht:
        verhoog telling van woord in de klasse (spam of ham)
    verhoog totaal aantal berichten in die klasse


functie likelihood(woord, klasse):
    als woord nooit gezien is:
        geef 1.0 terug    // onbekend woord telt niet mee
    geef (aantal keer woord in klasse) / (totaal aantal berichten in klasse) terug


functie isSpam(bericht):
    priorSpam = totaalSpam / totaalBerichten
    priorHam  = totaalHam / totaalBerichten

    scoreSpam = priorSpam
    scoreHam  = priorHam

    voor elk woord in bericht:
        scoreSpam = scoreSpam * likelihood(woord, spam)
        scoreHam  = scoreHam * likelihood(woord, ham)

    geef terug: scoreSpam > scoreHam
```

Dit werkt perfect op papier, en ook in code zolang je berichten kort zijn, maar wat als je berichten of teksten veel langer worden? Elke `likelihood` is een kans tussen 0 en 1. Vermenigvuldig je er genoeg van na elkaar, dan daalt het resultaat razendsnel richting nul. Stel dat elk woord gemiddeld een likelihood van ongeveer 0.01 heeft, en je bericht bevat 50 woorden:

$$0.01^{50} = 10^{-100}$$

Een `double` kan grofweg tot 10^{-308} nog een onderscheid maken tussen "heel klein" en "nul" én dat lijkt veel ruimte, maar bij voldoende lange berichten zak je daar wel degelijk onder. Zodra dat gebeurt, rondt de computer het getal gewoon af naar 0 (underflow).

> **⚠ Waarom underflow een echt probleem is**
>
> Als zowel `scoreSpam` als `scoreHam` allebei afronden naar 0, verlies je precies de informatie die je nodig had om te beslissen. `scoreSpam > scoreHam` wordt dan `0 > 0`, wat altijd `onwaar` teruggeeft — ongeacht wat het bericht eigenlijk zegt. Je classificatie is dan stuk, zonder dat er een expliciete fout optreedt.

Om dit op te vangen vervang je de vermenigvuldiging van kansen door een optelling van logaritmen. Door te werken met $\log(P)$ in plaats van P zelf, vermijd je underflow volledig, zonder het eindresultaat te veranderen.

Dat werkt dankzij een eigenschap van logaritmen: het logaritme van een product is de som van de logaritmes.

$$\log(a \cdot b) = \log(a) + \log(b)$$

Concreet betekent dit dat je in `isSpam` niet langer vermenigvuldigt, maar optelt:

```
functie isSpamLog(bericht):
  logScoreSpam = log(priorSpam)
  logScoreHam = log(priorHam)

  voor elk woord in bericht:
    logScoreSpam = logScoreSpam + log(likelihood(woord, spam))
    logScoreHam  = logScoreHam + log(likelihood(woord, ham))

  geef terug: logScoreSpam > logScoreHam
```

Het logaritme van een kleine kans wordt sterk **negatief** (bv. log(0.01) ≈ -4.6), maar dat getal blijft binnen een normaal, hanteerbaar bereik (het rondt niet af naar nul) hoe lang het bericht ook is. Vergelijk je de twee opgetelde sommen, dan wint nog steeds dezelfde klasse als voorheen: welke som het grootst is (dus het minst negatief), komt overeen met welke vermenigvuldiging oorspronkelijk het grootst zou geweest zijn. Het resultaat blijft dus wiskundig identiek, enkel de berekening zelf wordt numeriek stabiel.

4.5. *Aanbevelingen*

Is Naïve Bayes alleen bruikbaar voor spam? Niet echt, het algoritme doet in de kern niets anders dan: kies tussen twee klassen op basis van kenmerken die je telt. Nergens in dat recept staat dat die kenmerken per se woorden in een e-mail moeten zijn.

Neem aanbevelingssystemen, ook daar wil je een ja/nee-beslissing: vindt deze gebruiker dit product interessant, klasse 1, of niet, klasse 0? Dat is dezelfde vorm als spam/ham, alleen de klassen heten anders.

Expliciet om een beoordeling vragen - "geef dit 1 tot 5 sterren" - is meestal een drempel: de meeste gebruikers doen dat gewoonweg niet. Maar ze laten wél sporen na zonder het te beseffen. Een klik, een scroll, een aankoop: dat zijn de nieuwe "woorden", kenmerken die je gewoon telt, net zoals je telde hoe vaak "gratis" in spam voorkwam.

De drie bouwstenen vertalen rechtstreeks mee. De historische data van alle gebruikers, hoeveel procent doorgaans op dit soort item klikt, vormt de **prior**. Het specifieke gedrag van deze ene gebruiker met dit item is de **likelihood**. En de **posterior** is het antwoord waar het je om gaat: vindt déze gebruiker dít specifieke item interessant? Zo bouw je een gepersonaliseerd systeem zonder ooit om een ster te vragen.

4.6. *Denken & Doen*

4.6.1. Kernconcepten — vat samen in eigen woorden

Probeer elk begrip in één zin uit te leggen zonder terug te bladeren. Check nadien met de theorie.

- Prior, likelihood en posterior
- Voorwaardelijke kans versus gezamenlijke kans
- De stelling van Bayes
- De naïeve onafhankelijkheidsaanname
- Het nulprobleem
- Laplace smoothing
- Underflow
- Werken met logaritmes in plaats van kansen

4.6.2. Denken — kansrekenen als classificatie

- Waarom kan de basale Naïve Bayes-formule computationeel in de problemen komen bij extreem lange documenten, en hoe los je dit wiskundig op?
- In welke talige context faalt de onafhankelijkheidsaanname het hardst, en waarom leidt dit tot een foutieve classificatie?
- Wat is het effect van Laplace smoothing wanneer je trainingsdataset extreem klein is versus gigantisch groot?
- Een recommendation engine gebruikt Naïve Bayes zonder ooit expliciet om een score te vragen. Welke drie bouwstenen (prior, likelihood, posterior) vertalen zich naar klikgedrag, en wat is dan concreet elk van die drie?

4.6.3. Doen — spam of ham?

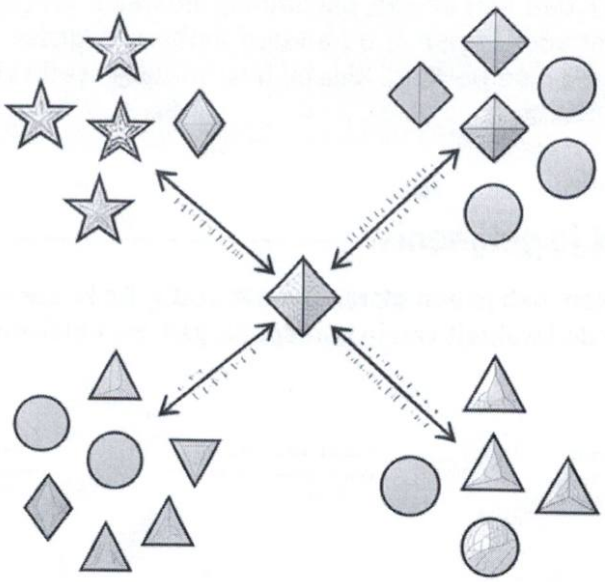
ID	Bericht	Label
D1	"send us your password"	Spam
D2	"send us your review"	Spam
D3	"review your password"	Ham
D4	"review us"	Ham
D5	"send us password"	Spam
D6	"send us your account"	Spam

Nieuw bericht: **"review your profile"**. Gebruik factor 1 voor woorden die nul keer voorkomen in een klasse.

- Bereken de prior kansen *P*(Spam) en *P*(Ham).
- Stel de likelihoods op voor de woorden **review**, **your** en **profile** per klasse.
- Bereken de ongenormaliseerde posteriorindicatoren en trek je conclusie.

Het k-Nearest Neighbor

CHAPTER 5



Stel je voor dat je net bent verhuisd naar een nieuwe wijk en wil weten of je een rustige of drukke buurt hebt gekozen. Je kijkt niet naar een kaart of statistieken, je stapt gewoon naar buiten en kijkt wie er rondom jou woont. Dat is precies de redenering achter het *k*-**Nearest Neighbor** (*k*NN) algoritme: een datapunt is wat zijn buren zijn.

In tegenstelling tot de algoritmen die je eerder zag, waarbij een model expliciet getraind wordt en wiskundige parameters worden bijgesteld onthoudt *k*NN simpelweg alle trainingsdata. Er is geen leerproces in de klassieke zin. Wanneer je een nieuw, onbekend punt aanbiedt, gaat het algoritme op zoek naar de meest gelijkende punten in de opgeslagen dataset. Die buren "stemmen" vervolgens over de uitkomst.

5.1. Hoe werkt *k*NN?

Het idee is intuïtief: zoek de *k* meest gelijkende datapunten, en gebruik die om een voorspelling te maken. Bij **classificatie** wint de meerderheidsklasse; bij **regressie** (zoals het voorspellen van een ontbrekende score in een aanbevelingssysteem) neem je het gemiddelde van de buren.

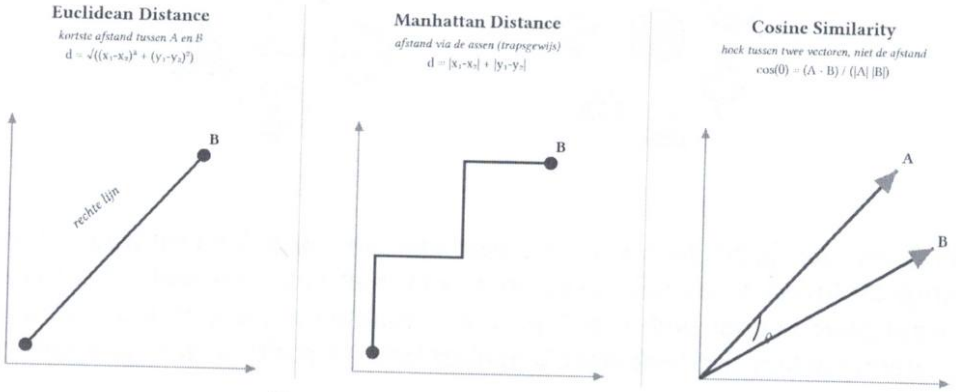
De variabele *k* is de enige parameter die je als ontwerper handmatig instelt. Ze bepaalt hoeveel buren mogen meestemmen. Kies je $k = 1$, dan kijk je uitsluitend

naar het allerdichtste punt. Dat maakt het model razendsnel en gevoelig voor lokale patronen, maar ook kwetsbaar: één uitbijter of fout in de data kan de volledige voorspelling verpesten. Een grote k werkt andersom: je kijkt naar veel buren tegelijk, waardoor de invloed van ruis afvlakt. Maar te groot en je model begint subtiele patronen te missen: het ziet het bos niet meer door de bomen.

> ⚠ **Let op bij een even k**
>
> Werk je met een binaire doelklasse (twee mogelijke uitkomsten) en kies je een even k? Dan kan er een patstelling ontstaan: precies de helft van de buren stemt voor klasse A, de andere helft voor klasse B. Je hebt dan geen beslissende meerderheid. Kies bij binaire classificatie bij voorkeur een oneven k.

5.2. *Hoe meet je gelijkenis?*

Om buren te vinden, heb je een **afstandsmaat** nodig. De keuze van die maat heeft grote invloed op de kwaliteit van je voorspellingen, we bekijken er drie.



Figuur 17: Afstanden berekenen

5.2.1. Euclidische afstand

De meest bekende maat is de **Euclidische afstand**: de rechte lijn tussen twee punten, eigenlijk gewoon de afstand in vogelvlucht. In twee dimensies is dit gewoon de stelling van Pythagoras. In n dimensies, wanneer elk kenmerk een dimensie is, veralgemeen je dat:

$$d(x, y) = \sqrt{\sum_{i=1}^{n} (x_i - y_i)^2}$$

Gebruik deze maat voor continue, fysieke data (lengte, gewicht, coördinaten) waarbij absolute schaalverschillen betekenisvol zijn.

Example 5.1. Twee gebruikers beoordeelden dezelfde drie films: $(x) = (5, 3, 4)$ en $(y) = (3, 1, 2)$.

$$d(x, y) = \sqrt{(5-3)^2 + (3-1)^2 + (4-2)^2} = \sqrt{16} \approx 3.46$$

Een relatief grote afstand: hoewel het **patroon** van hun scores gelijkaardig oogt, ligt het **niveau** ver uit elkaar.

5.2.2. Manhattan-afstand

De **Manhattan-afstand** meet de afstand langs een haaks raster, alsof je door de straten van een schaakbordpatroon rijdt:

$$d(x, y) = \sum_{i=1}^{n} |x_i - y_i|$$

Dit is handig voor data waarbij kenmerken niet vloeiend in elkaar overlopen, of wanneer je minder gevoelig wilt zijn voor extreme uitschieters. Een groot verschil op één kenmerk telt minder zwaar dan bij de Euclidische maat.

Example 5.2. Met dezelfde twee gebruikers $(x) = (5, 3, 4)$ en $(y) = (3, 1, 2)$:

$$d(x, y) = |5-3| + |3-1| + |4-2| = 2 + 2 + 2 = 6$$

Merk op: geen kwadraten, dus geen enkel verschil weegt extra zwaar door.

5.2.3. Cosine similarity

Voor gebruikersratings in aanbevelingssystemen (en voor tekstdata) zijn Euclidische maten vaak minder geschikt. Hier schittert de **cosine similarity**: in plaats van de afstand tussen twee punten meet ze de **hoek** tussen twee vectoren.

$$\text{Sim}(x, y) = \cos(\theta) = \frac{(x) \cdot (y)}{\|(x)\| \cdot \|(y)\|} = \frac{\sum x_i y_i}{\sqrt{\sum x_i^2} \cdot \sqrt{\sum y_i^2}}$$

Stel: gebruiker A geeft films steevast een 5, gebruiker B geeft exact dezelfde films een 3. Hun absolute scores liggen ver uit elkaar — maar hun smaakpatroon is identiek. De hoek tussen hun vectoren is nul graden, dus de cosinus is 1: een perfecte gelijkenis. Euclidische afstand zou hen onterecht als vreemden behandelen.

Example 5.3. Nog steeds $(x) = (5, 3, 4)$ en $(y) = (3, 1, 2)$:

$$\text{Sim}(x, y) = \frac{5 \cdot 3 + 3 \cdot 1 + 4 \cdot 2}{\sqrt{5^2 + 3^2 + 4^2} \cdot \sqrt{3^2 + 1^2 + 2^2}} = \frac{26}{7.07 \cdot 3.74} \approx 0.98$$

Een cosine similarity van 0.98: bijna perfect gelijkaardig — een heel ander oordeel dan de Euclidische afstand van 3.46 hierboven, en dat terwijl het over dezelfde twee gebruikers gaat.

> **Waarom hoek en niet afstand?**
>
> Wanneer je de lengte van een vector vergelijkt, meet je hoe intens iemand scoort. Maar bij aanbevelingen wil je weten of twee gebruikers **dezelfde**

smaakverhouding hebben, niet of ze even genereus zijn met punten geven. De cosine similarity laat de lengte bewust los en richt zich puur op de richting van de vector. Een veelbeoordelaar en een occasionele kijker met identieke smaakverhoudingen krijgen een score van 1 — terecht, want voor aanbevelingen zijn ze elkaars perfecte tweeling.

Formeel: de cosinus van de hoek θ tussen twee vectoren (x) en (y) is gelijk aan hun inwendig product gedeeld door het product van hun normen. De norm van een vector is gewoon een ander woord voor de lengte (of grootte) van die vector genoteerd als $\|(x)\|$. Bij een hoek van 0° is cos(0°) = 1 (identiek); bij 90° is cos(90°) = 0 (niets gemeenschappelijk).

5.3. Het belang van normalisatie

Afstandsmaten zijn blind voor de context van je kenmerken. Als je de Euclidische afstand berekent tussen huizen op basis van het aantal slaapkamers (schaal 1–5) én de catalogusprijs (schaal 100.000–1.000.000), dan domineert de prijs volledig. Een verschil van één slaapkamer (conceptueel enorm voor een koper) weegt nauwelijks mee tegenover een prijsverschil van tienduizenden euro's.

Normalisatie lost dit op door alle kenmerken te transformeren naar een uniform bereik, typisch 0 tot 1 (*min-max scaling*) of rond een gemiddelde van 0 met standaardafwijking 1 (*z-score normalisatie*). Hierna krijgt elk kenmerk een eerlijke, evenredige stem in de afstandsberekening.

> **Normalisatie is geen luxe**
>
> Bij *k*NN is normalisatie bijna altijd verplicht. Zonder normalisatie meet je niet "wie lijkt op wie", maar "welk kenmerk heeft de grootste getallen". Dat is zelden wat je wilt.

5.4. Sorteren en stemmen

Hoe ziet k-Nearest Neighbors er uit in de praktijk? Het algoritme zelf is verrassend eenvoudig: bereken de afstand tot alle gekende punten, sorteer, en kijk naar de *k* dichtstbijzijnde buren.

Bij classificatie wil je weten tot welke **klasse** een nieuw punt behoort, bijvoorbeeld: is deze gebruiker een actie- of een romantiekliefhebber?

```
functie kNN_classificatie(trainingdata, nieuwPunt, k):
  afstanden = []

  voor elk (punt, klasse) in trainingdata:
      d = bereken afstand(nieuwPunt, punt)   // Euclidisch, Manhattan of cosine
      afstanden.voeg_toe((d, klasse))

  sorteer afstanden van klein naar groot

  buren = eerste k elementen uit afstanden
```

```
  geef terug: meest voorkomende klasse onder buren
```

De laatste stap is een simpele meerderheidsstem: van de k buren die het dichtst bij liggen, wint de klasse die het vaakst voorkomt. Bij $k = 5$ en drie buren uit klasse "actie" tegenover twee uit klasse "romantiek", classificeer je het nieuwe punt dus als "actie".

Wil je geen klasse voorspellen maar een **getal**, bijvoorbeeld welke rating een gebruiker aan een film zou geven, dan verandert enkel de laatste stap.

```
functie kNN_regressie(trainingdata, nieuwPunt, k):
  afstanden = []

  voor elk (punt, waarde) in trainingdata:
      d = bereken afstand(nieuwPunt, punt)
      afstanden.voeg_toe((d, waarde))

  sorteer afstanden van klein naar groot

  buren = eerste k elementen uit afstanden

  geef terug: gemiddelde van de waarden van buren
```

In plaats van te stemmen op een klasse, neem je hier het **gemiddelde** van de k dichtstbijzijnde waarden. Geven de vijf dichtstbijzijnde buren respectievelijk 4, 5, 3, 4 en 5 sterren, dan is de voorspelling $\frac{4+5+3+4+5}{5} = 4.2$.

5.5. *Aanbevelingen*

De logica van kNN vertaalt zich direct naar aanbevelingssystemen. Wil je voorspellen of een gebruiker een bepaalde film goed zou vinden, dan zoek je simpelweg naar gebruikers met een gelijkaardig smaakpatroon, en kijk je wat die "buren" van die film vonden. Dezelfde logica werkt ook omgekeerd: in plaats van gelijkende gebruikers te zoeken, kan je ook gelijkende items zoeken, films die vaak door dezelfde mensen goed beoordeeld worden, bijvoorbeeld.

> ⚠ **Weinig overlapping = onbetrouwbare similariteit**
>
> Cosine similarity lijkt betrouwbaar, maar dat is ze niet als twee gebruikers slechts één of twee gemeenschappelijke films hebben beoordeeld. Eén toevallige overeenkomst geeft dan een hoge score, terwijl er eigenlijk niets te concluderen valt. Stel daarom altijd een **drempelwaarde** in voor het minimum aantal overlappende beoordelingen voordat je twee gebruikers als "gelijkend" beschouwt.

5.6. Denken & doen

5.6.1. Kernconcepten — vat samen in eigen woorden

Probeer elk begrip in één zin uit te leggen zonder terug te bladeren. Check nadien met de theorie.

- k als enige parameter, en het effect van een kleine versus grote k
- Classificatie versus regressie bij kNN
- Euclidische afstand
- Manhattan-afstand
- Cosine similarity
- Normalisatie (min-max en z-score)
- Lazy learning
- Drempelwaarde voor minimum overlap bij aanbevelingen

5.6.2. Denken — buren en afstand

- Wat is het gevaar van een te grote k, en hoe raakt dit de *bias-variance trade-off*?
- Methode A (Naïve Bayes) vat alle gegevens vooraf samen in vaste regels, terwijl Methode B (kNN) "lui" is en alle ruwe data gewoon bewaart tot het moment dat je een vraag stelt. Wat leert elk systeem tijdens de trainingsfase, en wat betekent dit voor de reactiesnelheid en het computergeheugen bij het geven van een antwoord?
- Twee gebruikers hebben elk maar één film gemeen beoordeeld, met toevallig dezelfde score: cosine similarity 1,000. Is dit een bruikbaar signaal?
- Je vergelijkt huizen op basis van aantal slaapkamers (1–5) en prijs (100.000–1.000.000). Waarom domineert de prijs de afstandsberekening zonder normalisatie, en waarom is dat een probleem?

5.6.3. Doen — naaste buur zoeken

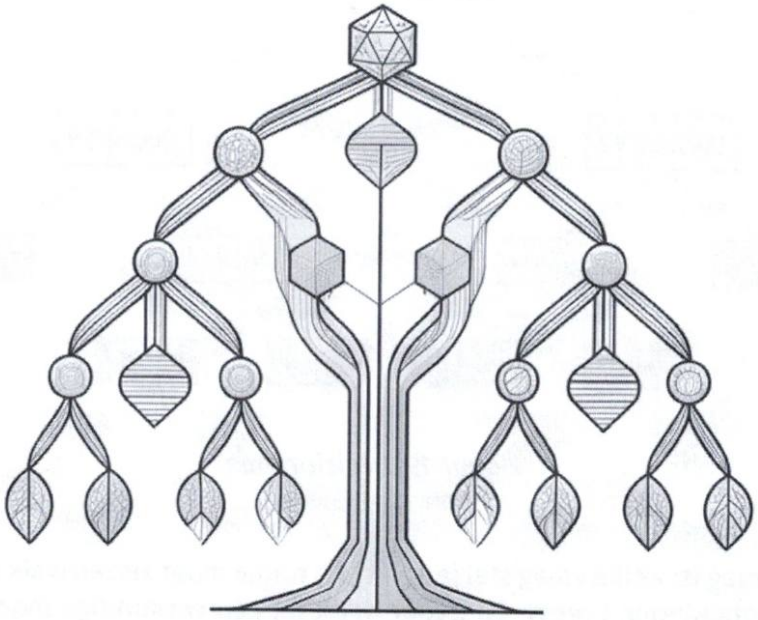
Gebruiker	Film 1	Film 2	Film 3	Film 4	Film 5
Gebruiker X	4	2	5	3	?
Gebruiker A	5	1	4	2	5
Gebruiker C	4	2	5	3	2

```
functie cosine_similarity(v1, v2):
  return dot(v1, v2) / (norm(v1) * norm(v2))
```

- Bereken de cosine similarity tussen Gebruiker X en A (enkel Film 1 t/m 4).
- Doe hetzelfde voor Gebruiker C. Wie is de naaste buur bij $k = 1$?
- Voorspel de ontbrekende rating van X voor Film 5 via het 1NN-principe.

Decision Trees

CHAPTER 6

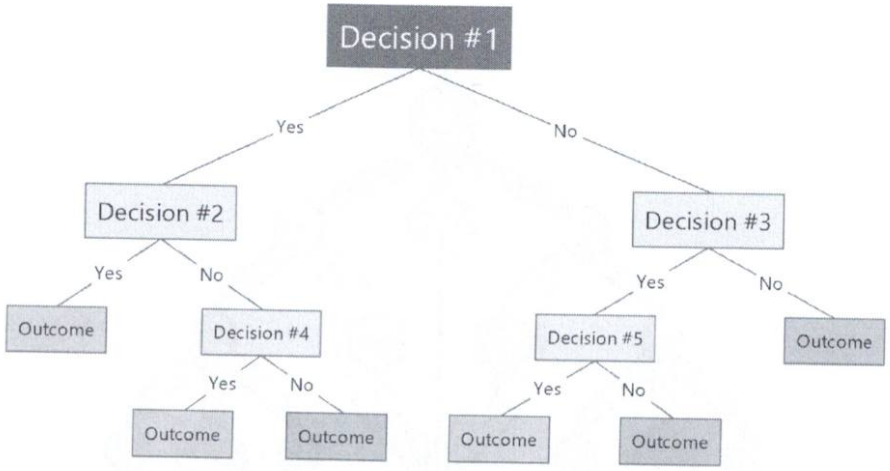


Stel je voor dat je op een zaterdagochtend naar buiten kijkt en twijfelt of je gaat wandelen. Wat je brein onbewust doet, is een reeks ja/nee-vragen stellen om tot een beslissing te komen. Dat is precies hoe een **Decision Tree** (*beslissingsboom*) werkt: het algoritme bootst dat menselijke keuzeproces na.

In tegenstelling tot de algoritmes uit de vorige hoofdstukken stelt een Decision Tree geen kansen op, en meet hij geen afstanden. Hij stelt gewoon de juiste vragen in de juiste volgorde.

6.1. Hoe een Decision Tree werkt

Een boom begint bovenaan bij de **wortel** (*root node*), stelt daar een vraag, en volgt dan een **tak** (*branch*) op basis van het antwoord. Dat herhaalt zich tot het algoritme een **blad** (*leaf node*) bereikt: een eindpunt met een definitief label.



Figuur 19: Decicion tree.
Bron: MindManager

De echte vraag is: welke vraag stel je eerst? De boom moet kiezen welk kenmerk de grootste opheldering brengt. Daarvoor heeft hij een wiskundige maatstaf nodig: entropie. **Entropie** is een maat voor de onzekerheid of onzuiverheid binnen een groep datapunten. Hoe gelijkmatiger de klassen verdeeld zijn, hoe hoger de entropie en hoe minder je weet wat het antwoord zal zijn.

Een dataset van 8 voorbeelden met 4x "Ja" en 4x "Nee" heeft maximale entropie: je gokt maar raak. Een dataset van 8x "Ja" heeft entropie nul: er is geen twijfel meer.

6.1.1. Informatiewinst

Om te beslissen **welk** kenmerk je gebruikt om de boom te splitsen, bereken je de **information gain** (*informatiewinst*). Dat is simpelweg: hoeveel daalt de entropie nadat je de data op een bepaald kenmerk opsplitst?

Het recept is telkens hetzelfde: je telt per groep hoeveel voorbeelden van elke klasse er zijn, zet dat om in een verhouding, en die verhouding bepaalt hoe "gemengd" die groep nog is. Doe dat voor de volledige dataset (de start-entropie) en voor elk van de deelgroepjes na een mogelijke splitsing. Het verschil tussen die twee is je informatiewinst.

Als je data bijvoorbeeld verdeelt op "Weer" en je krijgt een groep met louter zonnige sportdagen en een groep met louter regenachtige binnenzitdagen, is de onzekerheid na de splitsing nul. Maximale informatiewinst "Weer" is de perfecte eerste vraag.

Onthoud vooral dit

Hoe zuiverder de groepjes die uit een splitsing rollen, hoe hoger de informatiewinst. De boom probeert dus telkens: "welke vraag zorgt voor de netste, meest eenduidige deelgroepjes?" en stelt die vraag eerst. Een splitsing die één piepklein zuiver groepje oplevert en de rest in chaos achterlaat, telt trouwens niet mee als een goede splitsing: grote, zuivere groepen wegen zwaarder door dan kleine.

Optioneel — de formule voor wie ze wil zien

Je hebt de formule niet nodig om het concept te snappen, maar ze bestaat:

$$H(T) = -\sum_{i}(p_i \cdot \log_2(p_i))$$

$$\mathrm{IG}(T, a) = H(T) - \sum_{v \in \text{ values}(a)} \frac{|T_v|}{|T|} \cdot H(T_v)$$

$H(T)$ is de entropie van groep T, en p_i is het aandeel van klasse i in die groep. De logaritme zorgt ervoor dat zeldzame, onverwachte uitkomsten zwaarder doorwegen in de onzekerheid. De weging $\frac{|T_v|}{|T|}$ in de tweede formule is precies wat "grote, zuivere groepen tellen zwaarder" hierboven wiskundig uitdrukt.

6.1.2. Gini als alternatief

In de praktijk hoor je naast entropie ook de **Gini-impuriteit** (*Gini impurity*) als maatstaf. Gini meet hoe groot de kans is dat een willekeurig gekozen datapunt verkeerd gelabeld wordt als je het puur op basis van de kansverdeling in die groep een label geeft.

Beide maatstaven leiden in de praktijk bijna altijd tot dezelfde boom. Entropie is iets zwaarder wiskundig (logaritmen), Gini iets sneller te berekenen. De keuze tussen beide heeft zelden een grote impact op het eindresultaat.

6.1.3. Overfitting: wanneer een boom te gretig is

Een boom kan niet eeuwig blijven groeien. Als je hem zijn gang laat gaan, maakt hij voor elk uniek datapunt een eigen tak aan: perfecte prestatie op de trainingsdata, maar totale mislukking op nieuwe situaties. Dat is **overfitting**.

Je dwingt de boom te stoppen via zogenaamde *stopping criteria*: het knooppunt is al volledig puur, de ingestelde maximale diepte (*max depth*) is bereikt, of de subgroep is te klein geworden om statistisch verantwoord verder te splitsen.

Overfitting bij Decision Trees

Een onbegrensde boom is een reeks uitzonderingen op een regel, geen echte regel. Beperk de diepte van je boom altijd, ook al presteert een diepe boom beter op je trainingsset.

6.2. *Recursief splitsen*

Onder de motorkap van elke Decision Tree-implementatie zit hetzelfde recursieve recept: kies de beste vraag, splits de data, en herhaal op elk deel apart.

```
FUNCTIE BouwBoom(data, maxDiepte, huidigeDiepte):
    // Stopcriteria
    ALS data volledig zuiver is (alle labels gelijk):
        RETURN Blad(label = meerderheidslabel(data))
    ALS huidigeDiepte == maxDiepte OF |data| < minimumGrootte:
        RETURN Blad(label = meerderheidslabel(data))

    // Zoek de beste splitsing
    besteKenmerk = GEEN
    besteWinst = 0

    VOOR ELK kenmerk IN data.kenmerken:
        winst = GiniWinst(data, kenmerk)
        ALS winst > besteWinst:
            besteWinst = winst
            besteKenmerk = kenmerk

    ALS besteWinst == 0:
        RETURN Blad(label = meerderheidslabel(data))

    // Splits de data op het beste kenmerk en herhaal
    linkerdata, rechterdata = Splits(data, besteKenmerk)
    linkertak = BouwBoom(linkerdata, maxDiepte, huidigeDiepte + 1)
    rechtertak = BouwBoom(rechterdata, maxDiepte, huidigeDiepte + 1)

    RETURN Knoop(kenmerk = besteKenmerk, links = linkertak, rechts = rechtertak)

FUNCTIE GiniImpuriteit(data):
    totaal = |data|
    som = 0

    VOOR ELK klasse c IN unieke labels van data:
        p = aantal(data, label = c) / totaal
        som = som + p * p

    RETURN 1 - som


FUNCTIE GiniWinst(data, kenmerk):
    startGini = GiniImpuriteit(data)
    linkerdata, rechterdata = Splits(data, kenmerk)

    gewogenGini = (|linkerdata| / |data|) * GiniImpuriteit(linkerdata)
                + (|rechterdata| / |data|) * GiniImpuriteit(rechterdata)

    RETURN startGini - gewogenGini
```

Wat deze pseudocode duidelijk maakt

De boom bouwt zichzelf op door zichzelf steeds opnieuw aan te roepen (recursie) op een steeds kleinere deelverzameling van de data. Elke aanroep herhaalt dezelfde vraag: "welk kenmerk geeft hier de grootste informatiewinst?" tot een stopcriterium het proces afbreekt.

6.3. Denken & Doen

6.3.1. Kernconcepten — vat samen in eigen woorden

Probeer elk begrip in één zin uit te leggen zonder terug te bladeren. Check nadien met de theorie.

- Root node, branch en leaf node
- Entropie
- Information gain
- Gini-impuriteit
- Overfitting bij Decision Trees
- Stopping criteria (max depth, minimumgrootte, volledige zuiverheid)
- Recursief splitsen

6.3.2. Denken — splitsen en zuiverheid

- Waarom kruipt de entropie naar exact 0 wanneer een subknooppunt volledig homogeen is? Hoe matcht dit met het intuïtieve idee van "verrassing"?
- Vergelijk hoe een Decision Tree en een Naïve Bayes-classificator omgaan met sterk gecorreleerde kenmerken. Wie is gevoeliger, en waarom?
- Waarom is normaliseren van kenmerken van levensbelang bij *k*NN, maar irrelevant voor een Decision Tree?
- Een onbegrensde boom presteert perfect op de trainingsdata maar faalt op nieuwe, ongeziene data. Hoe heet dit fenomeen, en welke stopcriteria kan je gebruiken om het te vermijden?

6.3.3. Doen — entropie en information gain

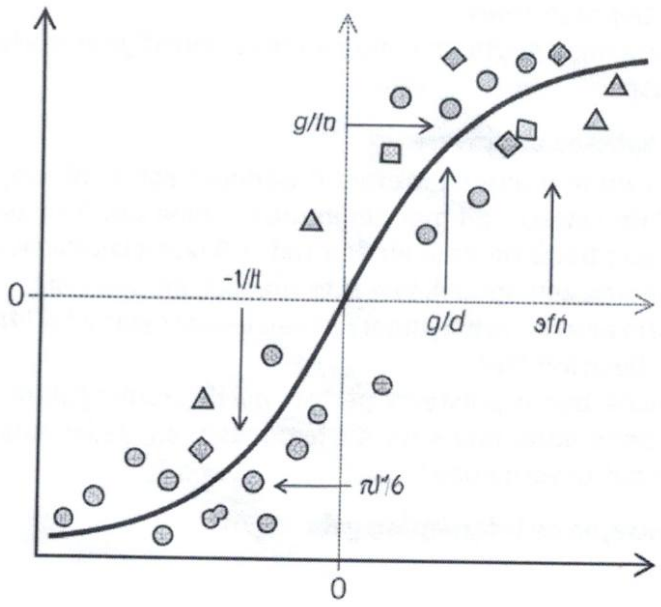
Gebruiker	Genre	Tempo	Binge?
1	SF	Snel	Ja
2	SF	Snel	Ja
3	SF	Traag	Ja
4	SF	Traag	Nee
5	Drama	Snel	Ja
6	Drama	Snel	Nee
7	Drama	Traag	Nee
8	Drama	Traag	Nee

Remark. Tip: $\log_2(1) = 0$, $\log_2(2) = 1$, $\log_2(4) = 2$.

- Bereken de start-entropie $H(\text{Total})$ van 'Binge?' (4 Ja, 4 Nee).
- Bereken de information gain bij splitsing op **Genre**.
- Bereken de information gain bij splitsing op **Tempo**.
- Welk kenmerk kiest de boom als root node? Wat betekent het als beide exact gelijk zijn?

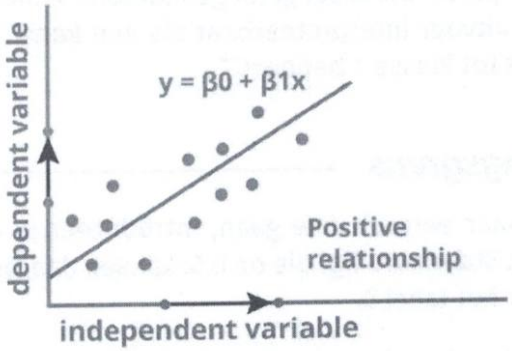
Logistic Regression

CHAPTER 7



In het vorige hoofdstuk stelde de Decision Tree een reeks vragen om tot een beslissing te komen. **Logistic Regression** (*logistische regressie*) doet iets fundamenteel anders: in plaats van vragen te stellen, berekent het een **kans** en neemt dan pas een beslissing.

De naam is misleidend: ondanks het woord "regressie" gebruik je dit algoritme steevast als **classificatie**-algoritme. Onder de motorkap berekent het model een continue waarde, een kans tussen 0 en 1, maar die kans zetten we direct om in een harde beslissing.



Figuur 21: Linear regression.
Bron: Medium

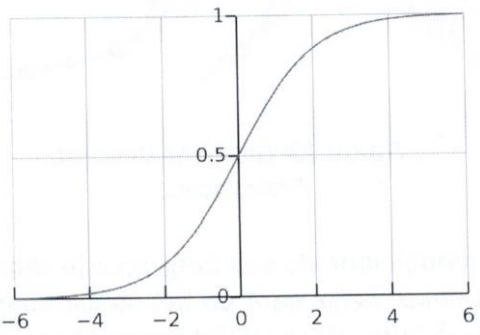
Die naam komt niet zomaar uit de lucht vallen. Je kent wellicht al lineaire regressie: een rechte lijn door je data die een getal voorspelt, via een gewogen som van kenmerken: $y = w_1x_1 + w_2x_2 + ... + b$. Logistic Regression hergebruikt exact diezelfde gewogen som, enkel het laatste stapje verschilt. Waar lineaire regressie die som rechtstreeks als antwoord geeft, perst Logistic Regression hem eerst door een functie die er een kans tussen 0 en 1 van maakt. Vandaar de naam: de *regressie* zit in de berekening, de *classificatie* in wat je met die kans doet.

7.1. De sigmoid-functie als vertaler

Een lineaire combinatie van kenmerken kan in theorie van $-\infty$ tot $+\infty$ lopen. Om dat te vertalen naar een bruikbare kans, gebruikt het algoritme de **sigmoid-functie** (*S-curve*):

$$S(x) = \frac{1}{1 + e^{-x}}$$

De term e^{-x} is altijd positief, ongeacht hoe groot of klein x is. Als x naar $+\infty$ gaat, krimpt e^{-x} naar 0 en wordt de uitvoer $\frac{1}{1+0} = 1$. Als x naar $-\infty$ gaat, explodeert e^{-x} naar oneindig en daalt de uitvoer naar 0. Bij $x = 0$ levert de formule precies 0{,}5 op, het symmetrische kantelpunt.



Figuur 22: Sigmoid function.

De S-vormige curve "perst" elk reëel getal genadeloos samen in het open interval $(0, 1)$. Dit maakt de uitvoer interpreteerbaar als een **kans**: "hoe waarschijnlijk is het dat dit datapunt tot klasse 1 behoort?"

7.2. De beslissingsgrens

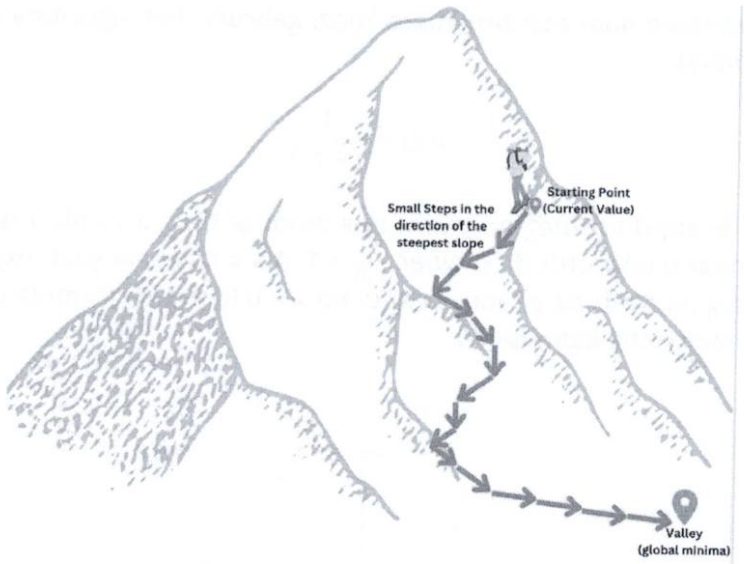
Om van een kans naar een label te gaan, introduceer je een **beslissingsgrens** (*decision boundary*). Standaard ligt die op 0,5: kansen daarboven krijgen het label 1, kansen daaronder het label 0.

In een tweedimensionale ruimte is deze grens een rechte lijn die de klassen scheidt. Dat is tegelijk de kracht én de beperking van Logistic Regression: de grens is altijd lineair.

Logistic Regression blinkt uit wanneer je klassen lineair scheidbaar zijn en wanneer je stabiele, vloeiende kansberekeningen nodig hebt. Decision Trees zijn je go-to wanneer de data complexe, niet-lineaire interacties bevat, zoals "als het regent **én** de temperatuur is onder 10 graden, dan pas gaan we niet."

7.3. Voorspellen en trainen

Het voorspellen zelf is simpel: gewichten vermenigvuldigen, optellen, door de sigmoid halen. Trainen is iets anders: je moet die gewichten stap voor stap bijstellen zodat de voorspelde kansen dichter bij de echte labels komen.



Figuur 23: Gradient Descent.
Bron: Medium

Stel je de fout van je model voor als een berg waar je vanaf wil dalen, in de mist, zonder kaart. Je voelt enkel welke kant op het steilst naar beneden gaat, en zet een klein stapje in die richting. Herhaal dat genoeg keer, en je belandt onderaan bij een model dat zo goed mogelijk voorspelt. Die aanpak heet **gradient descent**, en je zal ze nog twee keer tegenkomen in deze cursus: bij Matrix Factorization en

bij Neurale Netwerken. Hoe die "richting naar beneden" precies berekend wordt, zie je in detail bij Matrix Factorization, hier volstaat het te weten dát je die richting volgt.

```
functie gewogenSom(x, gewichten, bias):
    z = bias
    voor elk kenmerk i in x:
        z = z + gewichten[i] * x[i]
    geef z terug


functie voorspelKans(x, gewichten, bias):
    geef sigmoid(gewogenSom(x, gewichten, bias)) terug


functie voorspel(x, gewichten, bias):
    kans = voorspelKans(x, gewichten, bias)
    als kans >= 0.5:
        geef 1 terug
    anders:
        geef 0 terug


functie sigmoid(z):
    geef 1 / (1 + e^(-z)) terug

functie train(data, labels, leerSnelheid, aantalStappen):
    gewichten = nulvector(lengte = aantal kenmerken)
    bias = 0

    herhaal aantalStappen keer:
        voor elk (x, label) in (data, labels):
            voorspelling = voorspelKans(x, gewichten, bias)
            fout = voorspelling - label

            voor elk kenmerk i in x:
                gewichten[i] = gewichten[i] - leerSnelheid * fout * x[i]
            bias = bias - leerSnelheid * fout

    geef gewichten, bias terug
```

7.4. Aanbevelingen

Logistic Regression stelt steeds dezelfde vraag: hoort dit bij klasse 1, of bij klasse 0? Bij een aanbevelingssysteem is die vraag bijna letterlijk hetzelfde: gaat deze gebruiker op dit item klikken, ja of nee?

Dat maakt Logistic Regression een klassieker in de *ranking*-stap van productie-aanbevelingssystemen. Eerst selecteert een ander algoritme, bijvoorbeeld *k*NN of Matrix Factorization een honderdtal kandidaat-items. Logistic Regression sorteert die kandidaten daarna: voor elk kandidaat-item berekent het de kans dat deze specifieke gebruiker erop klikt, op basis van kenmerken zoals recente kliks, tijdstip, of het apparaat. De items met de hoogste kans komen bovenaan.

7.5. Denken & Doen

7.5.1. Kernconcepten — vat samen in eigen woorden

Probeer elk begrip in één zin uit te leggen zonder terug te bladeren. Check nadien met de theorie.

- Sigmoid-functie
- Beslissingsgrens (0,5)
- Gewichten en bias
- Lineaire scheiding
- Gradient descent
- Logistic Regression in de ranking-stap van een aanbevelingssysteem

7.5.2. Denken — een rechte door de kansruimte

- Waarom raakt een wiskundig formule-model (Logistieke Regressie) in de war als gegevens dubbelop zijn of sterk overlappen, terwijl een stroomdiagram met ja/nee-vragen (Beslisboom) daar moeiteloos mee omgaat?
- Wat betekent het inhoudelijk als de voorspelde kans exact 0{,}5 is? Waarom is dit punt het "moeilijkste" geval voor het model?
- Waarom normaliseer je kenmerken bij Logistic Regression, terwijl dit bij een Decision Tree geen enkel verschil maakt?
- Een aanbevelingssysteem gebruikt eerst *k*NN of Matrix Factorization om kandidaat-items te kiezen, en pas daarna Logistic Regression om ze te sorteren. Waarom heb je die twee stappen nodig, in plaats van meteen met Logistic Regression te beginnen?

7.5.3. Doen — sigmoid en classificatie

$w_1 = 0{,}8$, $w_2 = 1{,}2$, $b = -3{,}0$. $z = w_1 \cdot x_1 + w_2 \cdot x_2 + b$

E-mail	Uitroeptekens (x_1)	Links (x_2)
A	1	1
B	3	2
C	0	0

```
functie sigmoid(z):
  return 1 / (1 + e^(-z))
```

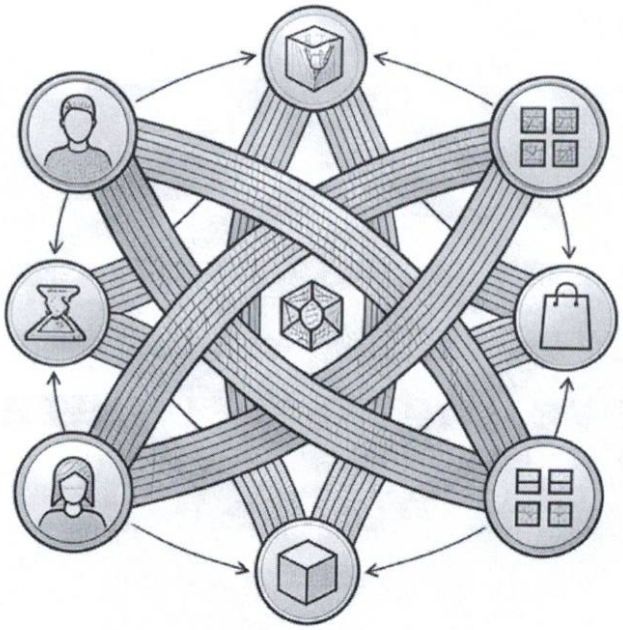
- Bereken z voor elk van de drie e-mails.
- Bereken de sigmoid $S(z)$ voor e-mail B (gebruik $e \approx 2{,}718$).
- Bepaal het label (Spam = 1/0) van elke e-mail, zonder $S(z)$ voor A en C volledig uit te rekenen — het teken van z volstaat.

Aanbevelingen & Neurale Netwerken

8. Recommender Systems
9. Matrix Factorization
10. Neurale netwerken

Recommandations

CHAPTER 8



Wanneer je Spotify opent voor je wekelijkse *Discover Weekly*, of door Netflix scrolt op zoek naar een film, bevind je je in het hart van een geavanceerde *recommendation engine*. Deze systemen lijken ondertussen heel vanzelfsprekend, bijna 'gewone software', maar eigenlijk is dat niets minder waar én is het best knap wat daar onder de motorkap afspeelt!

8.1. De brandstof: expliciete en impliciete data

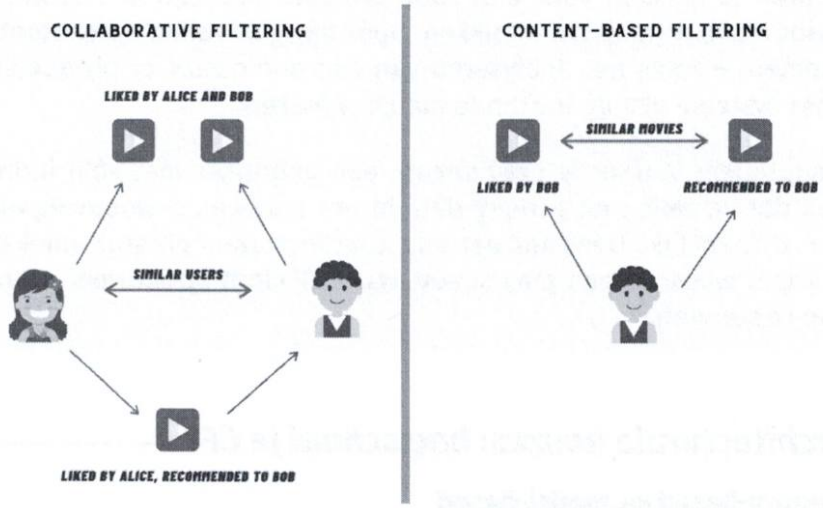
Elk aanbevelingssysteem heeft data nodig om te leren. Maar niet alle data is gelijkwaardig. Je maakt een fundamenteel onderscheid tussen twee types.

Expliciete data ontstaat wanneer je bewust een oordeel velt: een duimpje omhoog op Netflix, een 5-sterrenbeoordeling op Amazon. Deze data is zuiver en eenduidig maar ook uiterst schaars. Consumenten beoordelen zelden actief wat ze bekijken of kopen.

Impliciete data registreert onbewust gedrag: je klikpaden, hoever je scrolt op Zalando, of het feit dat je een Spotify-nummer na drie seconden skipt of net op repeat zet. Deze datastroom is veel rijker en omvangrijker, maar bevat ook meer ruis, maar toch vormt impliciete data de ware ruggengraat van moderne aanbevelingssystemen.

> **Kerninsicht**
>
> Expliciete data is zeldzaam maar betrouwbaar. Impliciete data is overvloedig maar ruis. Moderne systemen combineren beide, met een zwaar accent op impliciet gedrag.

8.2. *Hoe filtert een systeem? Drie basisstrategieën*



Figuur 25: Filtering.
Bron: Towards Data Science

8.2.1. Collaborative Filtering

De meest klassieke aanpak is **collaborative filtering** (*CF*). Het basisprincipe is puur sociaal: mensen die in het verleden op jou leken qua smaak, vinden dít vandaag leuk. Het systeem negeert de inhoud van het item volledig en kijkt puur naar overlappende gedragspatronen.

Als een groep onbekende Netflix-gebruikers net als jij genoten heeft van drie specifieke documentaires, en zij massaal naar een vierde serie kijken, is de kans groot dat jij die ook wilt zien. Je bent niet uniek (echt niet ;)) en dat is precies wat CF uitbuit.

8.2.2. Content-Based Filtering

Aan de andere kant staat **content-based filtering**. Hierbij kijkt het model niet naar andere mensen, maar puur naar de eigenschappen van de items die jij al leuk vond. Koop je op Zalando een zwarte leren jas, dan raadt het systeem je vliegensvlug een gelijkaardige jas aan op basis van metadata: kleur, materiaal, categorie.

Het systeem heeft geen andere gebruikers nodig, enkel jouw geschiedenis en de kenmerken van de items. Dat maakt het sterk bij weinig gebruikersdata, maar ook vatbaar voor een tunnelvisie: je krijgt steeds meer van hetzelfde.

8.2.3. Hybrid Filtering

Omdat beide benaderingen hun eigen blinde vlekken hebben, combineren platforms ze vrijwel altijd in een **hybrid aanpak**. Spotify mixt bijvoorbeeld content-gebaseerde audio-analyse met collaborative filtering om zijn sublieme playlists te genereren.

> ⚠ **Het cold start probleem**
>
> Collaborative filtering heeft een fundamenteel probleem: het werkt pas als er al data is. Voor een splinternieuwe gebruiker is er geen geschiedenis om buren te zoeken. Voor een splinternieuw item zijn er nul interacties, waardoor CF het nooit zal oppikken. Door terug te vallen op content-based elementen — zoals het analyseren van een audiotrack of productlabels — kan het systeem alsnog treffende matches maken.
>
> Een subtielere variant is **gray sheep**: een gebruiker met zo'n individuele smaak dat er, zelfs met genoeg data in het systeem, gewoonweg niemand anders op hem lijkt. Denk aan een kunstplatform waar elk stuk uniek is: bijna elke koper wordt er een gray sheep, want CF vindt structureel geen buren om op te steunen.

8.3. Architecturale keuzes: hoe schaal je CF?

8.3.1. Memory-based vs. model-based

Binnen collaborative filtering moet je kiezen hoe je de berekeningen structureert. Bij **memory-based** (ook wel *neighborhood-based*) filtering laad je de volledige historische interactiedata direct in het werkgeheugen om live buren te zoeken. Dit is intuïtief en altijd up-to-date, maar schaalt dramatisch slecht bij miljoenen gebruikers.

Kies je voor **model-based**, dan gebruik je technieken zoals *matrix factorisation* of *deep learning* om patronen vooraf offline te trainen in een compact wiskundig model. Dit model voorspelt vervolgens razendsnel en live de voorkeuren — zonder de volledige dataset telkens opnieuw te doorzoeken.

8.3.2. User-based vs. item-based

Een tweede keuze is de richting van de vergelijking. Bij **user-based filtering** zoek je naar gelijkaardige mensen. Bij **item-based filtering** bereken je de relaties tussen de producten zelf, op basis van hoe vaak ze samen worden geconsumeerd.

In nagenoeg alle commerciële systemen zoals Amazon, Zalando, Bol is item-based filtering vele malen stabieler. De reden? Het aantal gebruikers en hun individuele stemmingen fluctueren voortdurend. De onderlinge relatie tussen twee stabiele producten — twee specifieke boeken, twee jassen — evolueert veel trager en consistenter. Dat maakt item-gebaseerde gelijkenismatrices veel goedkoper om te onderhouden en te schalen.

8.4. De ratingmatrix en het sparsity-probleem

Waarom?

De fundamentele wiskundige structuur achter collaborative filtering is de **ratingmatrix**. Elke rij is een gebruiker, elke kolom een item, elke cel de bijbehorende rating — expliciet of impliciet.

In de praktijk is deze matrix echter nagenoeg volledig leeg. Denk aan Netflix met miljoenen gebruikers en duizenden films: een gemiddelde gebruiker bekijkt slechts een fractie van het totale aanbod. Dit fenomeen heet **sparsity** (*schaarsheid*). Een sparsity van 99% of meer is geen uitzondering, het is de norm.

Waarom is dat een probleem? Traditionele statistische berekeningen hebben voldoende overlap nodig om betrouwbare patronen te ontdekken. Lege cellen vertalen zich naar nullen, en vermenigvuldigen met nullen levert altijd nul op. Twee gebruikers die nauwelijks dezelfde items hebben bekeken, lijken dan wiskundig gezien totaal onvergelijkbaar, ook al delen ze misschien een heel specifieke niche-smaak.

De **cosine similarity** lost dit gedeeltelijk op. In plaats van te kijken naar absolute aantallen beoordelingen, meet ze de **hoek** tussen twee vectoren in een hoogdimensionale ruimte. Twee gebruikers die allebei dezelfde drie films hoog scoren, hebben een kleine hoek tussen hun vectoren — en dus een hoge gelijkenis — ook al hebben ze slechts drie items gemeenschappelijk.

$$\text{sim}(A, B) = \frac{\boldsymbol{A} \cdot \boldsymbol{B}}{\|\boldsymbol{A}\| \cdot \|\boldsymbol{B}\|}$$

Enkel de **co-rated items**, de items die beide gebruikers beoordeeld hebben worden meegenomen in de berekening. Zo vermijd je dat lege cellen de meting verstoren.

Deze cosine-similarity-berekening is exact wat je in het volgende hoofdstuk zal zien uitgewerkt in **matrix factorization**: een model-based techniek die dit sparsity-probleem structureel aanpakt in plaats van het per berekening op te lossen.

8.5. Denken & Doen

8.5.1. Kernconcepten — vat samen in eigen woorden

Probeer elk begrip in één zin uit te leggen zonder terug te bladeren. Check nadien met de theorie.

- Expliciete versus impliciete data
- Collaborative filtering
- Content-based filtering
- Hybrid filtering
- Cold start probleem en gray sheep
- Memory-based versus model-based
- User-based versus item-based, en waarom item-based vaak stabieler is
- De ratingmatrix en het sparsity-probleem
- Cosine similarity en co-rated items

8.5.2. Denken

- Je werkt aan een webshop voor tweedehands boeken die net gelanceerd is. Leg uit waarom collaborative filtering hier (nog) niet goed zal werken, en welke aanpak wél bruikbaar is vanaf dag één.
- Twee gebruikers hebben elk drie films beoordeeld, waarvan er twee overlappen. Leg uit waarom cosine similarity hen toch als gelijkaardig kan beoordelen, ook al hebben ze weinig items gemeen.
- Zou je voor een platform als Bol.com kiezen voor user-based of item-based filtering? Motiveer met een concreet argument uit de tekst.
- Een kunstplatform waar elk stuk uniek is, kampt met veel "gray sheep" gebruikers. Wat betekent dat concreet, en waarom lost meer data dit probleem niet vanzelf op?

8.5.3. Doen — sparsity oplossen met cosine similarity

Twee gebruikers beoordeelden samen slechts drie films (de co-rated items):

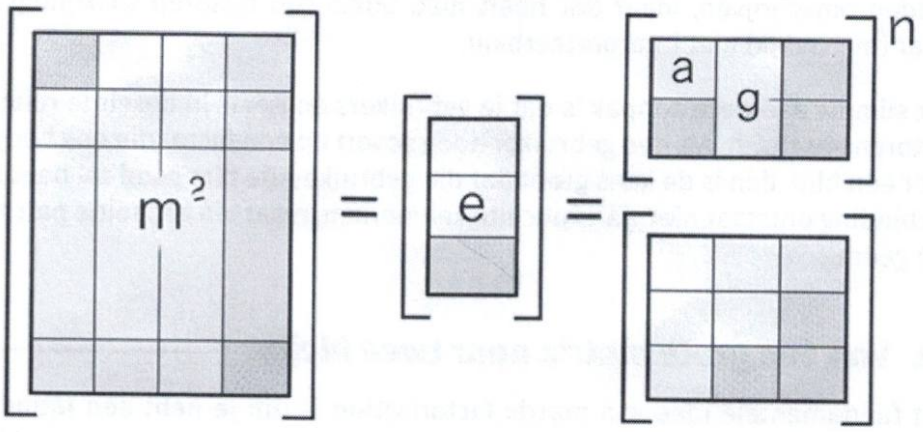
Gebruiker	Film 1	Film 2	Film 3
Gebruiker A	5	4	5
Gebruiker B	4	5	4

```
functie cosine_similarity(v1, v2):
  return dot(v1, v2) / (norm(v1) * norm(v2))
```

- Bereken de cosine similarity tussen Gebruiker A en B, enkel op basis van deze drie co-rated films.
- De rest van hun ratingmatrix (honderden andere films) staat vol lege cellen. Leg uit waarom je die cellen niet gewoon als 0 mag invullen voor deze berekening.
- Beide gebruikers hebben maar drie items gemeen op een catalogus van duizenden. Is deze similarity-score daarom minder betrouwbaar dan een score berekend op twintig gemeenschappelijke films? Verklaar.

Matrix Factorization

CHAPTER 9



In de vorige hoofdstukken heb je geleerd hoe *collaborative filtering* buren zoekt: gebruikers die op jou lijken en items vergelijkt die gelijkaardig beoordeeld worden. Dat werkt verrassend goed, tot de schaal een probleem wordt. In dit hoofdstuk gooi je die buurtbenadering overboord en vervang je ze door iets eleganter: **matrix factorization**.

9.1. Wanneer buren te duur worden

Stel je voor dat je een platform beheert met een miljoen gebruikers en honderdduizend films. Je ratingmatrix heeft dan theoretisch honderd miljard cellen maar in werkelijkheid zijn er misschien slechts tien miljoen ratings beschikbaar. Dat is minder dan 0,01% gevuld. Zo'n **sparse matrix** is het probleem van *memory-based collaborative filtering* in één oogopslag.

Elke keer dat je een aanbeveling wil maken, moet je die gigantische, bijna lege matrix doorzoeken op gelijkaardige gebruikers of items. Dat kost enorm veel geheugen én rekentijd. Bovendien: hoe meer lege cellen, hoe minder betrouwbaar de gelijkenisberekening. Je vergelijkt dan eigenlijk schaduwprofielen met elkaar.

De oplossing ligt in **dimensiereductie**: in plaats van één enorme sparse matrix bij te houden, leer je twee kleinere, dichte matrices. De ene beschrijft elke gebruiker via een handvol getallen; de andere beschrijft elk item op dezelfde manier. Die getallen zijn geen willekeurige codering maar ze vangen de essentie van smaak en inhoud.

9.2. Latente factoren — verborgen smaken

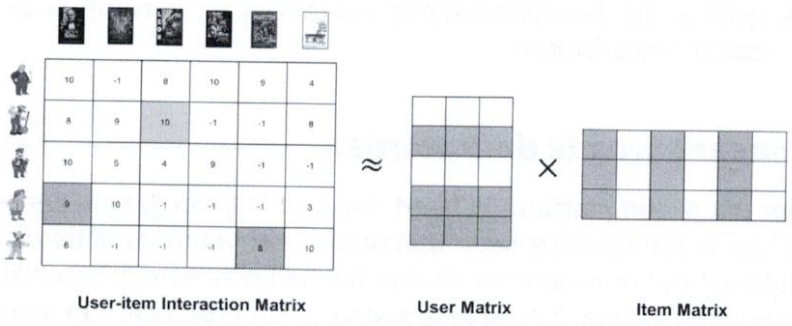
Stel dat je films beoordeelt. Wat bepaalt of je een film leuk vindt? Misschien is het genre, de regisseur, het tempo, of de hoeveelheid actie. Maar niemand heeft die eigenschappen expliciet ingegeven, het algoritme ontdekt ze zelf, puur op basis van de ratingspatronen.

Dat zijn **latente factoren**: verborgen eigenschappen die de data zelf onthult. Ze zijn "latent" omdat ze niet direct observeerbaar zijn, maar wel een verklarende rol spelen. Een factor kan overeenkomen met iets dat wij als "actievolle blockbuster" zouden omschrijven, maar dat hoeft niet. Soms zijn factoren wiskundig zinvol maar (menselijk) niet interpreteerbaar.

Het slimme aan deze aanpak is dat je gebruikers én items in dezelfde ruimte van factoren beschrijft. Als een gebruiker hoog scoort op een factor die ook hoog staat voor een film, dan is de kans groot dat die gebruiker die film goed zal beoordelen. Verbinding ontstaat niet via expliciete kenmerken, maar via gedeelde patronen in het gedrag.

9.3. Van één grote matrix naar twee kleine

Het fundamentele idee van matrix factorization is dit: je hebt een ratingmatrix R (gebruikers × items) en je wil die opsplitsen — *factorizeren* — in twee kleinere matrices U en V. U is de **gebruiker-factormatrix** (elke rij = één gebruiker beschreven door k latente factoren) en V is de **item-factormatrix** (elke kolom = één item beschreven door diezelfde k factoren).



Figuur 27: Opsplitsen van de matrix.
Bron: Medium

Als je U en V vermenigvuldigt, krijg je een benadering van R terug. De truc zit in de keuze van k: niet te groot (dan overfitte je), niet te klein (dan verlies je te veel nuance). Typisch kies je k tussen 10 en 100, afhankelijk van de dataset.

De dimensiereductie is ook de reden waarom deze aanpak schaalt. In plaats van een matrix van een miljoen bij honderdduizend op te slaan, sla je twee matrices op van respectievelijk een miljoen bij k en honderdduizend bij k. Dat is een wereld van verschil.

9.4. *Van volledige matrix naar Funk SVD*

Een klassieke manier om zo'n factorisatie te berekenen is **SVD** (*Singular Value Decomposition*): een techniek die een matrix ontbindt in gebruikers- en itemrepresentaties, gesorteerd naar hoeveel variantie in de data ze verklaren. Het probleem: SVD vereist een **volledige** matrix, maw er zijn geen lege cellen toegelaten. Maar jouw ratingmatrix is per definitie sparse. Je zou elke lege cel moeten opvullen (met nul, een gemiddelde, een schatting) voordat je SVD kan toepassen, en wat je ook kiest, je introduceert ruis in net de data die je probeert te analyseren.

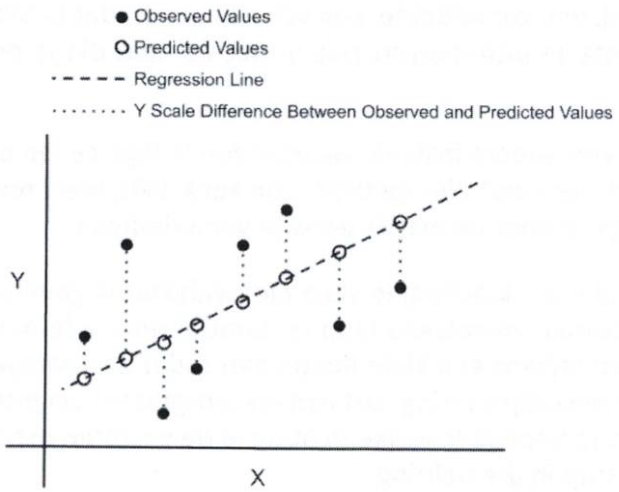
Simon Funk had een andere insteek: waarom zou je lege cellen opvullen als je ze ook gewoon kunt negeren? Zijn methode, de **Funk SVD**, leert rechtstreeks uit de bestaande ratings, zonder de matrix eerst te vervolledigen.

Het idee is simpel maar krachtig. Je start met willekeurig geïnitialiseerde gebruikers- en itemvectoren. Vervolgens loop je iteratief door alle bekende ratings en pas je de vectoren telkens een klein beetje aan zodat de voorspelling beter overeenkomt met de werkelijke rating. Dat aanpassen gebeurt via ***gradient descent***: je berekent de fout, je bepaalt in welke richting je de vectoren moet bijsturen, en je doet een kleine stap in die richting.

Na duizenden of zelfs miljoenen zulke kleine aanpassingen convergeren de vectoren naar representaties die de ratingpatronen goed weerspiegelen. De lege cellen hoef je nooit aan te raken want je leert puur uit wat je hebt.

9.5. Hoe weet je of je Funk SVD goed is?

Verberg een deel van de bekende ratings als testset, train op de rest, en vergelijk de voorspellingen met de achtergehouden werkelijke ratings. Zo simuleer je hoe het model omgaat met ratings die het nooit zag.



Figuur 28: Mean Squared Error.
Bron: The Science of ML and AI

Tijdens het trainen minimaliseer je eigenlijk al een foutfunctie: de *Mean Squared Error* (MSE). Voor elke bekende rating bereken je het verschil tussen de werkelijke rating $r_{u,i}$ en de voorspelde rating $\hat{r}_{u,i}$ — dezelfde `fout` die je verderop in Stap 2 berekent — en kwadrateer je dat verschil. Het gemiddelde daarvan, over alle N bekende ratings, is de MSE:

$$\text{MSE} = \frac{1}{N} \sum_{u,i} \left(r_{u,i} - \hat{r}_{u,i} \right)^2$$

Waarom kwadrateren? Als je de fouten zomaar optelt, heffen positieve en negatieve fouten elkaar op — een voorspelling die er +2 naast zit en één die er –2 naast zit lijken dan samen "perfect", terwijl dat allesbehalve zo is. Door te kwadreren worden alle fouten positief, én worden grote fouten zwaarder bestraft dan kleine: een fout van 2 weegt vier keer zo zwaar als een fout van 1, wat het algoritme dwingt om uitschieters serieus te nemen.

Het resultaat op je testset meet je met **RMSE** (*Root Mean Squared Error*): dezelfde MSE, maar met een vierkantswortel erover zodat de uitkomst weer op de schaal van de ratings staat. Een RMSE van 0,8 op een schaal van 1 tot 5 betekent dat je er gemiddeld minder dan één ster naast zit.

$$\text{RMSE} = \sqrt{\frac{1}{N} \sum_{u,i} \left(r_{u,i} - \hat{r}_{u,i} \right)^2}$$

RMSE heeft wel een blinde vlek: het behandelt alle ratings als even belangrijk, terwijl je systeem in de praktijk enkel een top-K aan aanbevelingen toont. Daarom kijk je ook naar **Precision@K**: van de K aanbevelingen die je toont, hoeveel zijn er daadwerkelijk relevant (bijvoorbeeld: een rating ≥ 3,5)?

$$\text{Precision@}K = \frac{\text{aantal relevante items in top-K}}{K}$$

9.6. Leren via gradient descent

Je weet ondertussen wat je zoekt: voor elke gebruiker een rijtje geheime smaakfactoren, voor elk item een rijtje geheime eigenschap-factoren, zodanig dat hun dot product zo dicht mogelijk bij de echte rating ligt. Wat je nog niet weet: hoe kom je aan die factoren? Bij een eenvoudiger model zou een formule je in één keer de optimale waarden geven, maar hier zijn de gebruikersvectoren én de itemvectoren allebei onbekend, en ze beïnvloeden elkaar in de foutfunctie — er bestaat geen gesloten formule die dat in één stap oplost. Je moet ze dus stap voor stap *leren*, op precies dezelfde manier als bij Logistic Regression in hoofdstuk 7: begin willekeurig, meet hoe fout je zit, zet een klein stapje in de richting die de fout verkleint, en herhaal dat duizenden keren.

9.6.1. Stap 1: begin met ruis

Je kent de echte factoren niet, dus je start met kleine willekeurige getallen (geen nullen, want dan blijft elke update ook nul!).

```
gebruikerFactoren = willekeurige kleine waarden (aantalGebruikers × aantalFactoren)
itemFactoren      = willekeurige kleine waarden (aantalItems × aantalFactoren)
```

Stel je een gebruiker voor met 2 factoren: `[0.10, 0.30]`, voorlopig betekenisloze getallen. Pas na training zal factor 1 misschien "houdt van actie" gaan betekenen, en factor 2 "houdt van oude films". Niemand kent die betekenis vooraf; het model ontdekt ze zelf.

9.6.2. Stap 2: meet de fout op één bekende rating

Je loopt enkel over ratings die je al kent (leeg = onbekend, daar train je niet op):

```
voorspelling = dotProduct(gebruikerFactoren[gebruiker], itemFactoren[item])
fout = werkelijkeRating - voorspelling
```

De voorspelling is een **dot product**: het meet hoe gelijkgericht twee vectoren zijn. Scoren gebruiker en item hoog op dezelfde factoren, dan is hun dot product groot — precies wat je wil, want als een gebruiker van actievolle films houdt én een film sterk actiegericht is, moet de voorspelde rating hoog zijn. Concreet: gebruiker heeft `[0.10, 0.30]`, item heeft `[0.20, 0.40]`. De voorspelling is (0.10 × 0.20) + (0.30 × 0.40) = 0.02 + 0.12 = 0.14. De echte rating was 5. De fout is dus 5 - 0.14 = 4.86 — enorm, want de factoren zijn nog puur ruis. Dat is normaal aan het begin van de training.

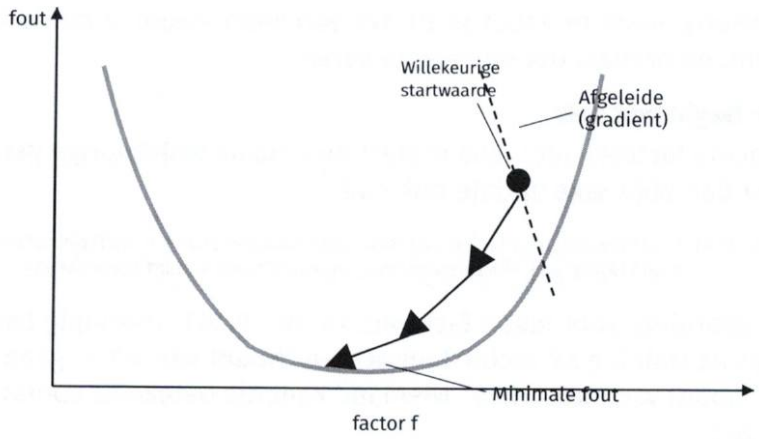
9.6.3. Stap 3: verdeel die fout over de factoren

Dit is de kern van de sectie, en het patroon ken je intussen al uit Logistic Regression:

```
voor elke factor k van 1 tot aantalFactoren:
    gebruikerFactoren[gebruiker][k] = gebruikerFactoren[gebruiker][k]
```

```
                                    + leerSnelheid * fout * itemFactoren[item][k]
    itemFactoren[item][k] = itemFactoren[item][k]
                          + leerSnelheid * fout * gebruikerFactoren[gebruiker][k]
```

> **Waarom deze twee regels, en niet zomaar willekeurig bijstellen?**
>
> Dit is **gradient descent** in actie: je berekent de afgeleide van de fout ten opzichte van elke factor, en zet een stap in de richting die de fout verkleint. Die afgeleide vertelt je hoe gevoelig de fout is voor een kleine aanpassing van die factor en die gevoeligheid blijkt hier simpelweg de waarde van de andere vector te zijn. Elke gebruikersfactor wordt dus bijgesteld in verhouding tot de *itemfactor* waarmee hij vermenigvuldigd werd — en omgekeerd. Dat is logisch: als factor 1 van het item groot was, dan had factor 1 van de gebruiker een grote invloed op de voorspelling, en verdient hij dus een groter deel van de correctie. Grote invloed = grote aanpassing.



Figuur 29: Gradient Descent.

Met leerSnelheid = 0.01: `gebruikerFactoren[0]` wordt $0.10 + 0.01 \times 4.86 \times 0.20 = 0.10 + 0.0097 \approx 0.11$. Een klein stapje, in de juiste richting. Herhaal dat duizenden keren, over alle bekende ratings, en die kleine stapjes stapelen zich op tot factoren die echt iets betekenen.

9.6.4. Stap 4: alles samen

```
functie funkSVD(bekendeRatings, aantalFactoren, leerSnelheid, aantalIteraties):
    gebruikerFactoren = willekeurige kleine waarden (aantalGebruikers × aantalFactoren)
    itemFactoren      = willekeurige kleine waarden (aantalItems × aantalFactoren)

    voor iteratie van 1 tot aantalIteraties:
        voor elke (gebruiker, item, werkelijkeRating) in bekendeRatings:

            voorspelling = dotProduct(gebruikerFactoren[gebruiker], itemFactoren[item])
            fout = werkelijkeRating - voorspelling

            voor elke factor k van 1 tot aantalFactoren:
                gebruikerFactoren[gebruiker][k] = gebruikerFactoren[gebruiker][k]
                                                + leerSnelheid * fout * itemFactoren[item]
[k]
```

```
                itemFactoren[item][k] = itemFactoren[item][k]
                                        + leerSnelheid * fout * gebruikerFactoren[gebruiker]
[k]

    geef gebruikerFactoren, itemFactoren terug


functie voorspelRating(gebruikerFactoren, itemFactoren, gebruiker, item):
    geef dotProduct(gebruikerFactoren[gebruiker], itemFactoren[item]) terug
```

Na `aantalIteraties` keer over alle bekende ratings lopen, dalen de fouten geleidelijk, niet enkel op de ratings waarop getraind is, maar (als het goed gaat) ook op de lege cellen. Dat is precies waar de aanbeveling vandaan komt: `voorspelRating()` werkt voor élk gebruiker-item-paar, ook eentje dat nooit een echte rating had.

> **Herhaling is de beste leerschool!**
>
> Dit is de tweede keer dat je exact dezelfde vier stappen ziet: begin willekeurig → voorspel → meet de fout → zet een klein stapje in de richting die de fout verkleint. Logistic Regression deed het met gewichten en een bias. Funk SVD doet het met twee factor-matrices tegelijk. In hoofdstuk 10 zie je het een derde keer, met nog meer gewichten, verspreid over meerdere lagen. De wiskunde wordt elke keer een beetje uitgebreider — het idee erachter blijft identiek.

9.7. Denken & Doen

9.7.1. Kernconcepten — vat samen in eigen woorden

Probeer elk begrip in één zin uit te leggen zonder terug te bladeren. Check nadien met de theorie.

- Sparse matrix en dimensiereductie
- Latente factoren
- Gebruiker-factormatrix en item-factormatrix
- SVD versus Funk SVD
- Gradient descent
- MSE, RMSE en Precision
- Leersnelheid (learning rate)

9.7.2. Denken

- Latente factoren worden door het algoritme zelf ontdekt, zonder dat iemand ze benoemt. Wanneer lijkt zo'n factor toch te matchen met iets herkenbaars zoals "genre", en wanneer is dat toeval?
- Klassieke SVD vereist een volledige matrix, terwijl je ratingmatrix bijna helemaal leeg is. Hoe lost Funk SVD dat op, en wat is de prijs die je daarvoor betaalt (bv. geen gesloten formule meer)?
- Wat zie je gebeuren aan het verloop van de RMSE als je de leersnelheid veel te groot kiest? En wat als ze veel te klein is?
- RMSE behandelt alle ratings als even belangrijk, ook al toont je systeem in de praktijk maar een top-K aan de gebruiker. Wat vertelt Precision dan dat RMSE niet vertelt, en wat zou je missen als je enkel op RMSE zou optimaliseren?

9.7.3. Doen — één stap Funk SVD met de hand

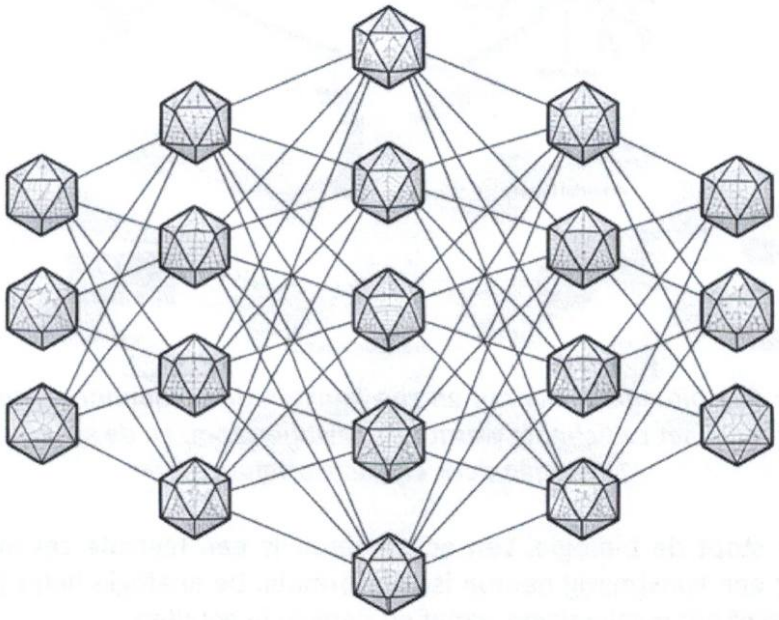
$\boldsymbol{u}_1 = (0.5, 0.8)$, $\boldsymbol{v}_A = (0.6, 0.4)$, $\alpha = 0.1$, $r_{1,A} = 5$

```
functie update_vector(u, v, fout, alpha):
  return u + alpha * fout * v
```

- Bereken de voorspelde rating $\hat{r}_{1,A} = \boldsymbol{u}_1 \cdot \boldsymbol{v}_A$.
- Bereken de fout $e = r_{1,A} - \hat{r}_{1,A}$.
- Werk de update van $\boldsymbol{u}_1$ uit met de pseudocode hierboven, en leg uit waarom $\boldsymbol{v}_A$ op exact dezelfde manier wordt bijgesteld, maar dan met $\boldsymbol{u}_1$ als factor.

Neurale netwerken

CHAPTER 10



Tot nu toe heb je algoritmes gezien die je zelf kon volgen, bij een neuraal netwerk verandert er iets. Je bouwt niet langer de regels, je bouwt een structuur die de regels zélf uit de data haalt.

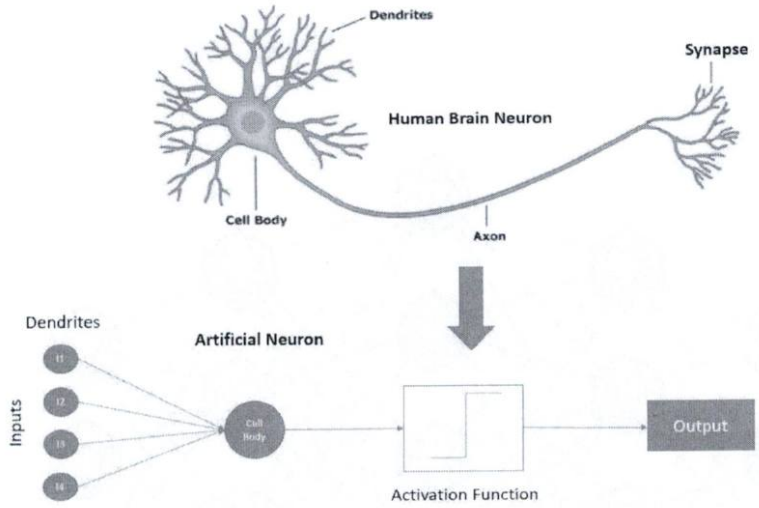
Dat idee komt niet uit het niets. Het is geleend van de biologie, en die analogie is een handige opstap. Maar je moet ze snel loslaten, want een neuraal netwerk is uiteindelijk gewoon wiskunde: optellen, vermenigvuldigen en één slimme knik.

In dit hoofdstuk bouw je het idee laag per laag op. Van één kunstmatig neuron naar een volledig netwerk, van een berekening vooruit naar een correctie achteruit. En op het einde stel je de eerlijke vraag: heb je zo'n netwerk eigenlijk wel nodig?

10.1. Van hersencel naar wiskundig model

Je hersenen zitten vol neuronen. Elk neuron krijgt signalen binnen via verbindingen die **synapsen** heten, kleine spleetjes waarlangs een prikkel van de ene cel naar de andere springt. Krijgt een neuron genoeg prikkeling binnen, dan "vuurt" het en geeft het zelf een signaal door.

Dat is precies de metafoor waarmee een kunstmatig neuraal netwerk werkt. Het probeert na te bootsen hoe een echt brein informatie verwerkt om tot een actie te komen. Elk knooppunt is een neuron, elke verbinding een synaps.



Figuur 31: Een biologisch neuron en zijn kunstmatige tegenhanger: dendrieten worden inputs, het cellichaam wordt een gewogen som, en de synaps wordt vervangen door een activatiefunctie.

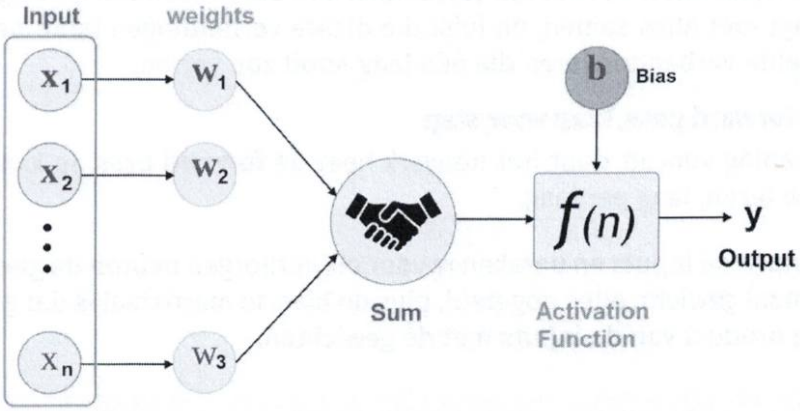
Maar hier stopt de biologie. Een echt neuron is een levende cel met chemie en timing; een kunstmatig neuron is een formule. De analogie helpt je het **idee** snappen, niet het mechanisme, vanaf nu denk je in getallen.

10.1.1. Het perceptron: gewichten, bias en activatie

Het eenvoudigste kunstmatige neuron heet een **perceptron**. Het krijgt een paar inputwaarden binnen, bijvoorbeeld x_1 en x_2, en elke input reist over een verbinding met een eigen **gewicht** (w_1, w_2).

Het neuron doet drie dingen. Eerst vermenigvuldigt het elke input met zijn gewicht en telt het alles op. Daarna telt het er een **bias** b bij, een extra getalletje dat bepaalt hoe makkelijk het neuron in beweging komt. Die gewogen som noemen we z.

Vervolgens gaat z door een **activatiefunctie**. Die functie beslist wat het neuron doorgeeft: vuurt het stevig, zwakjes, of helemaal niet? Zonder die functie zou een neuron alleen maar rechttoe-rechtaan optellen, en dat blijkt straks een probleem.



Figuur 32: Forward pass.
Bron: EJable

Bij de start zijn de gewichten gewoon willekeurige kleine getallen, ergens tussen -1 en 1. Ze betekenen nog niets. Pas door te trainen krijgen ze de waarde die het netwerk nuttig maakt.

10.1.2. Drie activatiefuncties, drie karakters

De **sigmoid** duwt elke waarde naar een getal tussen 0 en 1. Dat maakt ze ideaal wanneer je iets wil uitdrukken als een kans: hoe waarschijnlijk is het dat deze foto een kat is? Heel veel klassieke netwerken gebruiken haar net daarom.

Toch heeft de sigmoid een zwakte. Bij grote positieve of negatieve inputs wordt haar helling flinterdun. Het netwerk "voelt" dan nauwelijks nog verschil, en leert traag. Voor diepe netwerken is dat lastig.

Daarom grijp je in verborgen lagen vaak naar **ReLU**. Die functie is bruut simpel: negatieve waarden worden 0, positieve waarden blijven zichzelf. Ze rekent snel en behoudt haar helling voor positieve waarden, waardoor diepe netwerken vlotter leren.

Tanh lijkt op de sigmoid, maar loopt van -1 tot 1 en ligt gecentreerd rond nul. Dat gecentreerde helpt: negatieve en positieve signalen blijven bestaan in plaats van allemaal richting één kant geduwd te worden. Als vuistregel: sigmoid voor een kans aan de uitgang, ReLU als werkpaard in de verborgen lagen, tanh wanneer je gecentreerde output wil.

10.2. Het meerlaags netwerk

Eén perceptron kan niet veel. Zet je er echter veel naast en achter elkaar, dan krijg je een **meerlaags netwerk** (MLP). De neuronen zitten dan in een gewogen graaf: elk knooppunt een neuron, elke verbinding een synaps met een gewicht.

Zo'n netwerk heeft altijd drie soorten lagen. De **inputlaag** neemt je ruwe data aan. De **verborgen lagen** (er kunnen er meerdere zijn) verwerken die data stap voor stap. De **outputlaag** levert het uiteindelijke resultaat.

In een klassiek ANN is elk neuron verbonden met élk neuron van de volgende laag. Alles hangt met alles samen, en juist die dichte verbindingen laten het netwerk ingewikkelde verbanden leren die één laag nooit zou vatten.

10.2.1. De forward pass, stap voor stap

De berekening vooruit door het netwerk heet de **forward pass**. Je kan ze in een vast ritme lezen, laag per laag.

Eerst neem je de inputs en bereken je voor elk verborgen neuron de gewogen som *z*: input maal gewicht, alles opgeteld, plus de bias. In matrixtaal is dat gewoon het inwendig product van de inputs met de gewichten.

> **Forward pass als matrixvermenigvuldiging**
>
> Met twee inputs en drie verborgen neuronen wordt de forward pass:
>
> $$\boldsymbol{z} = \boldsymbol{x} \cdot \boldsymbol{W} + \boldsymbol{b}$$
>
> waarbij $\boldsymbol{x}$ een rijvector is met je inputs, $\boldsymbol{W}$ een matrix met één kolom per neuron (elke kolom bevat de gewichten van dat neuron), en $\boldsymbol{b}$ de bias per neuron.
>
> Concreet, met $x = [0.5, 0.8]$ en drie neuronen:
>
> $$(0.5 \;\; 0.8) \cdot \begin{pmatrix} 0.2 & -0.1 & 0.4 \\ 0.3 & 0.6 & -0.2 \end{pmatrix} + (0.1 \;\; 0.0 \;\; -0.1) = (0.44 \;\; 0.43 \;\; -0.06)$$
>
> Dat resultaat is meteen *z* voor alle drie de neuronen tegelijk. In code is dit precies wat `Matrix.DotProduct(input, gewichten)` doet: geen drie aparte lussen over gewichten, maar één matrixvermenigvuldiging die alle neuronen van de laag in één keer berekent.

Daarna stuur je elke *z* door de activatiefunctie. Zo krijgt elk verborgen neuron zijn output, een getal tussen 0 en 1 als je sigmoid gebruikt. Die outputs worden meteen de nieuwe "inputs" voor de volgende laag.

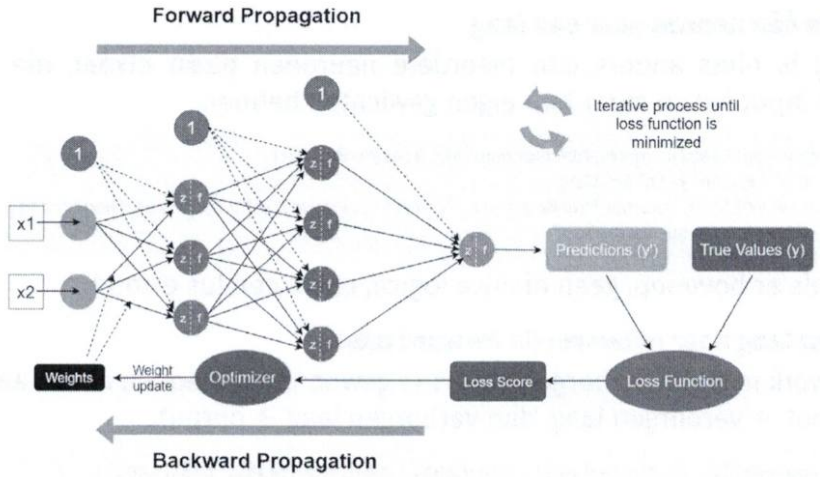
Je herhaalt dat spel tot je bij de outputlaag komt. Daar bereken je opnieuw een gewogen som en een activatie, en het getal dat eruit rolt is de voorspelling van het netwerk. Meer is een forward pass niet: optellen, activeren, doorgeven.

10.3. Leren van je fouten

Een netwerk met willekeurige gewichten geeft willekeurige antwoorden. Om te leren heb je twee dingen nodig: trainingsdata mét de juiste antwoorden, en een manier om te meten hoe fout je zit.

Die maat is de **verliesfunctie**. In de kern is ze de vergelijking tussen wat het netwerk voorspelde en wat het antwoord had moeten zijn. Hoe groter dat verschil, hoe groter het verlies. Trainen betekent: dit verlies zo klein mogelijk maken.

Maar wéten dat je fout zit is niet genoeg. Je moet weten **welke** gewichten schuld hebben, en hoeveel. Daar komt **backpropagation** binnen, "backward propagation of errors".



Figuur 33: De volledige leercyclus: een forward pass genereert een voorspelling, de verliesfunctie meet hoe ver die van de werkelijke waarde afligt, en de optimizer stuurt via backward propagation de gewichten bij. Dit herhaalt zich tot het verlies minimaal is..
Bron: Medium

Backpropagation berekent hoe gevoelig de fout is voor elk gewicht, oftewel de **gradiënt** van de fout ten opzichte van de gewichten. De gradiënt is niets anders dan de mate waarin een functie verandert, de helling.

Stel je de fout voor als een berg waar je vanaf wil dalen. De gradiënt vertelt je in welke richting het steilst naar beneden gaat. Bij elk gewicht neem je een klein stapje in die richting: dat is **gradient descent**. Doe je dat duizend keer, dan zakt de fout stelselmatig.

De naam "backprop" verwijst naar de volgorde. Je begint bij de fout aan de uitgang en werkt terug naar binnen, laag per laag. Onderweg deel je de schuld voor de fout uit over alle gewichten die eraan meewerkten. Elk gewicht krijgt een correctie op maat van zijn aandeel.

Na genoeg herhalingen zijn de gewichten niet langer willekeurig, maar afgestemd op de data. Op dat moment is het trainen klaar, en heeft je netwerk een patroon geleerd.

10.4. *Van één neuron naar een netwerk*

10.4.0a. Van één neuron (dit ken je al)

```
functie voorspelEenNeuron(x, gewichten, bias):
    z = bias
    voor elk input i:
```

```
        z = z + gewichten[i] * x[i]
    geef sigmoid(z) terug
```

Zie je het? Dit is exact dezelfde functie als bij Logistic Regression in hoofdstuk 7. Eén neuron = Logistic Regression. Alles wat hierna komt, is dit basisidee herhalen en combineren.

10.4.1. Van één neuron naar een laag

Een laag is niets anders dan meerdere neuronen naast elkaar, die elk naar dezelfde input kijken maar hun eigen gewichten hebben.

```
functie voorspelLaag(x, gewichtenPerNeuron, biasPerNeuron):
    voor elk neuron j in de laag:
        uitvoer[j] = voorspelEenNeuron(x, gewichtenPerNeuron[j], biasPerNeuron[j])
    geef uitvoer terug
```

Drie regels er bovenop, geen nieuwe logica, enkel een lus errond.

10.4.2. Van laag naar netwerk: de forward pass

Een netwerk met een verborgen laag is `()` gewoon twee keer na elkaar aanroepen: eerst input → verborgen laag, dan verborgen laag → output.

```
functie voorspel(x, gewichtenLaag1, biasLaag1, gewichtenLaag2, biasLaag2):
    verborgenLaag = voorspelLaag(x, gewichtenLaag1, biasLaag1)
    output        = voorspelLaag(verborgenLaag, gewichtenLaag2, biasLaag2)
    geef output, verborgenLaag terug   // verborgenLaag heb je nodig bij het trainen
```

10.4.3. Trainen: backpropagation

Voorspellen is dus niets nieuws meer. Het enige echt nieuwe idee is: hoe verdeel je de fout terug over alle gewichten die eraan bijdroegen? Dat is backpropagation: je begint bij de fout aan de uitgang, en werkt laag per laag terug naar binnen.

```
functie train(trainingdata, leerSnelheid, aantalStappen):
    initialiseer gewichtenLaag1, biasLaag1, gewichtenLaag2, biasLaag2 met kleine willekeurige
waarden

    herhaal aantalStappen keer:
        voor elk (x, gewensteOutput) in trainingdata:

            // Forward pass
            output, verborgenLaag = voorspel(x, gewichtenLaag1, biasLaag1, gewichtenLaag2,
biasLaag2)

            // Stap 1: fout aan de uitgang
            foutOutput = gewensteOutput - output
            gradientOutput = foutOutput * sigmoidAfgeleide(output)

            // Stap 2: verdeel de fout terug naar de verborgen laag
            voor elk neuron j in verborgenLaag:
                gradientVerborgen[j] = gradientOutput * gewichtenLaag2[j] *
sigmoidAfgeleide(verborgenLaag[j])

            // Stap 3: pas laag 2 aan (verborgen → output)
            voor elk neuron j in verborgenLaag:
                gewichtenLaag2[j] = gewichtenLaag2[j] + leerSnelheid * gradientOutput *
verborgenLaag[j]

            // Stap 4: pas laag 1 aan (input → verborgen)
            voor elk neuron j in verborgenLaag:
                voor elk input i in x:
                    gewichtenLaag1[j][i] = gewichtenLaag1[j][i] + leerSnelheid *
gradientVerborgen[j] * x[i]

    geef gewichtenLaag1, biasLaag1, gewichtenLaag2, biasLaag2 terug
```

```
functie sigmoidAfgeleide(sigmoidWaarde):
    geef sigmoidWaarde * (1 - sigmoidWaarde) terug
```

Merk op: stap 3 en stap 4 gebruiken dezelfde update-regel als je al kent uit Logistic Regression en Funk SVD: gewicht + leerSnelheid **gradient** input. Enkel de gradient zelf wordt in stap 2 eerst "doorgegeven" naar de vorige laag. Meer lagen toevoegen betekent gewoon: herhaal stap 2 nog een keer per extra laag, telkens één stap verder terug. De code wordt dan wat langer, maar het principe verandert niet.

Hetzelfde, maar als matrixoperaties

De pseudocode hierboven loopt over neuronen met `voor elk`-lussen, wat helder is om te lezen. In de praktijk gebeurt dit als matrixoperaties, in één keer voor alle neuronen van een laag samen:

1. **Voorspellen:** z = input $\cdot$ W (matrixvermenigvuldiging), dan sigmoid toepassen op elk element
2. **Fout:** error = gewenste output - voorspelling (element per element)
3. **Gradiënt:** error × sigmoid'(voorspelling) — hier **geen** matrixvermenigvuldiging, gewoon element-per-element vermenigvuldigen
4. **Aanpassing:** W^T $\cdot$ gradiënt (de getransponeerde input maal de gradiënt)
5. **Update:** W = W + aanpassing

Dezelfde vier stappen als in de `voor elk`-lussen, maar een GPU kan een matrixvermenigvuldiging voor duizenden neuronen tegelijk berekenen. Dat is precies waarom neurale netwerken in de praktijk op matrices draaien, niet op lussen.

10.5. *Wanneer kies je een NN boven klassieke ML?*

Een neuraal netwerk klinkt indrukwekkend, maar het is niet altijd het juiste gereedschap. Voor kleine, overzichtelijke datasets doen een beslisboom of logistic regression het vaak beter, én je kan hun beslissing nog uitleggen.

Een NN verdient zijn plaats wanneer het verband tussen input en output te grillig is voor een rechte lijn of een handjevol regels. Denk aan beelden, geluid of taal, waar de patronen in lagen van complexiteit zitten. Precies daarin blinken netwerken met meerdere lagen uit.

De prijs is echter reëel: je hebt veel data nodig, veel rekenkracht, en je verliest transparantie. Een netwerk dat "werkt" kan je zelden precies uitleggen. Kies dus een NN wanneer de complexiteit van je probleem het rechtvaardigt, niet omdat het modieus is.

10.6. Denken & Doen

10.6.1. Kernconcepten — vat samen in eigen woorden

Probeer elk begrip in één zin uit te leggen zonder terug te bladeren. Check nadien met de theorie.

- Perceptron: gewichten, bias en activatie
- Activatiefunctie (sigmoid, ReLU, tanh)
- Inputlaag, verborgen laag en outputlaag
- Forward pass
- Verliesfunctie
- Backpropagation
- Gradiënt en gradient descent
- Eén neuron versus Logistic Regression

10.6.2. Denken

- Wat gebeurt er wiskundig als je in alle neuronen van een netwerk van tien lagen de activatiefunctie gewoon weglaat? Waarom worden die tien lagen dan zinloos?
- Vergelijk sigmoid, ReLU en tanh. Voor welke plek in het netwerk (verborgen laag of output) zou je elk kiezen, en welk nadeel van de sigmoid maakt ReLU aantrekkelijk in diepe netwerken?
- Waarom zou een netwerk moeilijker leren als alle gewichten bij de start exact hetzelfde getal hebben, in plaats van willekeurige kleine getallen?
- Eén neuron met een sigmoid-activatie is wiskundig identiek aan Logistic Regression uit hoofdstuk 7. Wat voegt een heel netwerk dan nog toe, en wanneer volstaat die ene laag eigenlijk gewoon?

10.6.3. Doen — forward pass met de hand

Klein netwerk: 2 inputs, 2 verborgen neuronen (sigmoid), 1 outputneuron (sigmoid).

$x_1 = 1$, $x_2 = 2$. Naar h_1: $w_{11} = 0.5$, $w_{21} = 0$, $b_{h_1} = -0.5$. Naar h_2: $w_{12} = -0.5$, $w_{22} = 0.5$, $b_{h_2} = 0$. Naar output: $w_{o_1} = 1$, $w_{o_2} = -1$, $b_o = 0$. Target $t = 1$.

z	-1	-0.5	-0.12	0	0.5	1	2
$\sigma(z)$	0.27	0.38	0.47	0.50	0.62	0.73	0.88

```
functie forward(x1, x2):
  h1 = sigmoid(w11x1 + w21x2 + bh1)
  h2 = sigmoid(w12x1 + w22x2 + bh2)
  return sigmoid(wo1h1 + wo2h2 + bo)
```

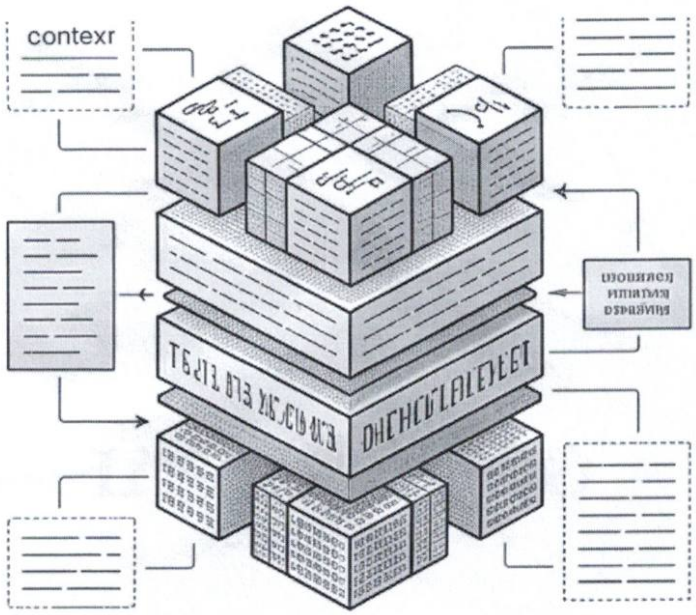
- Bereken z en de sigmoid-output van h_1 en h_2.
- Bereken z en de sigmoid-output van de outputnode.
- Bereken de fout t.o.v. $t = 1$ en het verlies $\frac{1}{2}(\text{fout})^2$.

Generatieve AI

11. Large Language Models
12. AI-agents

Large Language Models

CHAPTER 11

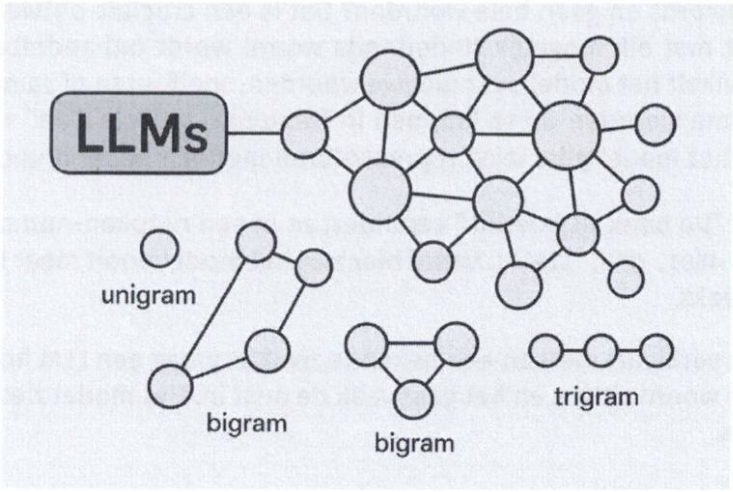


Wanneer je met een chatbot praat, lijkt het alsof er iemand aan de andere kant zit die je "begrijpt". Dat is een illusie, maar wel een indrukwekkend goed gebouwde illusie. Onder de motorkap zit geen begrip, maar een gigantische wiskundige voorspellingsmachine die telkens opnieuw één vraag beantwoordt: wat is het meest waarschijnlijke volgende *token*?

In dit hoofdstuk open je die motorkap. Je ziet hoe tekst wordt omgezet in getallen, hoe betekenis een richting wordt in een hoge-dimensionale ruimte, en hoe het *self-attention* mechanisme context opbouwt. Onderweg kom je oude bekenden tegen: afstandsmaten, kansverdelingen en latente factoren. Een LLM is geen magie, het is een stapeling van technieken die je grotendeels al kent.

11.1. Van tellen naar voorspellen

Hoe raad je het volgende woord in een zin? De oudste aanpak was puur tellen. Klassieke **n-gram modellen** berekenden hoe vaak een opeenvolging van n woorden voorkwam in een groot tekstcorpus. De naam verraadt meteen hoeveel woorden het model bekijkt: bij $n = 1$ tel je losse woorden (een **unigram**), bij $n = 2$ paren van opeenvolgende woorden (een **bigram**), en bij $n = 3$ drietallen (een **trigram**). Zag het model vaak de opeenvolging "de bakker bakt" (een trigram), dan voorspelde het na "de bakker" het woord "bakt".



Figuur 35: Een LLM houdt de volledige context tegelijk in het oog; een n-gram model kijkt slechts naar een vast, beperkt aantal voorgaande woorden.
Bron: Subhash Nair, LinkedIn-post over LLM's, ngrams en taalmodellen.

Dat werkt, tot op zekere hoogte. Maar een *trigram*-model dat enkel naar de twee voorgaande woorden kijkt, is de rest van de zin meteen vergeten. De context van "De vrouw die gisteren de marathon won, zei dat ze..." past simpelweg niet in twee woorden. Bovendien: een woordcombinatie die nooit in het corpus stond, krijgt kans nul — hoe logisch ze ook is.

De doorbraak kwam met **neurale taalmodellen**. In plaats van te tellen, leren die met diepe neurale netwerken (hoofdstuk over neurale netwerken) patronen herkennen in veel langere tekstreeksen. Het model onthoudt geen letterlijke woordcombinaties meer, maar leert abstracte structuren: grammatica, stijl, en zelfs feitenkennis, allemaal geperst in de gewichten van het netwerk.

> **De rode draad**
>
> Een LLM doet in essentie niets anders dan wat je in dit vak al vaker zag: een functie leren die input omzet in een kansverdeling over mogelijke outputs. Het verschil zit in de schaal — miljarden parameters — en in de slimme architectuur die context vasthoudt.

11.2. *Tokenisatie: tekst in hapklare stukjes*

Een computer rekent met getallen, niet met letters. Voordat een model iets met jouw invoer kan doen, moet de tekst dus worden omgezet naar een numerieke vorm. Die eerste stap heet **tokenisatie**.

Definition 11.1. *Een* token *is de kleinste teksteenheid waarmee een taalmodel rekent. Moderne tokenizers splitsen tekst niet in volledige woorden, maar in* subwords*: veelvoorkomende stukjes van woorden. Alle unieke tokens samen vormen het* ***vocabularium*** *van het model: een vaste lijst van tienduizenden tekstfragmenten, elk met een uniek identificatienummer.*

Waarom subwords en geen hele woorden? Dat is een cruciale ontwerpkeuze. Een woordenlijst met élk mogelijk Nederlands woord wordt onhandelbaar groot en dan nog struikelt het model over nieuwe woorden, spelfouten of samenstellingen. Door zeldzame woorden op te knippen in frequente stukjes ("on" + "voorstel" + "baar") kan het model **elke** tekst representeren met een beperkt vocabularium.

Jouw invoer "De bank zit heerlijk" verandert zo in een rij token-nummers, bijvoorbeeld `[318, 4501, 992, 12074]`. Vanaf hier ziet het model nooit meer letters, enkel deze cijferreeks.

Remark. Dit verklaart meteen een bekende zwakte: vraag een LLM hoeveel letters "r" er in een woord zitten, en het gaat vaak de mist in. Het model ziet geen letters, maar tokens.

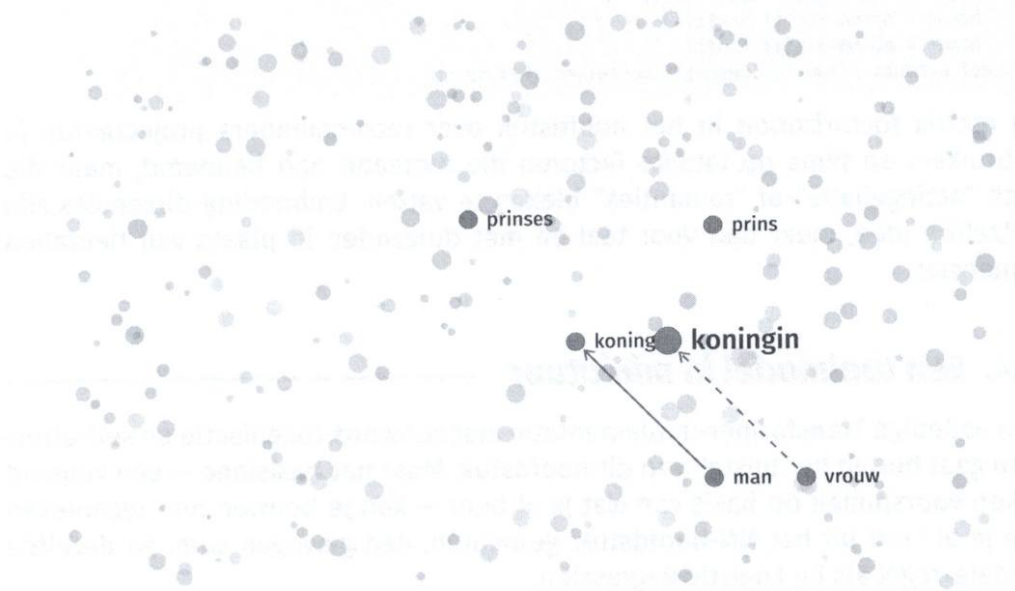
11.3. Embeddings: betekenis als richting

Een token-nummer zoals `4501` is nog steeds betekenisloos, het is een indexnummer, geen betekenis. De echte magie gebeurt in de **embedding-laag**: die zet elk token om in een **vector**, een punt in een wiskundige ruimte van vaak duizenden dimensies.

In die ruimte betekenen de losse getallen niets, maar hun onderlinge verhoudingen alles. Dit noemen we **semantische nabijheid**: woorden die een gelijkaardige betekenis hebben of vaak in dezelfde context opduiken, liggen dicht bij elkaar. "Hond" en "kat" zijn buren; "hond" en "hypotheek" wonen in andere wijken.

Een beroemd voorbeeld is dat je in deze ruimte kunt *rekenen* met betekenis:

$$\text{vector(koning)} - \text{vector(man)} + \text{vector(vrouw)} \approx \text{vector(koningin)}$$



Figuur 36: Semantisch verwante woorden liggen dicht bij elkaar in de ruimte.
Geïnspireerd op TensorFlow Embedding Projector.

De richting "van man naar vrouw" is blijkbaar een consistente as in de ruimte. Trek je die as door vanaf "koning", dan land je in de buurt van "koningin". Niemand heeft dat geprogrammeerd, het model heeft die structuur zelf ontdekt door miljarden zinnen te lezen.

Cosine similarity en semantische betekenis

Eerder in de cursus berekende je de gelijkenis tussen datapunten met afstandsmaten zoals de Euclidische afstand. Bij *embeddings* gebruiken we liever de **cosine similarity**: niet de afstand tussen de eindpunten, maar de **hoek** tussen twee vectoren.

$$\text{sim}(A, B) = \cos(\theta) = \frac{A \cdot B}{\|A\| \cdot \|B\|}$$

Waarom de hoek en niet de afstand? De *lengte* van een embedding-vector wordt mee bepaald door bijzaken, zoals hoe vaak een token in de trainingsdata voorkwam. De *richting* draagt de betekenis. Door te delen door beide normen $\|A\|$ en $\|B\|$ neutraliseer je de lengte volledig: wat overblijft is puur richting.

In code doe je het dan zo

```
functie cosineSimilarity(a, b):
  product = 0
  normA = 0
  normB = 0
  voor elk dimensie i:
```

```
    product = product + a[i] * b[i]
    normA = normA + a[i] * a[i]
    normB = normB + b[i] * b[i]
  geef product / (wortel(normA) * wortel(normB)) terug
```

Bij *matrix factorization* in het hoofdstuk over recommenders projecteerde je gebruikers en films op latente factoren die niemand had benoemd, maar die toch "actiegehalte" of "romantiek" bleken te vatten. Embedding-dimensies zijn hetzelfde idee, maar dan voor taal en met duizenden in plaats van tientallen dimensies.

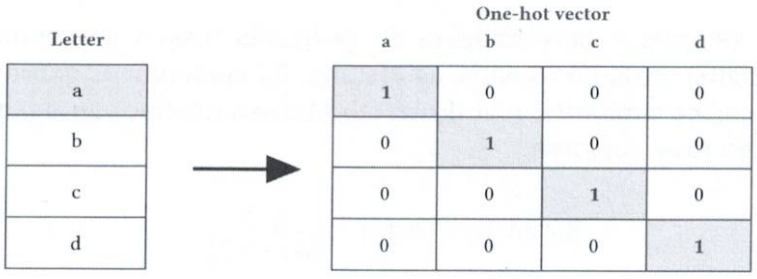
11.4. Een taalmodel in miniatuur

Een volledige Transformer-implementatie met subword-tokenisatie en self-attention gaat buiten het bestek van dit hoofdstuk. Maar het basisidee — een volgend teken voorspellen op basis van wat je al hebt — kan je bouwen met technieken die je al kent uit het NN-hoofdstuk: gewichten, een gewogen som, en dezelfde update-regel als bij Logistic Regression.

Voor we de code bekijken, eerst even **one-hot encoding** zelf: hoe zet je een letter om in getallen waarmee een netwerk kan rekenen?

Definition 11.2. ***One-hot encoding*** *zet een categorie (hier: een letter) om in een vector die overal* 0 *is, behalve op precies één plaats, waar een* 1 *staat. Die ene* 1 *zit op de positie die overeenkomt met die specifieke letter.*

Bij een alfabet van 26 letters krijgt elke letter dus een vector van lengte 26. De letter 'a' wordt $(1, 0, 0, \ldots, 0)$, met de 1 op de eerste plaats. De letter 'b' wordt $(0, 1, 0, \ldots, 0)$, met de 1 op de tweede plaats. Enzovoort.



Figuur 37: One-hot encoding: elke letter krijgt een vector met exact één 1'tje op haar eigen positie, en verder enkel nullen. Bij een alfabet van 26 letters heeft elke vector dus lengte 26.

Waarom niet gewoon een letter als getal doorgeven (bv. 'a' = 1, 'b' = 2, 'c' = 3)? Omdat dat een **ordening** zou suggereren die er niet is — het netwerk zou kunnen denken dat 'c' twee keer zo veel is als 'a', of dat 'b' "tussen" 'a' en 'c' ligt qua betekenis. One-hot encoding vermijdt dat: alle letters staan even ver van elkaar, zonder enige impliciete rangorde.

⚠ De beperking van one-hot encoding

One-hot vectoren zeggen niets over **gelijkenis**. De vector voor 'a' ligt in deze ruimte even ver van 'b' als van 'z' — er zit geen enkele betekenisvolle structuur in. Dat is precies waarom de embedding-laag zo waardevol is: die zet deze betekenisloze one-hot vector om in een kleinere, dichte vector die wél gelijkenis en betekenis vastlegt.

```
functie oneHotEncoding(letter):
    vector = lijst van 26 nullen
    index = positie van letter in het alfabet
    vector[index] = 1
    geef vector terug


functie train(tekst, embeddingGrootte, leerSnelheid, aantalEpochs):
    initialiseer embeddingMatrix (26 × embeddingGrootte) met kleine willekeurige waarden
    initialiseer outputGewichten (embeddingGrootte × 26) met kleine willekeurige waarden

    herhaal aantalEpochs keer:
        voor elk paar (huidigeLetter, volgendeLetter) in tekst:
            input = oneHotEncoding(huidigeLetter)
            gewenst = oneHotEncoding(volgendeLetter)

            // Forward pass
            embedding = input · embeddingMatrix        // one-hot → dichte vector
            voorspelling = embedding · outputGewichten     // dichte vector → letterkansen

            // Fout aan de uitgang
            fout = gewenst - voorspelling

            // Update laag 2 (embedding → output)
            voor elk gewicht[j, k] in outputGewichten:
                outputGewichten[j, k] = outputGewichten[j, k] + leerSnelheid * fout[k] *
embedding[j]

            // Verdeel de fout terug naar de embedding-laag
            voor elk element j in embedding:
                gradientEmbedding[j] = som over k van (fout[k] * outputGewichten[j, k])

            // Update laag 1 (one-hot → embedding)
            voor elk gewicht[i, j] in embeddingMatrix:
                embeddingMatrix[i, j] = embeddingMatrix[i, j] + leerSnelheid *
gradientEmbedding[j] * input[i]

    geef embeddingMatrix, outputGewichten terug


functie voorspelVolgendeLetter(letter, embeddingMatrix, outputGewichten):
    input = oneHotEncoding(letter)
    embedding = input · embeddingMatrix
    voorspelling = embedding · outputGewichten
    geef letter terug met de hoogste waarde in voorspelling
```

⚠ Wat dit model nog niet doet

Dit model voorspelt enkel op basis van de vórige letter, niet op basis van de volledige context. Er is geen self-attention: elke letter "kijkt" alleen naar zichzelf, niet naar wat ervoor kwam in de zin. Dat is exact de beperking die de volgende sectie oplost.

11.5. *De Transformer en self-attention*

Hoe weet het model dat "bank" in "De bank zit heerlijk" een meubelstuk is, en geen financiële instelling? Het antwoord is de **Transformer-architectuur**: de motor achter elke moderne LLM.

Stel je de Transformer voor als een diep netwerk van tientallen gestapelde lagen. De token-vectoren reizen laag per laag omhoog, en bij elke laag wordt hun representatie rijker: van "los woord" naar "woord in deze specifieke context".

Het belangrijkste ingrediënt is het **self-attention** (zelf-aandacht) mechanisme. Bij het verwerken van een zin "kijkt" elk token naar álle andere tokens in die zin, en bepaalt het per token hoe relevant dat is voor zijn eigen betekenis. In "De bank zit heerlijk" verbindt *self-attention* het token "bank" sterk met "zit" en die verbinding duwt de vector van "bank" richting het meubelstuk-gebied van de ruimte.

Elk token verzamelt zo, met gewogen aandacht, informatie uit zijn hele omgeving. En omdat elk token naar elk ander token kan kijken verliest het model de context van het begin van een lange tekst niet. Dat was exact het probleem waar n-gram modellen op stukliepen.

Van attention scores naar een kansverdeling: softmax

Tijdens de *self-attention* stap berekent het model voor elk paar tokens een ruwe score: hoe sterk horen deze twee samen? Die ruwe scores kunnen elk reëel getal zijn, van sterk negatief tot sterk positief. Daar kan het netwerk niets mee: het wil **gewichten** die zich gedragen als een kansverdeling en samen optellend tot 100%.

Dat regelt de **softmax**-functie. Voor een vector van scores $z_1, \ldots, z_n$:

$$\text{softmax}(z_i) = \frac{e^{z_i}}{\sum_{j=1}^{n} e^{z_j}}$$

Volg de berekening stap voor stap, met de scores (2, 1, 0) als voorbeeld.

Stap 1 — blaas elke score op met de e-macht. Dat maakt elke waarde positief, en grote scores worden extra uitvergroot:

$$e^2 \approx 7.39 \qquad e^1 \approx 2.72 \qquad e^0 = 1.00$$

Stap 2 — tel alle opgeblazen scores samen. Die som gebruik je zo dadelijk om alles te herschalen:

$$7.39 + 2.72 + 1.00 = 11.11$$

Stap 3 — deel elke opgeblazen score door die som. Zo krijgt elke score een aandeel tussen 0 en 1, en samen tellen ze exact op tot 1:

$$\frac{7.39}{11.11} \approx 0.67 \qquad \frac{2.72}{11.11} \approx 0.24 \qquad \frac{1.00}{11.11} \approx 0.09$$

Het resultaat is de gewichtenverdeling (0.67, 0.24, 0.09). Het model kan nu wiskundig stabiel zeggen: "token A richt 67% van zijn aandacht op token B, 24% op token C, en 9% op token D."

11.6. *Pre-training en fine-tuning: eerst lezen, dan opvoeden* —

De kennis die impliciet in een taalmodel zit, komt uit twee heel verschillende trainingsfasen.

Tijdens de **pre-training** leest het model gigantische hoeveelheden internettekst met één simpele opdracht: voorspel telkens het volgende token. Door die taak miljarden keren te oefenen, leert het model als bijproduct de structuur van taal, grammatica, en verrassend veel feiten over de wereld. Niemand voert die feiten expliciet in, ze zitten impliciet in de tekst.

Maar een puur getraind model is geen bruikbare assistent. Het is een tekstaanvuller: geef het een vraag, en het genereert misschien gewoon méér vragen, omdat dat statistisch gezien een plausibel vervolg is. Het heeft ook geen enkele rem op schadelijke of foute output — het imiteert het internet, met alles erop en eraan.

Daarom volgt de **fine-tuning** fase, en die verloopt zelf in twee stappen.

Eerst krijgt het model **supervised fine-tuning** (SFT): het wordt verder getraind op een verzameling zorgvuldig geschreven voorbeelden van goede instructie-antwoord-paren. Zo leert het al een stuk dichter bij "een vraag beantwoorden" komen dan bij "een tekst aanvullen".

Daarna volgt **RLHF** (*reinforcement learning from human feedback*). Het model genereert meerdere mogelijke antwoorden op dezelfde vraag, en menselijke beoordelaars rangschikken die van beste naar slechtste. Met die rangschikkingen train je een apart **reward model**: een netwerk dat leert inschatten hoe goed een antwoord door een mens beoordeeld zou worden. Vervolgens wordt het taalmodel zelf verder getraind, niet meer op vaste voorbeelden, maar door het te belonen wanneer het antwoorden genereert die het reward model hoog inschat — een klassieke *reinforcement learning*-lus, alleen komt de beloning hier onrechtstreeks van menselijke voorkeuren.

Zo verschuift het gedrag van "aanvullen wat statistisch volgt" naar "antwoorden zoals een behulpzame assistent". De onderliggende voorspellingsmachine blijft dezelfde — alleen de knoppen staan anders afgesteld.

> ⚠ **Voorspellen is geen weten**
>
> Ook na *fine-tuning* blijft een LLM een voorspeller van plausibele tekst. Klinkt een fout antwoord plausibel, dan produceert het model dat met evenveel zelfvertrouwen als een juist antwoord. Dit fenomeen heet *hallucinatie*, en het is geen bug die er nog uit moet — het is een direct gevolg van hoe het systeem werkt.

11.7. *Aan de knoppen: temperatuur*

Wanneer het model klaar is met rekenen, produceert de outputlaag een kansverdeling over het volledige vocabularium: voor élk mogelijk volgend token een kans. Welk token uiteindelijk gekozen wordt, bepaal jij mee via de **temperatuur** (*T*). Bij een lage temperatuur kiest het model bijna altijd de meest waarschijnlijke tokens: voorspelbaar en accuraat. Bij een hoge temperatuur krijgen ook minder waarschijnlijke tokens een serieuze kans: creatiever, maar minder betrouwbaar.

11.8. Denken & Doen

11.8.1. Kernconcepten — vat samen in eigen woorden

Probeer elk begrip in één zin uit te leggen zonder terug te bladeren. Check nadien met de theorie.

- N-gram model (unigram, bigram, trigram)
- Tokenisatie en subwords
- Vocabularium
- One-hot encoding
- Embedding en semantische nabijheid
- Cosine similarity (via de hoek tussen twee vectoren)
- Self-attention
- Softmax
- Pre-training versus fine-tuning (SFT en RLHF)
- Hallucinatie
- Temperatuur

11.8.2. Denken

- Een trigram-model kijkt maar naar de twee voorgaande woorden. Waarom wint het Transformer-model dit structureel van een n-gram model, en niet enkel door meer rekenkracht?
- One-hot encoding geeft elke letter een vector die even ver van élke andere letter ligt. Waarom is dat een probleem, en wat lost de embedding-laag daaraan op?
- Waarom meten we de gelijkenis tussen embeddings met de **hoek** tussen vectoren, en niet met de afstand tussen hun eindpunten? Wat "vervuilt" de lengte van een embedding-vector precies?
- Een taalmodel na pre-training is een tekstaanvuller, geen assistent. Wat doen SFT en RLHF elk om dat gedrag te veranderen, en waarom is er geen garantie dat het model daarna nooit meer hallucineert?
- Je bouwt een tool die medische verslagen samenvat, en een tool die reclameteksten verzint. Welke temperatuur past bij elk, en wat is het risico van de verkeerde keuze?

11.8.3. Doen — rekenen met mini-embeddings

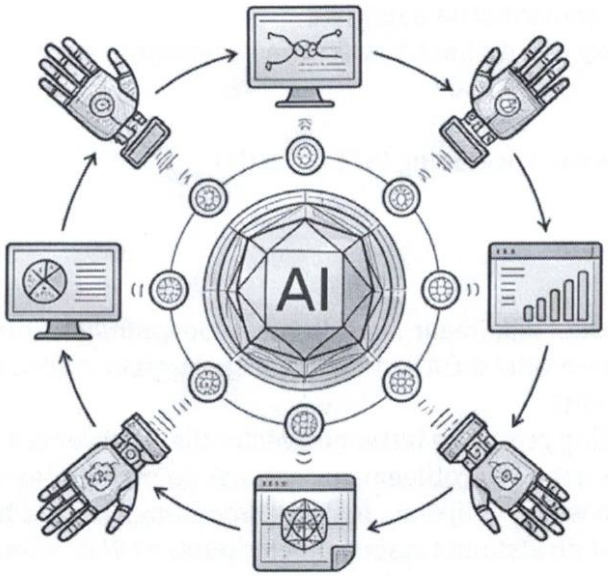
$A = (2, 1)$, $B = (4, 2)$, $C = (-1, 2)$

```
functie cosineSimilarity(a, b):
  product = 0
  normA = 0
  normB = 0
  voor elk dimensie i:
  product = product + a[i] * b[i]
  normA = normA + a[i] * a[i]
  normB = normB + b[i] * b[i]
  geef product / (wortel(normA) * wortel(normB)) terug
```

- Bereken de cosine similarity tussen A en B, en tussen A en C. Welke twee liggen dichter bij elkaar in "betekenis"?
- Drie kandidaat-tokens krijgen ruwe scores (2, 1, 0). Bereken de softmax-gewichten met de formule uit de theorie.
- Herhaal de softmax-berekening, maar deel eerst elke score door $T = 0\{,\}5$ voor je de e-macht neemt. Welk effect van temperatuur zie je terug in je eigen cijfers?

AI-agents

CHAPTER 12



De voorbije hoofdstukken heb je gezien hoe een *Large Language Model* tekst voorspelt, token per token. Indrukwekkend, maar in essentie blijft zo'n model reactief: het wacht op jouw input, genereert een antwoord op basis van patronen in zijn trainingsdata, en stopt daar.

Een **AI-agent** breekt door die passieve grens heen. Wat een agent fundamenteel onderscheidt van een chatbot is **autonomie**, **doelgerichtheid** en het vermogen om zelfstandig actie te ondernemen in een externe omgeving. Waar een chatbot een gesprek voert, lost een agent een probleem op: hij stippelt een workflow uit, roept *tools* aan en delegeert taken, zonder dat jij hem bij elke tussenstap bij de hand moet nemen.

Klinkt dat als magie? Dat is het niet. Onder de motorkap zit nog steeds hetzelfde taalmodel dat je al kent. Het verschil zit in de architectuur die we eromheen bouwen. Dat is precies wat we in dit hoofdstuk doen.

12.1. Van chatbot naar agent

Waar ligt de grens tussen een slimme chatbot en een agent? Een chatbot met toegang tot een kennisbank (RAG) kan al verrassend goede antwoorden geven. Maar hij blijft antwoorden, meer niet.

Definition 12.1. *Een **AI-agent** is een systeem dat een LLM als redeneermotor gebruikt om autonoom een doel te bereiken: het observeert zijn omgeving, plant een strategie, voert acties uit via externe tools, en evalueert het resultaat.*

Drie eigenschappen verschuiven de balans van chatbot naar agent. Ten eerste **autonomie**: de agent beslist zelf welke stappen nodig zijn. Ten tweede **doelgerichtheid**: hij werkt niet per bericht, maar per doel. Ten derde **actie**: hij grijpt in op de buitenwereld, een API aanroepen, een bestand wegschrijven, een order plaatsen.

Een LLM zelf "doet" niets. Het genereert tekst. De agent-architectuur eromheen vertaalt die tekst naar echte acties. Het model is het brein; de architectuur levert de handen.

12.1.1. De Sense→Plan→Act→Learn cyclus

Hoe structureer je dat autonome gedrag? Een agent doorloopt continu een vaste cyclus, die je misschien herkent uit de robotica.

Eerst absorbeert de agent zijn omgeving of de binnenkomende gebruikersvraag (**Sense**). Vervolgens stelt hij zelfstandig een strategie op om het einddoel te bereiken (**Plan**). In de actiefase voert hij de noodzakelijke stappen uit (**Act**), zoals het bevragen van een database of een externe API. Ten slotte evalueert hij het resultaat om zijn aanpak te verfijnen of fouten te corrigeren (**Learn**).

Die laatste stap maakt het verschil tussen een script en een agent. Een script crasht als stap drie mislukt. Een agent merkt op dat het resultaat niet klopt, past zijn plan aan en probeert opnieuw.

> ⚠ **Autonomie is geen betrouwbaarheid**
>
> Een agent die zelfstandig plant, kan ook zelfstandig de mist ingaan. LLM-output is variabel: dezelfde vraag kan tot een ander plan leiden. Bouw daarom altijd controlepunten in, zeker wanneer een agent acties uitvoert met echte gevolgen, zoals een bestelling plaatsen of data verwijderen.

12.2. *Tools en plugins: handen voor het brein*

Wat kan een puur taalmodel eigenlijk uitvoeren? Niets. Het kan alleen tekst genereren. Je kunt het vergelijken met een slim brein zonder handen.

De meeste AI-agent frameworks (zoals **Semantic Kernel**, **Microsoft Agent Framework**, of **LangChain**) lossen dit op dezelfde manier op: je geeft dat brein handen via **plugins** of **tools**. Dat zijn native functies of semantische prompts die één taak uitvoeren: een weersvoorspelling ophalen, een order wegschrijven, een klant opzoeken. De naamgeving verschilt van framework tot framework, maar het onderliggende idee is telkens hetzelfde.

Binnen zo'n framework fungeert een **planner** (of *orchestrator*, afhankelijk van het framework) als dirigent. In plaats van dat jij hardcodeert welke functie wanneer draait, inspecteert de planner de metadata en beschrijvingen van de beschikbare

tools. Op basis van jouw prompt redeneert hij welke tools opeenvolgend nodig zijn en stelt hij dynamisch een uitvoeringsplan op.

```
// WeerPlugin: een groep functies die de AI-agent ter beschikking staan
functie getVoorspelling(stad, datum):
    beschrijving: "Haalt de weersvoorspelling op voor een stad op een bepaalde datum"
    parameter stad: "naam van de stad"
    parameter datum: "datum in formaat jjjj-mm-dd"

    data = roepWeerApiAan(stad, datum)
    geef data terug
```

Merk op hoeveel tekst er in die code staat. Dat is geen toeval. De planner leest geen implementatie; hij leest beschrijvingen. Een vage beschrijving betekent een verkeerde of geen match — dat geldt ongeacht welk framework je gebruikt.

12.2.1. Hoe "kiest" een LLM een functie?

Hier gebeurt iets merkwaardigs: je vertelt het model niet wát het moet doen. Je geeft het een lijst met beschikbare functies, inclusief tekstuele beschrijvingen van hun parameters. Het model beslist vervolgens zelf welke tool past. Dit heet *function calling*, en het is een capaciteit van het onderliggende taalmodel zelf — niet iets dat een specifiek framework toevoegt. Elk framework bouwt enkel de laag eromheen die dat proces structureert.

> **Waarom werkt function calling?**
>
> De intuïtie ligt in de pre-training. Tijdens het trainen op gigantische hoeveelheden code én documentatie heeft het model geleerd hoe natuurlijke taal en codefuncties met elkaar correleren. "Wat voor weer wordt het morgen in Gent?" ligt in de latente ruimte dicht bij beschrijvingen als "haalt de weersvoorspelling op voor een stad".
>
> Het model matcht de intentie in jouw vraag dus statistisch met de semantische beschrijving van elke functie — hetzelfde principe als de gelijkenis tussen *embeddings* die je al kent. Vervolgens genereert het niet het resultaat, maar een gestructureerde JSON-output:
>
> ```
> { "function": "GetVoorspelling",
>   "arguments": { "stad": "Gent", "datum": "2026-07-18" } }
> ```
>
> Jouw applicatie voert die functie daarna zelf en veilig uit. Het model raadt dus geen weer; het "raadt" welke functie het weer kent. Dat verklaart meteen waarom een goede beschrijving cruciaal is: die beschrijving is letterlijk de vector waartegen de gebruikersintentie wordt afgewogen.

12.3. *Geheugen: kort en lang*

Waarom vergeet een agent halverwege een lang gesprek plots je naam? Omdat zijn **kortetermijngeheugen** binnen de *context window* van het LLM leeft. Daarin zitten de lopende chatgeschiedenis en de actuele variabelen van de sessie. Zodra dat venster volraakt, treedt er geheugenverlies op.

De oplossing ken je al uit het hoofdstuk over *embeddings*: je koppelt de agent aan een **langetermijngeheugen** in de vorm van een **vectorstore**. Relevante documenten, historische interacties en feiten over de gebruiker worden via een embedding-model omgezet in numerieke vectoren en permanent opgeslagen. De agent kan er later semantisch in zoeken.

Waarom een vectorstore en nearest-neighbor search?

Een agent kan onmogelijk de volledige bedrijfshistorie in zijn context window proppen. Hij moet dus selectief ophalen wat relevant is — en "relevant" is hier een wiskundig begrip.

De semantische betekenis van een tekst bepaalt de richting van zijn vector in de latente ruimte. Om relevante herinneringen op te halen, berekent de agent de **cosine similarity** tussen de vector van de gebruikersvraag A en elke opgeslagen vector B:

$$\text{cosine similarity} = \frac{A \cdot B}{\|A\| \, \|B\|}$$

De teller is het inproduct: hoe sterk wijzen beide vectoren dezelfde kant op? De noemer deelt de lengtes weg, zodat alleen de richting telt, een lange tekst over koken is niet "relevanter" dan een korte tekst over koken.

Met een *nearest-neighbor search* (zoals k-NN, dat je kent uit het blok over probabilistisch leren) selecteert de vectorstore de k herinneringen met de hoogste score, het dichtst bij 1. Exact dezelfde wiskunde als bij tekstanalyse en *recommendation engines*, maar nu als razendsnel, semantisch opzoeksysteem voor het geheugen van de agent.

Zie je de rode draad? Cosine similarity dook op bij tekstanalyse, bij recommenders, en nu bij agent-geheugen. Eén formule, drie totaal verschillende toepassingen. Dat is wat een goed wiskundig gereedschap doet.

12.4. *De agent als vertaallaag*

Wat doet een agent nu eigenlijk in een applicatie, als je alle onderdelen samenlegt? In essentie fungeert hij als een dynamische **vertaallaag**.

Het proces begint bij mensentaal: de prompt. De agent ontleedt die intentie en vertaalt ze naar een gestructureerde functieaanroep, bijvoorbeeld een JSON-payload voor een API. Hij ontvangt het technische resultaat van dat systeem, en vertaalt die ruwe data ten slotte terug naar een begrijpelijk, vloeiend antwoord in mensentaal.

Example 12.1. Gebruiker: "Wat moet ik vanavond kijken?"

De agent herkent de intentie (filmadvies), stelt vast welke data ontbreekt (wie is de gebruiker?), roept `GetMovieHistory(userId: 42)` aan, krijgt een JSON-array

met film-ID's en scores terug, en formuleert daaruit: "Op basis van je recente sciencefiction-marathon zou ik vanavond *Arrival* proberen."

Merk op dat de gebruiker nooit JSON ziet, en de API nooit mensentaal. De agent staat er precies tussenin in beide richtingen.

12.5. *Multi-agent patronen: wanneer één agent niet volstaat*

Wordt de applicatie complexer, dan kom je al snel uit bij **multi-agent patronen**. In plaats van één agent alles te laten doen, zet je gespecialiseerde teams op.

Bij een **sequentieel patroon** geeft de ene agent zijn output door als input voor de volgende: een assemblagelijn. Bij een **parallel patroon** voeren meerdere agenten tegelijk deeltaken uit. Een **supervisor-patroon** gebruikt één centrale agent die taken verdeelt onder sub-agenten en hun output controleert.

Wanneer is dat gerechtvaardigd? Je gebruikt multi-agent systemen wanneer de logica te complex wordt voor één prompt-context, of wanneer taken strikt gescheiden expertise vereisen — bijvoorbeeld een agent die juridische documenten leest naast een agent die code schrijft.

> ⚠ **Pas op voor overengineering**
>
> Elke extra agent betekent extra latency, extra kosten en extra onvoorspelbaarheid want elke schakel in de keten kan immers "hallucineren". Kan een taak opgelost worden met een duidelijke prompt-chain of één agent met meerdere tools? Doe dat dan. Een multi-agent systeem is een architecturale beslissing, geen statussymbool.

12.6. *Aan de slag*

Voor de ontwerpoefening en het project heb je een minimale, werkende basis nodig. Hieronder het concept in pseudocode — bewust framework-onafhankelijk, want de exacte syntax van tools zoals Semantic Kernel of Microsoft Agent Framework verandert regelmatig. Kijk voor de actuele, werkende schrijfwijze naar de officiële documentatie van het framework dat je zelf kiest.

```
// 1. Maak een kernel/agent aan en verbind met een taalmodel
kernel = maakKernel(model: "gpt-4o-mini", apiKey: JOUW_API_KEY)

// 2. Registreer je eigen plugin(s)/tool(s)
kernel.registreerPlugin(WeerPlugin)

// 3. Laat het model zelf beslissen of en welke functie het nodig heeft
resultaat = kernel.vraag(
  "Wat voor weer wordt het morgen in Gent?",
  functieKeuze: "automatisch" // schakelt function calling in
)

toon resultaat
```

Die laatste instelling — `functieKeuze: "automatisch"` — is de kern: zonder die schakelaar negeert het model je plugins volledig en genereert het gewoon tekst,

zonder ooit een functie aan te roepen. Elk framework heeft hiervoor zijn eigen naamgeving, maar de schakelaar zelf bestaat vrijwel overal.

> **Waar vind je de actuele syntax?**
>
> - **Officiële documentatie**: learn.microsoft.com/semantic-kernel — start bij de *Quick Start Guide* voor de meest actuele, werkende voorbeelden.
> - **Code-voorbeelden**: de GitHub-repository van Semantic Kernel bevat kant-en-klare, up-to-date C#-, Python- en Java-voorbeelden.
> - **Let op**: Microsoft positioneert intussen **Microsoft Agent Framework** (MAF) als opvolger van Semantic Kernel, met extra functionaliteit voor multi-agent orchestratie. Bestaande SK-plugins blijven werken zonder aanpassing — de kernconcepten uit dit hoofdstuk blijven dus overdraagbaar, welk framework je ook uiteindelijk kiest.

12.7. *Denken & Doen*

12.7.1. Kernconcepten — vat samen in eigen woorden

Probeer elk begrip in één zin uit te leggen zonder terug te bladeren. Check nadien met de theorie.

- Autonomie, doelgerichtheid en actie
- Sense → Plan → Act → Learn
- Tools / plugins en de planner (orchestrator)
- Function calling
- Kortetermijngeheugen versus langetermijngeheugen
- Vectorstore en nearest-neighbor search
- De agent als vertaallaag
- Sequentieel, parallel en supervisor-patroon
- Overengineering bij multi-agent systemen

12.7.2. Denken

- Wat is het fundamentele verschil tussen een chatbot met toegang tot een kennisbank (RAG) en een autonome AI-agent? Welke van de drie eigenschappen (autonomie, doelgerichtheid, actie) verschuift die balans het meest?
- Beschrijf hoe de Sense → Plan → Act → Learn-cyclus verloopt bij de vraag "Wat moet ik vanavond kijken?". Wat gebeurt er in elke fase, en waar kan het misgaan?
- De planner van een agent leest geen implementatiecode, enkel de beschrijving van een tool. Waarom is een nauwkeurige beschrijving dan zo cruciaal, en hoe hangt dat samen met wat je weet over embeddings?
- Cosine similarity dook in deze cursus al op bij tekstanalyse en bij recommenders, en nu opnieuw bij het geheugen van een agent. Wat is telkens de "vraag"-vector en wat zijn de "opgeslagen" vectoren, in elk van die drie toepassingen?
- Wanneer is een multi-agent systeem met een supervisor-patroon gerechtvaardigd, en wanneer volstaat één agent met meerdere tools? Neem een standpunt in en verdedig het met de kosten die elke extra agent toevoegt.

12.7.3. Doen — geheugen ophalen uit een vectorstore

$M_1 = (1, 0, 1)$, $M_2 = (0, 1, 1)$, $M_3 = (1, 1, 0)$, vraag $Q = (1, 0, 0)$

```
functie haal_meest_relevante_herinnering(q, geheugen):
  beste = leeg
  hoogste_score = -oneindig
  voor elke m in geheugen:
  score = cosine_similarity(q, m)
  als score > hoogste_score:
  hoogste_score = score
  beste = m
  return beste
```

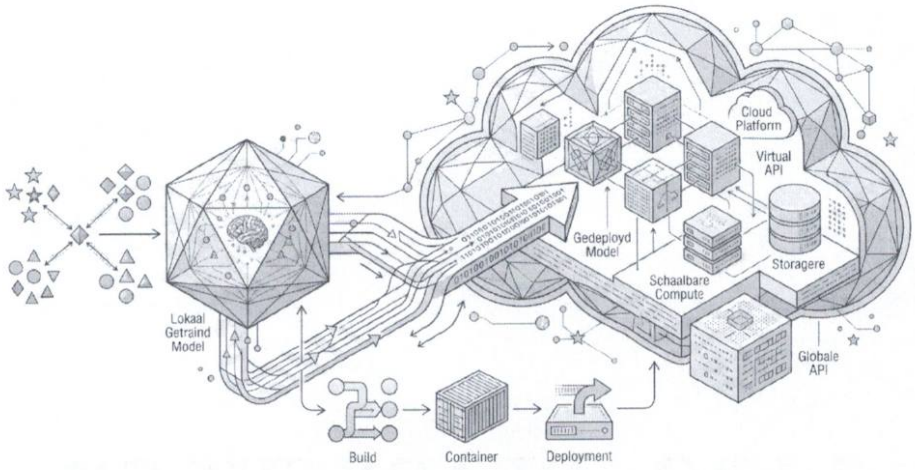
- Bereken de cosine similarity tussen Q en elk van de drie herinneringen. Welke haalt de agent op bij een top-1 zoekopdracht?
- Stel dat alle vectoren vooraf genormaliseerd zijn tot lengte 1: welke term valt dan weg uit de formule, en waarom is dat bij miljoenen entries een groot voordeel?

LABO - READY SET GO

Van model naar cloud

Van model naar cloud

CHAPTER 13



Tot nu toe bouwde je algoritmes die draaien in een console-venster, op je eigen laptop, met data die je zelf klaarzette. Dat is de plek waar elk model begint, maar het is niet waar het eindigt. Een recommendation engine die niemand kan bereiken, beveelt niets aan.

Dit hoofdstuk is een **introductie**, geen volledige handleiding. Het toont het grote plaatje: hoe je van een werkend model naar een bereikbare cloud-applicatie gaat in concepten en richtlijnen, niet in kant-en-klare code om over te typen. De reden daarvoor is bewust: de exacte tools en commando's veranderen snel, en welk platform jij gebruikt is jouw eigen keuze. Dit hoofdstuk leert je **wat** er moet gebeuren en **waarom**; het **hoe**, met de actuele syntax van jouw gekozen tools, leer je in Cloud & Ops zelf.

> **Cloud & Ops is een apart opleidingsonderdeel**
>
> Dit labo bereidt voor op **Cloud & Ops**, een afzonderlijk vak. Dat vak werkt fundamenteel anders dan deze cursus: in plaats van hoorcolleges volg je grotendeels **zelfstudie** via gratis officiële leerpaden (Microsoft Learn, AWS Skill Builder, Google Skills, Docker- en Databricks-documentatie), aangevuld met coachingsmomenten. Je realiseert een semesterbrede projectopdracht: een eigen AI-applicatie, gedeployed in de cloud met tussentijdse milestones, een portfolio, logboek en een afsluitende live demo.

Belangrijk: je kiest zelf je platform (Azure, AWS **of** Google Cloud) en behaalt onderweg een gratis credential (bv. Microsoft Applied Skills of een Google Cloud skill badge) als onderdeel van je portfolio. Pin je dus niet vast op één specifieke tool of aanpak. De vaardigheid om snel je weg te vinden in officiële documentatie is hier minstens even belangrijk als de tool zelf.

13.1. *Van script naar service*

Je recommendation engine uit hoofdstuk 7 en 8 draait vandaag waarschijnlijk als een consoleapplicatie: je start het programma, het leest data in, het print een voorspelling, en het stopt. Dat is prima om een algoritme te **begrijpen**, maar onbruikbaar om het te **gebruiken**. Niemand start jouw laptop op om een filmaanbeveling te krijgen.

De eerste stap is dus: je model omzetten van een script naar een **service** — een applicatie die luistert naar binnenkomende vragen, en telkens een antwoord teruggeeft, zonder dat jij het manueel moet herstarten. In de praktijk betekent dat: een web-API.

Definition 13.1. *Een **web-API** (*Application Programming Interface*) is een applicatie die luistert op een netwerkpoort, en die op een binnenkomend verzoek (een* request*) reageert met een antwoord (een* response*) — meestal in JSON-formaat. Andere applicaties, of gewoon een browser, kunnen die API aanspreken zonder te weten wat er intern gebeurt.*

Conceptueel ziet zo'n minimale recommendation-API er zo uit, ongeacht in welke taal of framework je ze uiteindelijk bouwt:

```
bij opstarten van de applicatie:
    laad het getrainde recommendation-model (één keer, niet per aanvraag)

definieer endpoint "/aanbevelingen/{gebruikerId}":
    ontvang gebruikerId
    aanbevelingen = model.voorspel(gebruikerId, aantal: 5)
    geef aanbevelingen terug als JSON

start de applicatie en blijf luisteren op een poort
```

Vanaf nu is je model een **dienst**: start je deze applicatie, dan blijft ze draaien en beantwoordt ze elk verzoek naar `/aanbevelingen/42` met een lijst voorspellingen voor gebruiker 42. Dat is de vorm die je straks containeriseert en deploy.

> ⚠ **Train niet bij elke aanvraag**
>
> Een veelgemaakte fout: het model trainen binnen de request-handler. Dat betekent dat elke gebruiker moet wachten tot het volledige model opnieuw getraind is, telkens opnieuw. Train je model **één keer** bij het opstarten van de applicatie (of nog beter: offline, en laad enkel de resulterende gewichten in), en gebruik die getrainde staat om razendsnel te voorspellen.

13.2. DevOps: waarom deze stap bestaat

Voor je in containerisatie en cloud-deployment duikt, is het de moeite waard om stil te staan bij **waarom** deze stap in je opleiding zit, en niet toevallig.

Definition 13.2. ***DevOps*** *is een verzameling praktijken die* ***development*** *(het bouwen van software) en* ***operations*** *(het draaiend houden van software) samensmelt. In plaats van dat ontwikkelaars code "over de muur gooien" naar een apart operations-team, is hetzelfde team verantwoordelijk voor code die werkt* ***en*** *die betrouwbaar in productie draait.*

Voor een AI-toepassing is dat niet zomaar een organisatorisch detail. Een model dat perfect werkt op jouw laptop, met jouw geïnstalleerde versies en jouw dataset lokaal op schijf, kan op een andere machine om tien verschillende redenen falen. DevOps-praktijken — automatisering, containerisatie, gecontroleerde deployments — bestaan precies om dat "het werkt op mijn machine"-probleem structureel op te lossen.

> **De kernprincipes in het kort**
>
> - **Automatiseer wat herhaalbaar is.** Bouwen, testen en deployen gebeurt via scripts en pipelines, niet via handmatige stappen die je "wel zal onthouden".
> - **Kleine, frequente wijzigingen.** Liever tien kleine deployments dan één grote — een kleine wijziging is makkelijker te debuggen als er iets misloopt.
> - **Meet en monitor.** Je wil weten of je applicatie draait, hoe ze presteert, en wat ze kost.

13.3. Containerisatie: je applicatie in een doos

Je API werkt lokaal. Maar "lokaal" betekent: op jouw machine, met de versies die jij toevallig geïnstalleerd hebt, met de omgevingsvariabelen die jij toevallig hebt ingesteld. Zodra je die applicatie ergens anders wil laten draaien zoals een cloudserver, de laptop van een medestudent of een productieomgeving loop je het risico dat iets subtiel anders is, en dat je applicatie daardoor faalt.

Docker lost dat op door je applicatie, samen met alles wat ze nodig heeft om te draaien (de juiste runtime, bibliotheken, configuratie), te verpakken in een **container**: een geïsoleerde, draagbare eenheid die overal exact hetzelfde gedraagt.

Definition 13.3. *Een* ***container*** *is een lichtgewicht, geïsoleerde omgeving die een applicatie en al haar dependencies bundelt. In tegenstelling tot een volledige virtuele machine deelt een container de kernel van het besturingssysteem met de host, waardoor ze veel sneller opstart en minder overhead heeft.*

Je definieert een container via een **Dockerfile**: een tekstbestand met stapsgewijze instructies om je applicatie te bouwen. Conceptueel doorloopt zo'n bestand altijd dezelfde logica:

```
DOCKERFILE (conceptueel):

stap 1 – bouwomgeving:
    vertrek van een basisimage met de volledige ontwikkelomgeving
    kopieer je broncode naar binnen
    compileer/bouw de applicatie

stap 2 – eindimage:
    vertrek van een lichtere basisimage met enkel de runtime
    kopieer het gebouwde resultaat uit stap 1 over (niet de volledige broncode)
    stel in hoe de applicatie moet opstarten
```

Deze constructie met twee stappen (*multi-stage build*) is bewust: de bouwstap heeft een volledige ontwikkelomgeving nodig, maar de uiteindelijke container die je deployt bevat enkel de kleinere **runtime** en je gecompileerde applicatie. Kleinere containers starten sneller op en zijn goedkoper om te draaien en op te slaan.

Eens dat bestand klaarstaat, bouw en start je de container met twee commando's.

```
bouw een container-image vanuit je Dockerfile
start die container, en maak een poort bereikbaar vanaf je eigen machine
```

> ⚠ **Eén container, één verantwoordelijkheid**
>
> Een veelgemaakte beginnersfout is alles in één container proppen: de API, de database, een cache. Volg in plaats daarvan het principe van **één proces per container**. Heb je een database nodig? Draai die in een aparte container. Dat maakt elk onderdeel apart vervangbaar, schaalbaar en herstelbaar, exact wat je in een cloudomgeving nodig hebt.

13.4. *CI/CD: automatiseer de weg naar productie*

Elke keer manueel een container bouwen, testen en deployen is foutgevoelig en tijdrovend. **Continuous Integration / Continuous Deployment** (CI/CD) automatiseert die volledige keten.

Definition 13.4. ***Continuous Integration*** *(CI) betekent dat elke codewijziging automatisch gebouwd en getest wordt zodra ze wordt ingediend, zodat fouten meteen aan het licht komen.* ***Continuous Deployment*** *(CD) bouwt daarop verder: een wijziging die alle testen doorstaat, wordt automatisch uitgerold naar een omgeving zonder handmatige tussenkomst.*

Een typische pipeline ziet er conceptueel zo uit, ongeacht welk platform (GitHub Actions, Azure DevOps, GitLab CI, ...) je uiteindelijk gebruikt:

```
PIPELINE: bij elke wijziging op de hoofdbranch

    haal de nieuwste broncode op
    bouw het project
    voer de automatische testen uit

    als alle testen slagen:
        bouw de container-image
        publiceer die image naar een container-registry
        deploy de nieuwe versie naar het cloudplatform
```

```
als een test faalt:
    stop de pipeline en meld de fout
    (er wordt niets gedeployed)
```

Het cruciale idee: een menselijke fout ("ik vergat de testen te draaien", "ik deployde per ongeluk de oude versie") wordt structureel onmogelijk gemaakt, omdat de pipeline elke stap in exact dezelfde volgorde uitvoert, elke keer opnieuw.

> **Waarom dit past bij een AI-applicatie**
>
> Bij een recommendation engine verandert niet enkel je code, maar ook je **model**: je traint het vaak opnieuw met verse data. Een goede CI/CD-pipeline testt niet alleen of je API-code werkt, maar kan ook automatisch controleren of het nieuw getrainde model minstens even goed presteert als het vorige, vóór het live gaat.

13.5. *Deployen naar de cloud*

Je container is gebouwd en getest. De laatste stap is ze effectief laten draaien op een cloudplatform, zodat ze bereikbaar is voor eender wie, niet enkel voor jouw eigen machine.

De grote clouds bieden hiervoor diensten die specifiek gemaakt zijn om **containers** te draaien, zonder dat jij zelf servers moet beheren:

Tabel 1: Vergelijkbare diensten, drie verschillende cloudplatformen. In Cloud & Ops kies je er zelf één, op basis van wat je wil leren of wat je al kent.

Cloud	Dienst voor containers	Typisch gebruikt voor
Azure	Azure Container Apps / App Service	Snel een API deployen zonder cluster-beheer
AWS	Elastic Container Service (ECS) / App Runner	Container draaien met automatische schaling
Google Cloud	Cloud Run	Serverless containers die enkel draaien bij verkeer

Het gemeenschappelijke idee achter alle drie: jij levert een container-image aan (via een **container registry**, zoals Docker Hub of een cloud-eigen registry), en het platform zorgt voor de rest: de server opstarten, verkeer doorsturen, en indien nodig automatisch opschalen bij meer vraag.

> **Vergeet je omgevingsvariabelen niet**
>
> Zaken zoals API-sleutels, connectiestrings naar een database, of configuratie die verschilt tussen lokaal en productie horen **niet** hardcoded in je container.

> Cloudplatformen laten je omgevingsvariabelen instellen buiten de container. Gebruik die in plaats van gevoelige gegevens rechtstreeks in je code te schrijven.

13.6. *Kostenbeheer in de cloud*

Een cloudplatform factureert doorgaans op basis van **gebruik**: hoeveel rekentijd, geheugen en netwerkverkeer je applicatie effectief verbruikt. Dat is aantrekkelijk (je betaalt niet voor een server die stilstaat), maar het betekent ook dat een onoplettend ontworpen applicatie onverwacht duur kan uitvallen.

Een paar concrete aandachtspunten voor een AI-applicatie specifiek:

- **Schaal naar nul wanneer mogelijk.** Diensten zoals Azure Container Apps of Google Cloud Run kunnen je container volledig afsluiten wanneer er geen verkeer is, en automatisch heropstarten bij de eerstvolgende aanvraag. Voor een labo- of demo-applicatie is dat vaak de goedkoopste keuze.
- **Let op de kost van modeltraining versus voorspelling.** Een model opnieuw trainen kost doorgaans veel meer rekenkracht (en dus geld) dan een voorspelling maken met een reeds getraind model. Train niet onnodig vaak.
- **Monitor voor je verrast wordt.** Elk cloudplatform biedt een kostendashboard. Stel een budgetwaarschuwing in, zodat je een melding krijgt vóór de rekening je verrast, niet erna.

> **Project Cloud & Ops**
>
> Deze vijf onderdelen — API, container, CI/CD-pipeline, cloud-deployment en kostenbewaking — zijn precies de bouwstenen van je Cloud & Ops-project. De vier tussentijdse milestones volgen typisch deze volgorde: (1) je recommendation engine werkt als API, (2) ze draait in een container, (3) een pipeline bouwt en test ze automatisch, (4) ze is bereikbaar via een cloudplatform. Het portfolio/logboek documenteert de keuzes die je onderweg maakte — en waarom, inclusief het gratis credential dat je onderweg behaalt.

13.7. *Hoe je jezelf hierin bijschoolt*

Dit hoofdstuk geeft je de concepten en het vocabularium. De concrete vaardigheid, hoe je effectief een Dockerfile schrijft, hoe je een pipeline opzet in het platform van jouw keuze, hoe je exact een container deployt op Azure, AWS of Google Cloud bouw je op via zelfstudie, wekelijkse coaching en de officiële leerpaden die je in Cloud & Ops aangereikt krijgt:

- **Microsoft Learn**, **AWS Skill Builder** en **Google Skills** bieden gratis, gestructureerde leerpaden per platform.
- De officiële **Docker**-documentatie is de meest betrouwbare bron voor containerisatie, ongeacht welk cloudplatform je nadien kiest.

- Je behaalt onderweg een gratis **credential** (bv. Microsoft Applied Skills of een Google Cloud skill badge), neem die mee op in je portfolio want het toont aan dat je je zelfstandig een nieuwe technologie kan eigen maken.

> **Waarom geen kant-en-klare commando's?**
>
> Bewust geen copy-paste-commando's hier: Docker-syntax, cloud-CLI's en pipeline-configuraties veranderen regelmatig, en jij kiest zelf je platform. Wat blijft, zijn de **concepten**: een container bouwt in twee stappen, een pipeline test vóór ze deployt, een cloud-dienst schaalt op gebruik. Die concepten herken je terug, welke tool je ook kiest, en welke versie ook actueel is wanneer jij dit leert.

13.8. *Denken & Doen*

13.8.1. Denken — jouw eigen project

- Welk cloudplatform (Azure, AWS of Google Cloud) spreekt jou het meest aan voor je eigen project, en waarom? Wat wil je daar specifiek uit leren?
- Denk aan je eigen recommendation engine. Waarom is "train niet bij elke aanvraag" voor jouw model een reële valkuil, en hoe zou jij dat vermijden?
- Zou "schalen naar nul" een goede keuze zijn voor jouw applicatie, of net niet? Waarom?
- Wat zou voor jou het lastigste onderdeel zijn om zelfstandig bij te leren: de API, de container, de pipeline, of de cloud-deployment? Waarom net dat onderdeel, en hoe pak je dat aan?

13.8.2. Doen — jouw eigen project op papier

- Schrijf, naar het voorbeeld in de theorie, het endpoint van je eigen recommendation-API in pseudocode: welke naam geef je het, welke parameter(s) ontvangt het, en wat geeft het terug?
- Kies een cloudplatform voor je eigen project en noteer welke dienst voor containers je zou gebruiken (zie de tabel in de theorie), en waarom die bij jouw project past.
- Schets de vier stappen van je eigen CI/CD-pipeline: wat gebeurt er concreet bij elke stap, specifiek voor jouw applicatie?

EINDNOTEN

- Bibliografie
- Nawoord

Bibliografie

CHAPTER 14

14.1. Boeken

Clinton, D. *The complete obsolete guide to generative AI.*

Hermans, F. *The programmer's brain: What every programmer needs to know about cognition.* Manning Publications.

Hurbans, R. *Grokking artificial intelligence algorithms.* Manning Publications.

Hurbans, R. *Grokking AI algorithms* (2de editie). Manning Publications.

Lanham, M. *AI agents in action* (2de editie). Manning Publications.

Russell, S., & Norvig, P. *Artificial intelligence: A modern approach.* Pearson.

Serrano, L. *Grokking machine learning.* Manning Publications.

Symeonidis, P., & Zioupos, A. *Practical recommender systems.* Manning Publications.

Boonen, E., . *Expeditie AI: Keuzes, kansen & kritiek.* VAN IN.

14.2. Overig materiaal

Artificial Intelligence, full course sinds 2020 E. Boonen

von Neumann, J. (1951). Various techniques used in connection with random digits. *National Bureau of Standards Applied Mathematics Series*, 12, 36–38.

Docker. *Docker documentation.* `https://docs.docker.com/`

Google Cloud. *What is artificial intelligence (AI)?*
`https://cloud.google.com/learn/what-is-artificial-intelligence`

IBM. *What is artificial intelligence (AI)?*
`https://www.ibm.com/topics/artificial-intelligence`

Khan Academy. `https://www.khanacademy.org/`

Microsoft. *AI agents for beginners* . Microsoft Learn.
`https://microsoft.github.io/ai-agents-for-beginners/`

Microsoft. *Semantic Kernel GitHub-repository.*
`https://github.com/microsoft/semantic-kernel`

Microsoft Learn. *Semantic Kernel documentatie.*
https://learn.microsoft.com/en-us/semantic-kernel/

Random.org. *True random number service.* https://www.random.org/

Wolfram Alpha. https://www.wolframalpha.com/

Athar, A.Building advanced recommendation systems: Matrix factorization and deep learning. *Medium.*
https://afafathar.medium.com/building-advanced-recommendation-systems-matrix-factorization-and-deep-learning-19dabbc1f4ed

EJable. Deep learning and how it works.
https://www.ejable.com/tech-corner/ai-machine-learning-and-deep-learning/deep-learning-and-how-it-works/

Generative Design Primer. What is a genetic algorithm?
https://www.generativedesign.org/02-deeper-dive/02-04_genetic-algorithms/02-04-01_what-is-a-genetic-algorithm

Kumar, A. Understanding Naive Bayes: A simple guide to text classification. *Medium.*
https://medium.com/@aadityakumar26082004/understanding-naive-bayes-a-simple-guide-to-text-classification-176228fff212

MindManager. Decision tree diagrams.
https://blog.mindmanager.com/decision-tree-diagrams/

Nair, S. *LinkedIn-post over LLM's, n-grams en taalmodellen.*

RMIT University, Learning Lab. Scalar product.
https://learninglab.rmit.edu.au/maths-statistics/linear-algebra/vectors-getting-started/v3-scalar-product/

Sankalp. A beginner's guide to neural networks: Forward and backward propagation explained. *Medium.*
https://medium.com/@xsankalp13/a-beginners-guide-to-neural-networks-forward-and-backward-propagation-explained-a814666c73ab

Singh, A. Linear regression: The cornerstone of predictive analytics. *Medium.*
https://medium.com/@abhaysingh71711/linear-regression-the-cornerstone-of-predictive-analytics-12fd31f094c3

TensorFlow. *Embedding Projector.* https://projector.tensorflow.org/

The Science of ML and AI. Mean squared error.
https://www.ml-science.com/mean-squared-error

Towards AI. Gradient descent and the melody of optimization algorithms. *Medium.*
https://pub.towardsai.net/gradient-descent-and-the-melody-of-optimization-algorithms-244830ea2516

Towards Data Science. User-user collaborative filtering for jokes recommendation.
https://towardsdatascience.com/user-user-collaborative-filtering-for-jokes-recommendation-b6b1e4ec8642/

Tracy, A. R. The minimax algorithm. *Medium*.
https://medium.com/@aidenrtracy/the-minimax-algorithm-f6e8e0a1eadb

Vereniging IJslandse Hond. Genetische diversiteit.
https://verenigingijslandsehond.nl/?page_id=210

Wiskundeleraar. De helling van een functie.
https://www.wiskundeleraar.nl/page3.asp?nummer=11621

Nawoord

CHAPTER 15

Deze cursus is niet zomaar uit het niets ontstaan. Ze is opgebouwd vanuit mijn eigen, oudere lesmateriaal waar ik al jaren aan sleutel, en dat ik afgelopen zomer grondig herwerkt en aangevuld heb tot wat nu voor je ligt.

Bij die herwerking heb ik bewust AI-tools gebruikt, en dat vertel ik je hier gewoon eerlijk. NotebookLM hielp bij het verzamelen en doorzoeken van bronmateriaal. Claude zette ik in om teksten en aanvullingen daadwerkelijk te herschrijven en uit te diepen, en om denk- en doe-vragen te bedenken. Gemini genereerde de openingsprenten bij elk hoofdstuk.

Geen van die tools schreef deze cursus in mijn plaats. Elke tekst is nagelezen, gecorrigeerd en waar nodig herschreven. AI was hier een hulpmiddel, geen auteur. Er zullen ongetwijfeld nog heel wat foutjes ingeslopen zijn, want de tijd ontbrak om hier — zoals eerder bij mijn boek, zie bibliografie ;) — nog veel langer aan te blijven schaven. De deadline van 20 augustus om het manuscript in te dienen dwong me om te gaan voor "goed genoeg". Beschouw deze cursus dus gerust als een *work in progress*: ik werk er dit hele semester met jou in de klas verder aan, en volgend academiejaar krijgt ze een nieuwe, bijgeschaafde versie.

Ik geef je deze cursus bewust niet digitaal. Zo dwing ik je om écht te studeren, zelf tekeningen te maken en aan de slag te gaan want dit zijn geen makkelijke concepten om je hoofd rond te krijgen. Gebruik gerust zelf AI-tools om te begrijpen en te studeren, maar zorg dat je eerst zelf door de materie geworsteld hebt.

Dit alles is trouwens gemaakt in Typst, en I love it ;) Zeker de moeite om zelf eens te checken.

En dan nu: veel plezier dit jaar met deze cursus en met het OPO. We gaan er samen ongetwijfeld een boeiend en fijn jaar van maken!

Tot in de klas!